

MODULE 1: INTRODUCTION

Contents

1. Intelligence	1
2. Artificial Intelligence (AI):	1
3. Key Concepts in AI	2
4. Benefits of AI.....	2
5. Limitations of AI.....	2
6. Classification of AI	2
6.1. Based on Functionality.....	2
6.2. Based on Capabilities.....	3
6.3. Based on Technology	3
6.4. Based on Applications.....	4
7. Agent.....	4
8. Environment	5
9. Key Concepts.....	7
10. PEAS (Performance measure, Environment, Actuators, Sensor)	7
11. Uncertainty in Artificial Intelligence.....	9

1. Intelligence: is the ability to learn, understand, and apply knowledge, solve problems, and adapt to new situations. It encompasses various cognitive abilities like reasoning, problem-solving, abstract thinking, and learning. Types of Intelligence:

- **Natural Intelligence:** This is the intelligence observed in humans and animals, allowing them to process information, learn from experiences, and adapt to their environment.
- **Emotional Intelligence:** The ability to recognize, understand, and manage one's emotions and the emotions of others.
- **Social Intelligence:** The capacity to interact effectively with others and understand social dynamics.
- **Logical-Mathematical Intelligence:** The ability to analyse problems logically, carry out mathematical operations, and investigate issues scientifically.

2. Artificial Intelligence (AI): refers to the simulation of human intelligence in machines that are programmed to think and learn like humans. AI systems are designed to perform tasks that typically require human intelligence, such as visual perception, speech recognition, decision-making, and language translation.

3. Key Concepts in AI

1. **Machine Learning:** A subset of AI that involves training machines to learn from data and improve their performance over time without being explicitly programmed.
2. **Neural Networks:** Computational models inspired by the human brain's network of neurons, used in deep learning to recognize patterns and make decisions.
3. **Natural Language Processing (NLP):** The ability of AI to understand, interpret, and generate human language, allowing for interactions between computers and humans in a natural way.
4. **Robotics:** The branch of AI that involves the design, construction, and operation of robots, which can perform tasks in the real world.
5. **Computer Vision:** The field that enables machines to interpret and understand visual information from the world, like human vision.
6. **Expert Systems:** AI systems that mimic the decision-making abilities of a human expert. They use a knowledge base of facts and rules to solve complex problems.
7. **Fuzzy Logic:** A form of logic used in AI that allows for reasoning with uncertain or imprecise information, like human reasoning.

4. Benefits of AI

1. **Automation:** AI automates repetitive tasks, improving efficiency.
2. **Decision-Making:** Analyses large datasets to inform better decisions.
3. **Personalization:** Delivers tailored experiences based on user behaviour.
4. **Accuracy:** Reduces errors in fields like healthcare and finance.
5. **24/7 Availability:** Operates continuously without breaks.
6. **Cost Savings:** Reduces operational costs by improving efficiency.
7. **Scalability:** Easily handles large volumes of tasks or data.
8. **Innovation:** Aids in generating creative solutions and ideas.
9. **Healthcare:** Enhances early diagnosis and treatment personalization.
10. **Security:** Improves real-time detection of threats like fraud or cyberattacks.

5. Limitations of AI

1. **Bias:** AI can reflect biases present in training data.
2. **Narrow Scope:** AI often struggles to generalize across different domains.
3. **High Costs:** Developing advanced AI systems requires significant investment.
4. **Ethical Concerns:** Raises issues related to surveillance and autonomous systems.
5. **Job Losses:** Automation can displace workers in certain sectors.
6. **Data Dependency:** Performance relies on high-quality data.
7. **Opacity:** Some AI models, like deep learning, lack explainability.
8. **Security Risks:** Vulnerable to adversarial attacks that deceive AI systems.
9. **Limited Creativity:** Lacks genuine human creativity and intuition.
10. **Regulatory Challenges:** Rapid AI development complicates legal frameworks and accountability.

6. Classification of AI

6.1. Based on Functionality:

1. **Narrow AI (Weak AI):** AI systems that are designed and trained to perform a specific task or a narrow range of tasks. These systems do not possess general intelligence and

cannot perform tasks outside their designated scope. **Examples:** Voice assistants like Siri and Alexa, Recommendation systems on Netflix or Amazon, Facial recognition software.

2. **General AI (Strong AI):** A hypothetical AI that possesses the ability to understand, learn, and apply knowledge across a wide range of tasks, similar to human intelligence. General AI can perform any intellectual task that a human can.
3. **Superintelligent AI:** A hypothetical AI that surpasses human intelligence in all aspects, including creativity, problem-solving, and decision-making. This AI would have the capability to outperform humans in virtually every field.

6.2. Based on Capabilities:

1. **Reactive Machines:** The most basic type of AI, these systems can only react to specific inputs with pre-programmed responses. They do not have memory or the ability to learn from past experiences. **Examples:** IBM's Deep Blue chess-playing computer, which could calculate millions of moves to play chess but had no understanding beyond that.
2. **Limited Memory:** AI systems that can use past experiences to inform current decisions, but their memory is not permanent. They can remember and use data for a short period of time to improve their actions. **Examples:** Self-driving cars that use data from the environment to make real-time decisions, like recognizing traffic patterns or obstacles.
3. **Theory of Mind:** An advanced type of AI that, in theory, would be able to understand emotions, beliefs, intentions, and thoughts of other beings. This level of AI would interact with humans in a way that is socially and emotionally aware. **Examples:** This type of AI is still in the research stage and has not been fully developed.
4. **Self-Aware AI:** The most advanced form of AI, which would possess self-awareness and consciousness. This AI would have its own perceptions, desires, and beliefs. This is still theoretical and not realised till date. They would be smarter and more capable than human mind **Examples:** Self-aware AI remains purely speculative and has not been achieved.

6.3. Based on Technology:

1. **Rule-Based AI:** AI systems that operate based on predefined rules and logic. These systems follow explicit instructions given by human programmers. **Examples:** Expert systems, like MYCIN, which was designed to diagnose bacterial infections.
2. **Machine Learning-Based AI:** AI systems that improve their performance over time by learning from data, without being explicitly programmed for every possible scenario.
Subcategories:
 - **Supervised Learning:** Trained on labelled data (e.g., spam detection).
 - **Unsupervised Learning:** Trained on unlabelled data (e.g., clustering algorithms).
 - **Reinforcement Learning:** Learns through trial and error, receiving rewards or penalties based on actions (e.g., AI in video games).
3. **Deep Learning-Based AI:** A subset of machine learning that uses neural networks with many layers (deep networks) to model complex patterns in data. Deep learning is particularly effective in tasks such as image and speech recognition. **Examples:** Google's

AlphaGo, which defeated a world champion in the game of Go, Autonomous vehicles using deep learning for object detection.

4. **Neural Networks:** AI systems modelled after the human brain, using interconnected nodes (neurons) to process information. They are the foundation of deep learning and are used for tasks like pattern recognition and predictive modelling. **Examples:** Image classification models, speech recognition systems, natural language processing.

6.4. Based on Applications:

1. **Artificial Narrow Intelligence (ANI):** AI that specializes in one area, capable of performing a single task extremely well. This is the most common form of AI currently in use. **Examples:** Virtual assistants, chatbots, recommendation engines.
2. **Artificial General Intelligence (AGI):** AI that can understand, learn, and apply intelligence across a wide range of tasks, much like a human. AGI would be capable of performing any intellectual task that a human can. **Examples:** AGI remains theoretical and has not been achieved.
3. **Artificial Superintelligence (ASI):** AI that surpasses human intelligence across all fields. This AI would be capable of outperforming humans in every aspect, from scientific discovery to artistic creativity. **Examples:** ASI is a speculative concept, often discussed in the context of future AI developments and ethical considerations.

7. Agent: An agent is an entity that perceives its environment through sensors and acts upon that environment through actuators. In AI, an agent can be a software program, a robot, or any system that can make decisions to achieve certain goals. **Types of Agents:**

1. **Learning Agents:** Improve their performance over time based on past experiences.
2. **Intelligent Agent:** is an autonomous entity that perceives its environment, makes decisions, and takes actions to achieve specific goals or objectives. It operates based on the principles of AI and is designed to function independently, adapt to changes, and improve its performance through learning and interaction.
3. **Simple Reflex Agents:** act solely based on the current percept, ignoring the history of precepts. They select actions based on a set of condition-action rules, often referred to as "if-then" rules. **Example:** A thermostat that turns the heater on when the temperature drops below a certain point and off when it rises above a threshold.

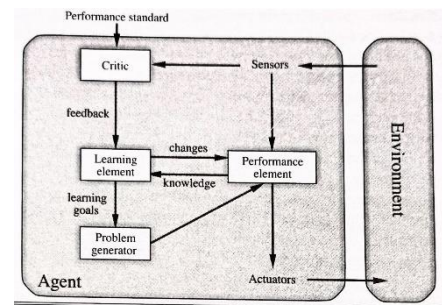


Figure 2.15 A general learning agent.

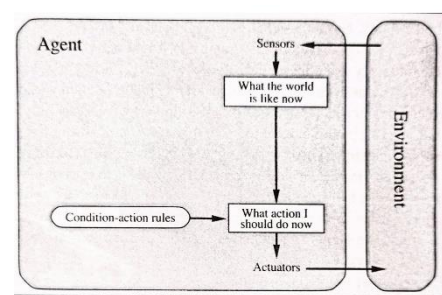


Figure 2.9 Schematic diagram of a simple reflex agent.

4. **Model-Based Reflex Agents:** maintain an internal model of the world, which helps them keep track of the parts of the environment that are not directly observable at the current moment. **Example:** A robot vacuum cleaner that builds a map of the room as it cleans, allowing it to avoid areas it has already cleaned.

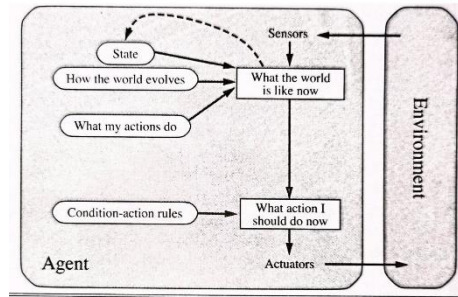


Figure 2.11 A model-based reflex agent.

5. **Goal-Based Agents:** Goal-based agents make decisions based not only on the current situation but also on the goals they are trying to achieve. These agents choose actions that lead them closer to achieving their goals. **Example:** A self-driving car that navigates toward a destination while considering the current.

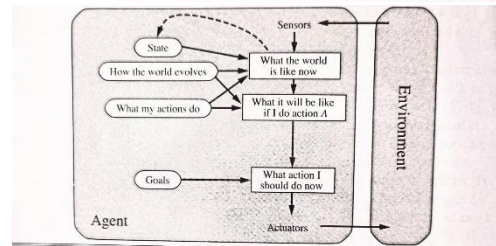


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

6. **Utility-Based Agents:** go beyond goal-based agents by not only aiming to achieve goals but also by considering the best way to achieve those goals. **Example:** An investment AI that not only aims to increase profit (goal) but also considers risk and time horizon to maximize expected returns (utility).

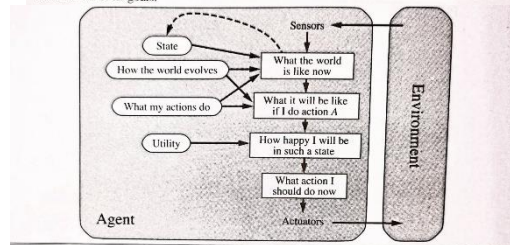


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Feature	Simple Reflex Agents	Model-Based Reflex Agents	Goal-Based Agents	Utility-Based Agents
Basis of Operation	Operate on the current percept only.	Use an internal model to keep track of the world state.	Act to achieve defined goals.	Act to maximize a utility function.
Environment Interaction	React to stimuli with conditioned actions.	Update internal state from current state and action history.	Consider future consequences of actions.	Evaluate success based on a utility function.
Flexibility	Very limited; cannot handle new situations well.	More flexible, can handle unseen scenarios to an extent.	Flexible; can adapt actions to achieve goals in changing environments.	Highly flexible; aims for optimal performance.
Learning Ability	None; they do not learn from past actions.	Limited; can improve the model with new experiences.	Can learn better strategies to achieve goals.	Can adjust utility function based on outcomes.
Example	Thermostat controlling a heating system.	A car with anti-lock braking system.	Chess-playing AI considering future moves.	Investment AI maximizing portfolio return.

8. Environment: The environment refers to the external world in which the agent operates. It includes everything that the agent interacts with or perceives, and it can be anything from a physical space, like a room or a battlefield, to a virtual space, like a game world or the internet.

1. **Fully Observable vs. Partially Observable:** In a fully observable environment, the agent's sensors give it always access to the complete state of the environment. In a partially

observable environment, the agent has only partial information about the state. **Unobservable:** If the agent has no sensors at all then the environment is unobservable.

2. **Deterministic vs. Stochastic:** In a deterministic environment, the next state of the environment is completely determined by the current state and the action taken by the agent. In a stochastic environment, there is some level of unpredictability.
3. **Static vs. Dynamic:** A static environment remains unchanged except by the actions of the agent, while a dynamic environment can change on its own over time.
4. **Discrete vs. Continuous:** A discrete environment has a limited number of distinct states and actions, while a continuous environment has a range of possible states and actions.
5. **Single-Agent Environment:** In a single-agent environment, only one agent exists and interacts with the environment. **Example:** A chess-playing AI where the environment consists only of the chessboard and the agent makes decisions based on it. **Multi-Agent Environment:** In a multi-agent environment, multiple agents coexist and interact with the environment, as well as with each other. **Example:** A market simulation where multiple AI agents are competing to maximize their profit.
6. **Episodic Environment:** In an episodic environment, the agent's actions are divided into separate, independent episodes. Each episode consists of a single interaction with the environment. **Example:** Image classification tasks where the AI model classifies images independently of each other. **Sequential Environment:** In a sequential environment, the current decision can affect future states and decisions. The agent's actions have a long-term impact. **Example:** Playing a game of chess, where each move influences subsequent moves and the eventual outcome.
7. **Known Environment:** In a known environment, the agent has complete information about the environment, including the rules, states, and the effects of its actions. **Example:** A robot navigating a pre-mapped building where it knows the layout and possible obstacles in advance.
8. **Unknown Environment:** In an unknown environment, the agent has incomplete or no prior knowledge of the environment's dynamics. The agent must learn about the environment as it interacts with it. **Example:** A robot exploring an unfamiliar terrain where it must learn the layout and obstacles through exploration.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

9. Key Concepts

1. **Percept:** A percept is the input that an agent receives from its environment at any given time. It is essentially the agent's perception or observation of the environment. **Example:** For a robot, a percept might be the data from its cameras and sensors, such as detecting an obstacle in its path. For a software agent in a chess game, a percept could be the current configuration of the chessboard.
2. **Percept Sequence:** The percept sequence is the complete history of everything the agent has perceived. It includes all percepts that the agent has received from the environment over time, from the initial state to the current state. **Example:** In the case of a robot vacuum cleaner, the percept sequence could include a record of all the obstacles it has encountered and the positions it has cleaned so far.
3. **Agent Program:** The agent program is the software or algorithm that runs on the agent and determines its behaviour. It takes the current percept (and possibly the percept sequence) as input and outputs an action for the agent to take. **Example:** For a chess-playing AI, the agent program might use a decision tree or a search algorithm to determine the best move based on the current board configuration.
4. **Agent Function:** The agent function is a mathematical mapping from the percept sequence to an action. It is an abstract concept that describes what the agent does for any given percept sequence. The agent program is the practical implementation of this function. **Example:** If an agent's task is to navigate a maze, the agent function would define how the agent chooses its actions (e.g., move forward, turn left) based on the sequence of percepts it has received (e.g., walls detected, path ahead clear).

10. PEAS (Performance measure, Environment, Actuators, Sensor)

PEAS is used to describe the components of an intelligent agent's environment and its functioning. It stands for Performance measure, Environment, Actuators, and Sensors. These components are crucial when designing an intelligent agent, as they help in defining the agent's interaction with the world.

1. **Performance Measure:** defines the criteria by which the agent's success is evaluated. It specifies what counts as a successful outcome in the given environment. **Example:** In a self-driving car, the performance measure could be the safety of the passengers, fuel efficiency, or minimizing travel time.
2. **Environment:** is the world in which the agent operates. It consists of everything that surrounds the agent and influences its decisions. **Example:** For a vacuum cleaner robot, the environment would be the rooms, dirt on the floor, and obstacles.
3. **Actuators:** are the mechanisms through which the agent acts upon its environment. They allow the agent to perform actions. **Example:** In a robot, actuators include wheels, arms, or any mechanism used for movement or manipulation of objects.
4. **Sensors:** Sensors allow the agent to **perceive** its environment by collecting data about the state of the world. The data from sensors is used by the agent to make informed decisions. **Example:** For a self-driving car, sensors include cameras, LiDAR, radar, and GPS systems.

Different Agents	Performance Measure	Environment	Actuators	Sensors
Self - Driving Car	Safety, minimizing travel time, fuel efficiency	Roads, other vehicles, pedestrians, traffic lights	Steering, accelerator, brakes	Cameras, radar, GPS, LiDAR
Vacuum Cleaner Robot	Cleanliness of the floor, time taken to clean	The floor, furniture, walls, and dirt	Wheels, vacuum motor, brushes	Dirt sensors, bump sensors, cameras for navigation
Chess AI	Winning the game or maximizing score	The chessboard and the opponent's moves	Moving the pieces on the board	Perceiving the opponent's moves, board state
Smart Thermostat	Maintaining desired room temperature efficiently, energy savings	Indoor room, temperature, time of day, external weather conditions	Heating, ventilation, air conditioning (HVAC) systems	Temperature sensors, humidity sensors, occupancy sensors
Autonomous Drone	Safe navigation, avoiding obstacles, completing tasks	Airspace, obstacles (buildings, trees), weather conditions	Propellers, camera tilt, landing gear	GPS, cameras, altimeters, LiDAR, accelerometers
Speech Recognition System	Accuracy of speech recognition, response relevance, user satisfaction	User's speech, background noise, language context	Voice responses, text display, executing user commands	Microphone, ambient noise detection, voice recognition software
Warehouse Robot/ Part-picking robot	Efficient item picking and placement, reducing errors, timely task completion	Warehouse layout, shelves, items to pick, obstacles	Robot arms for picking items, wheels for movement	Cameras, barcode scanners, proximity sensors, weight sensors
Online Shopping Recommendation System	User engagement, sales conversion, relevance of recommendations	User browsing data, purchase history, product catalog	Displaying product recommendations, emails, notifications	User clicks, searches, purchase data, browsing behavior
Medical Diagnosis System	Accuracy of diagnosis, treatment recommendations, patient outcomes	Patient medical history, symptoms, test results	Displaying diagnosis or treatment options to doctors, suggesting medications	Input from doctors, medical records, patient data (symptoms, tests)
Automated Trading System	Profit maximization,	Stock market data, financial news,	Buying/selling financial assets,	Market data feed, financial analysis tools, news inputs

	risk management, transaction speed.	economic indicators	generating trade signals	
Satellite image analysis system	Correct image categorization.	Downlink from orbiting satellite.	Display of scene categorization.	Color pixel arrays.
Interactive English tutor	Student's score on test.	Set of students, testing agency.	Display of exercise, suggestions, corrections	Keyboard entry

Importance of PEAS

1. **Defines Agent Behaviour:** PEAS helps specify how an AI agent should act in its environment.
2. **Performance Measurement:** Establishes clear criteria to evaluate the agent's success.
3. **Environment Understanding:** Describes the surroundings in which the agent operates.
4. **Action Specification:** Identifies the actions the agent can perform using actuators.
5. **Input Handling:** Clarifies how sensors help the agent perceive the environment.
6. **Task Clarification:** Outlines the task the agent is expected to perform effectively.
7. **Structured Problem Solving:** Provides a systematic way to design intelligent agents.
8. **Adaptability:** Ensures the agent can adjust its actions based on sensor inputs.
9. **Goal Achievement:** PEAS ensures the agent is aligned with achieving its objectives.
10. **Efficiency:** Optimizes how an agent interacts with its environment for better performance.

11. Uncertainty in Artificial Intelligence

Uncertainty in AI refers to the situation where an AI agent cannot predict outcomes with complete accuracy due to incomplete, ambiguous, or noisy information about the environment. It involves the inability to be certain about the state of the world or the results of actions.

Causes of Uncertainty:

1. **Incomplete Information:** The agent does not have access to all relevant data or facts about the environment.
2. **Ambiguity:** The available data might have multiple interpretations (e.g., vague descriptions).
3. **Stochastic (Random) Nature:** The environment may behave unpredictably or involve random events.
4. **Sensor Noise:** Sensors used by the agent to gather data can be faulty or noisy, providing inaccurate information.
5. **Dynamic Environments:** Changes in the environment over time may introduce uncertainty if the agent's knowledge is not updated in real-time.

Sources of Uncertainty:

1. **Perception:** Errors or noise in the data collected by sensors (e.g., vision systems, microphones).
2. **External Factors:** Unpredictable or uncontrollable events in the environment (e.g., weather changes, human intervention).
3. **Inherent Complexity:** The world itself may have elements that are inherently unpredictable, such as market fluctuations or biological processes.
4. **Lack of Knowledge:** The agent may lack critical knowledge about the world or the effects of its actions.
5. **Non-deterministic Actions:** Certain actions may have probabilistic outcomes rather than a guaranteed result (e.g., rolling a dice).

Mechanisms for Handling Uncertainty:

1. **Probability Theory:** Uses probabilistic models to represent uncertainty in knowledge and outcomes. Bayesian networks and probabilistic reasoning are commonly used to update beliefs based on new evidence. **Example:** Predicting weather where the likelihood of rain is 60%, based on historical data and current observations.
2. **Fuzzy Logic:** Fuzzy logic deals with vagueness and ambiguity by allowing degrees of truth rather than binary true/false outcomes. It is useful when the problem involves imprecise or vague data. **Example:** An AI controlling a thermostat can use fuzzy logic to adjust temperature based on subjective terms like “slightly cold” or “very warm.”
3. **Belief Networks (Bayesian Networks):** A belief network is a graphical model that represents the dependencies between variables using probability. It helps in reasoning under uncertainty by updating beliefs when new evidence is observed. **Example:** A medical diagnosis system uses a Bayesian network to update the likelihood of a disease given new test results.
4. **Decision Theory:** Combines probability (expected outcomes) with utility theory(cost/benefit) to make rational decisions under uncertainty. **Example:** In an AI-driven stock trading system, decision theory helps balance risk and reward by selecting trades with the highest expected utility.
5. **Monte Carlo Simulation:** A computational technique that relies on repeated random sampling to estimate the probabilities of different outcomes. It is useful for systems where calculating exact probabilities is complex. **Example:** Monte Carlo methods are used in games like poker to simulate possible future game states and calculate the best move.
6. **Non-probabilistic Methods (e.g., Rule-based Systems):** Some systems handle uncertainty using heuristic or rule-based approaches without relying on probabilities. These systems may use logical rules to infer outcomes based on known conditions. **Example:** An expert system for medical diagnosis may use a set of rules like "If symptom X is present, then disease Y is likely."

MODULE 2: SEARCH TECHNIQUES

Contents

1.	Comparative Study of Search Strategies	2
2	Uniform Search Strategies (Uninformed search)	2
2.1	Breadth-First Search (BFS).....	2
2.2	Depth-First Search (DFS)	3
2.3	Depth-Limited Search (DLS).....	4
2.4	Uniform Cost Search (UCS)	4
2.5	Bidirectional Search	5
2.6	Iterative Deepening Search (IDS)	6
2.7	Comparative Study	7
3	Heuristic Search Strategies.....	7
3.1	Best-First Search	8
3.2	Beam Search	9
3.3	A* Search.....	10
3.4	AO* Search (And-Or Graph Search).....	11
3.5	Memory-Bounded Heuristic Search	12
3.5.1	Hill-Climbing Search.....	13
3.5.2	Simulated Annealing Search.....	13
3.5.3	Local Beam Search Genetic Algorithm.....	14
3.6	Constraint Satisfaction Problems (CSPs).....	14
3.7	Local Search for CSPs.....	15
3.8	Comparative study.....	15
4	Adversarial Search	16
4.1	Games	16
4.2	Optimal Decisions & Strategies in Games.....	16

1. Comparative Study of Search Strategies

Criterion	Uniform Search	Heuristic Search	Adversarial Search
Information Used	No domain-specific knowledge	Uses domain-specific heuristics	Considers opponent's moves and strategies
Efficiency	Inefficient in large spaces (BFS, DFS)	More efficient if heuristics are good (A*)	Depends on tree size; alpha-beta improves speed
Optimality	BFS, UCS are optimal in specific conditions	A* is optimal with admissible heuristics	Minimax is optimal in zero-sum games
Completeness	BFS, UCS are complete	A*, Greedy Best-First can be complete (A*)	Minimax with alpha-beta pruning is complete
Memory Use	BFS is memory-heavy, DFS is memory-efficient	A* can be memory-intensive	Memory complexity depends on depth (DFS-based)
Examples of Use	Maze solving, pathfinding (UCS)	GPS navigation (A*), optimization (Sim. Anneal)	Chess, Go, other competitive games

2 Uniform Search Strategies (Uninformed search)

These strategies search the problem space without any additional information or heuristics about the goal location. They explore all possible paths systematically but do not guide the search based on any domain-specific information.

2.1 Breadth-First Search (BFS)

BFS is an uninformed search strategy that explores all nodes at the current depth level before moving on to the next level. It searches layer by layer, expanding all nodes at a given depth before going deeper.

- **Complete:** Yes, BFS will always find a solution if one exists.
- **Optimal:** Yes, if all edges or path costs are equal, BFS will find the shallowest (least cost) solution.
- **Time Complexity:** $O(b^d)$, where b is the branching factor (average number of successors per node) and d is the depth of the shallowest solution.
- **Space Complexity:** $O(b^d)$, BFS stores all nodes at the current level, so it uses a large amount of memory.

Advantages:

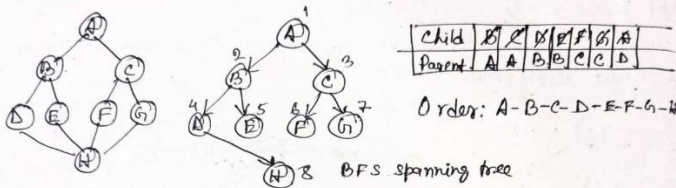
- Guarantees finding the optimal solution when all actions have the same cost.
- Complete and systematic, exploring the entire state space level by level.

Disadvantages:

- Memory-intensive due to the large number of nodes kept in memory, making it impractical for large depths.
- Slow for problems with high branching factors and deep solutions.

② BFS

① FIFO-BFS (this is default)



If edge costs = unit, then BFS-ST solves SSSP (Right?) problem in the most optimal way

TC of DFS & BFS depends on repr. of graph

(i) Adj. Matrix = $O(n^2)$

(ii) Adj. List = $O(n+e)$

SC = $O(n)$

2.2 Depth-First Search (DFS)

DFS is an uninformed search strategy that explores as far as possible along a branch before backtracking. It delves deep into the search tree first and only explores alternative branches when it reaches a dead end (leaf node) or the goal.

- **Optimal:** No, DFS does not guarantee finding the least cost or shallowest solution.
- **Complete:** No, DFS may fail to find a solution if it goes down an infinite path or gets stuck in a loop (unless cycles are handled or the state space is finite).
- **Time Complexity:** $O(b^m)$, where m is the maximum depth of the search tree.
- **Space Complexity:** $O(bm)$, which is much more memory-efficient than BFS as DFS only stores nodes on the current path.

Advantages:

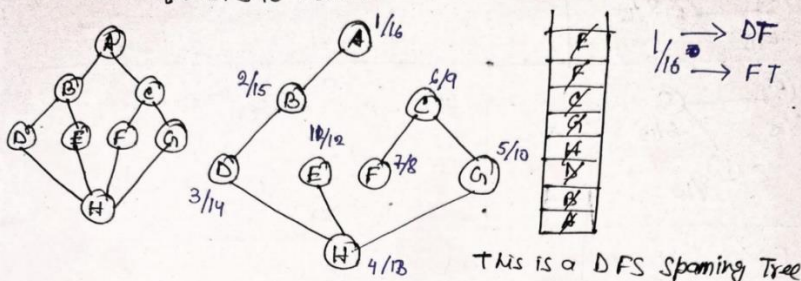
- Memory-efficient compared to BFS.
- Can find solutions quickly if they exist in deep branches.

Disadvantages:

- Not guaranteed to find the shallowest or optimal solution.
- Can get stuck in infinite loops unless precautions like cycle checking are implemented.

① DFS in undirected connected graph

Condition for DFS to come to halt is Stack becomes empty



Eg: which of the following orders are valid

- (a) A, B, E, H, D, F, C, G ✓
- (b) A, C, F, E, H, G, B, D ✗
- (c) H, D, B, E, A, C, F, G ✓
- (d) H, F, C, A, D, F, G ✗
- (e) G, C, F, H, E, B, D, A ✓

2.3 Depth-Limited Search (DLS)

DLS is a variation of DFS that imposes a limit on the depth to which the algorithm can search. It explores all nodes up to a predetermined depth and stops once it reaches this limit.

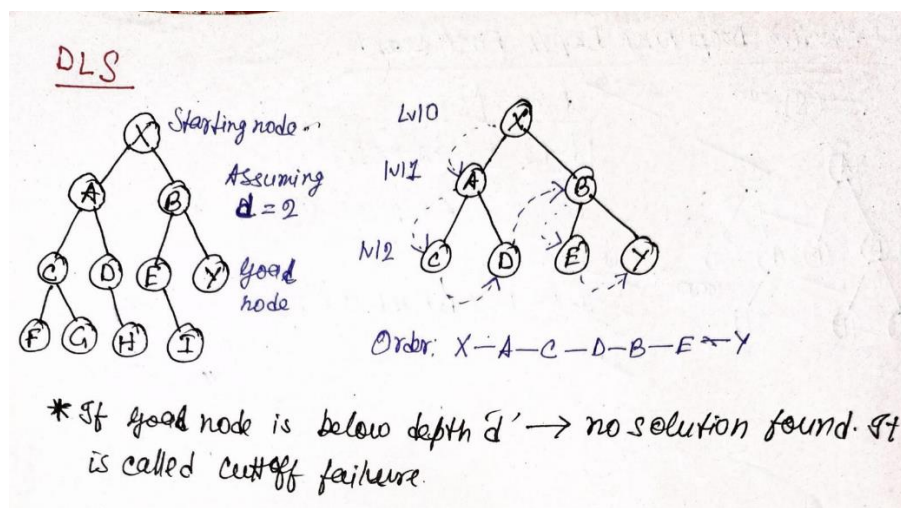
- **Complete:** Yes, if the depth limit l is greater than or equal to the solution depth.
- **Optimal:** No, like DFS, it is not guaranteed to find the shallowest or least-cost solution.
- **Time Complexity:** $O(b^l)$, where l is the depth limit.
- **Space Complexity:** $O(bl)$, making it more memory-efficient than BFS.

Advantages:

- Prevents DFS from going down infinite paths or getting stuck in cycles.
- More memory-efficient and suitable for deep search trees where the solution is expected within a certain depth.

Disadvantages:

- Requires a good estimate of the depth limit to avoid missing solutions.
- If the solution is deeper than the limit, it will not be found.
- Not optimal since it does not guarantee the least-cost solution.



2.4 Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is a search algorithm that expands the node with the lowest cumulative cost from the start node, instead of prioritizing the depth of the node or using a heuristic. UCS is essentially a variant of Dijkstra's algorithm and is used to find the least-cost path in a weighted graph where all edge costs may not be uniform.

Properties of Uniform Cost Search

- **Completeness:** UCS is complete, meaning it is guaranteed to find a solution if one exists.
- **Optimality:** UCS is optimal, meaning it finds the least-cost path to the goal.
- **Time Complexity:** The time complexity of UCS is $O(b^{\lceil C^*/\epsilon \rceil})$, where:
 - B is the branching factor (the number of successors each node has).
 - C^* is the cost of the optimal solution.
 - ϵ is the minimum cost of any action (positive).

This means that UCS can take a long time if the solution involves many small-cost actions or if there are many nodes with similar costs.

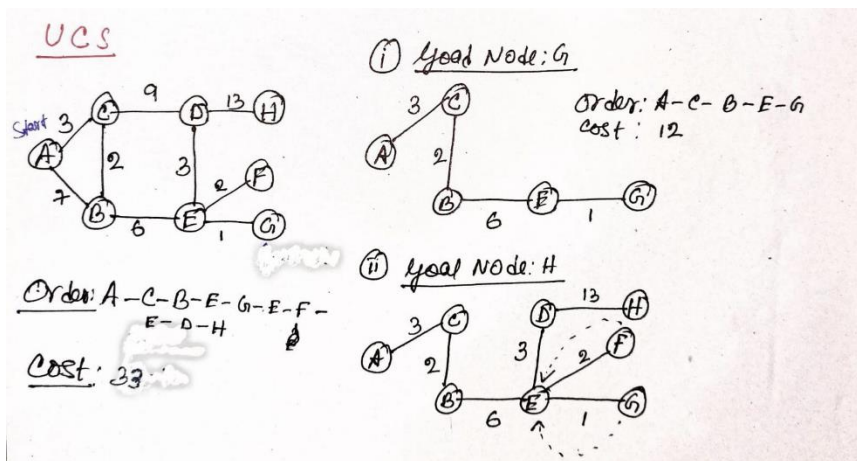
- **Space Complexity:** The space complexity is the same as time complexity, $O(b^{\lceil C^*/\epsilon \rceil})$, as UCS stores all the explored nodes in memory.

Advantages of Uniform Cost Search

- **Guarantees Optimality:** UCS guarantees to find the least-cost path to the goal, making it ideal for applications where finding the optimal solution is critical.
- **Applicable to Weighted Graphs:** Unlike simpler algorithms like breadth-first search (BFS), UCS works with weighted graphs where the cost of moving between nodes is not uniform.
- **No Heuristics Required:** UCS does not rely on heuristics, making it more straightforward to implement in situations where heuristic information is unavailable or unreliable.

Disadvantages of Uniform Cost Search

- **Time-Consuming:** UCS can be slow because it expands nodes in increasing cost order, which may lead to the exploration of many unnecessary nodes before reaching the goal.
- **High Memory Usage:** UCS stores all nodes in memory, including those with very low or negligible chances of contributing to the final solution.
- **Sensitive to Small Costs:** If the minimum edge cost ϵ is very small, UCS can take a long time to terminate, as it might explore many small-cost nodes before finding the goal.



2.5 Bidirectional Search

Bidirectional search is an uninformed search strategy that simultaneously searches forward from the initial state and backward from the goal, stopping when the two searches meet in the middle. Any search technique can be used here.

Properties:

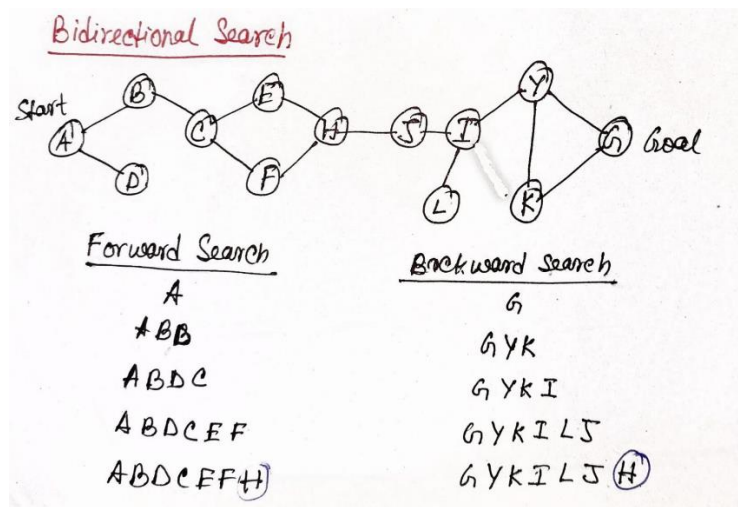
- **Complete:** Yes, if a solution exists, bidirectional search will find it.
- **Optimal:** Yes, if both searches use BFS and the path costs are uniform.
- **Time Complexity:** $O(b^{d/2})$, because each search covers half the depth, significantly reducing the time required compared to BFS or DFS.
- **Space Complexity:** $O(b^{d/2})$, making it more memory-efficient than BFS.

Advantages:

- Drastically reduces the time and memory needed by cutting the search space in half.
- Works well for large state spaces where both the start and goal are known.

Disadvantages:

- Requires a clear goal state, which might not always be feasible.
- Implementing a backward search can be challenging for some problem types.
- Memory requirements are still significant, though better than BFS.



2.6 Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) is a search strategy that combines the space efficiency of **Depth-First Search (DFS)** with the completeness and optimality of **Breadth-First Search (BFS)**. IDS repeatedly applies DFS with increasing depth limits, gradually expanding the search boundary until the solution is found.

Properties of Iterative Deepening Search

- **Completeness:** IDS is **complete**, meaning that if a solution exists, it is guaranteed to find it.
- **Optimality:** IDS is **optimal** when the cost of all steps (actions) is equal.
- **Time Complexity:** For a branching factor b and depth d , the time complexity is $O(b^d)$.
- **Space Complexity:** IDS uses **very little memory**, with a space complexity of $O(d)$, where d is the depth of the search.

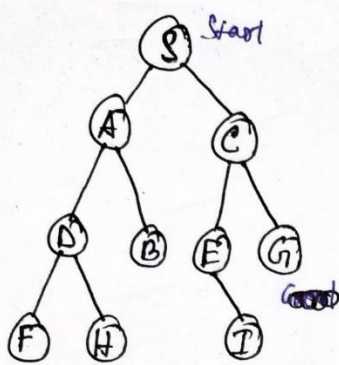
Advantages of Iterative Deepening Search

- **Memory Efficiency:** IDS uses the memory efficiency of DFS.
- **Completeness and Optimality:** IDS gives the **completeness** and **optimality** of BFS.
- **Simple Implementation:** IDS is relatively easy to implement.
- **Handles Infinite Depths:** IDS can handle search spaces that have infinite depth.

Disadvantages of Iterative Deepening Search

- **Repetition of Work:** A common criticism of IDS is that it repeatedly expands nodes at shallower depths.
- **Not Suitable for High-Cost Problems:** IDS works well when all edges or steps have equal cost.

Iterative Deepening Depth First Search



Goal = G

$d=0$ [S]

$d=1$ [S → A → C]

$d=2$: [S A D B C E G]

Goal = I

$d=3$: [S A D F H B C E I]

2.7 Comparative Study

Criterion	BFS	DFS	DLS	Bidirectional
Search Approach	Explores level by level (breadth-wise)	Explores as deep as possible (depth-wise)	Depth-first with a limit	Simultaneous forward and backward search
Completeness	Yes	No, unless space is finite	Yes, if the limit is sufficiently deep	Yes, if the two searches meet
Optimality	Yes, if all steps have equal cost	No	No	Yes, if using BFS with uniform costs
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^{d/2})$
Memory Usage	High	Low	Low	Moderate, but less than BFS
Depth Limitation	No limit	No limit	Yes, imposed limit	No depth limit
Best for	Shallow solutions with equal costs	Deep solutions with minimal memory use	Deep searches with known depth limit	Large state spaces with known goals
Examples	Shortest path in a maze	Puzzle-solving with deep solutions	Problems with known depth constraints	Shortest path search with clear goals

3 Heuristic Search Strategies

Heuristic search algorithms use problem-specific knowledge (heuristics) to guide the search towards the goal more efficiently than uniform strategies.

- They are goal-directed (info about the goal state is present)

- Aiming to reduce the search space significantly by estimating how close a node is to the goal.
- Finds optimal solution to reach goal state using a heuristic function $F(n) = G(n) + H(n)$ [$G(n)$: Path cost, $H(n)$ Heuristic value]

Type of Heuristic

1. **Admissible Heuristics:** A heuristic is **admissible** if it **never overestimates** the true cost of reaching the goal from any given node. If $h(n)$ be the heuristic function and $h^*(n)$ be the true cost from node n to the goal. The heuristic is admissible if: $h(n) \leq h^*(n)$.
2. **Non-Admissible Heuristics:** A heuristic is **non-admissible** if it **can overestimate** the true cost of reaching the goal. If $h(n)$ be the heuristic function and $h^*(n)$ be the true cost from node n to the goal. The heuristic is admissible if: $h(n) > h^*(n)$.

Criteria	Admissible Heuristics	Non-Admissible Heuristics
Definition	Never overestimates the cost to reach the goal.	Can overestimate the cost to reach the goal.
Optimality	Guarantees finding the optimal solution (e.g., with A*).	May not find the optimal solution.
Performance	Often slower due to exploring all equally promising nodes.	Can be faster but risks missing the optimal solution.
Examples	Straight-line distance (Euclidean distance), Manhattan distance.	Overestimated distance, inaccurate heuristics.
Search Behaviour	Thorough, explores all possible optimal paths.	Can quickly explore paths but may lead to suboptimal solutions.

3.1 Best-First Search

Greedy Best-First Search is an informed search strategy that expands the node closest to the goal, according to a heuristic function $h(n)$ which estimates the cost to reach the goal from a given node. It "greedily" picks the node that appears to be closest to the goal based on this heuristic.

Properties:

- **Complete:** No, it can get stuck in loops or local optima if the heuristic is poorly defined.
- **Optimal:** No, because it does not account for the cost of reaching a node, only the estimated cost to the goal.
- **Uses priority Queue:** to store cost of nodes
- **Time/Space Complexity:** Depends heavily on the quality of the heuristic, but generally $O(b^m)$ where b is the branching factor and m is the maximum depth.

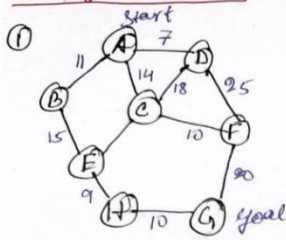
Advantages:

- Can be faster than uninformed search algorithms when a good heuristic is available.
- Uses a heuristic to focus the search on promising paths.

Disadvantages:

- Not guaranteed to find the optimal solution.
- Can perform poorly in large or complex search spaces if the heuristic is inaccurate.

Best first Search

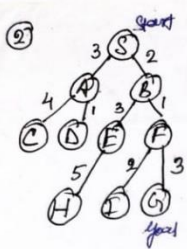


Straight line distances (heuristics)

$A \rightarrow G = 40$	$G \rightarrow G = 0$
$B \rightarrow G = 32$	$H \rightarrow G = 10$
$C \rightarrow G = 25$	
$D \rightarrow G = 35$	
$E \rightarrow G = 19$	
$F \rightarrow G = 17$	

open	closed	Description
[A]	[]	A is read and other nodes adjacent to A are put in open in ascending order of open heuristics.
[C, B, D] 25 32 35	[A]	C is read & same
[F, E, B, D]	[A, C]	F is read & same.
[G, E, B, D]	[A, C, F]	G and same.
[H, E, B, D]	[A, C, F, G]	Stop as we got goal G.

Path: $A \xrightarrow{14} C \xrightarrow{10} F \xrightarrow{20} G$ Cost = 44



$A \rightarrow 12$	open	closed
$B \rightarrow 4$	[S]	[S]
$C \rightarrow 3$	[B, A]	[S]
$D \rightarrow 3$	[F, E, A]	[S, B]
$E \rightarrow 8$	[G, I, E, A]	[S, B, F]
$F \rightarrow 2$	[I, E, A]	[S, B, F, G]
$G \rightarrow 0$		
$H \rightarrow 4$		
$I \rightarrow 19$		
$S \rightarrow 13$		

Path: $S \rightarrow B \rightarrow F \rightarrow G$ Cost: 6

3.2 Beam Search

Beam Search is a **heuristic search algorithm** that explores a graph by expanding the most promising nodes, but it restricts the memory and time usage by limiting the number of nodes it keeps at each level. Beam search works like **Breadth-First Search (BFS)**, but instead of exploring all successors of a node, it keeps only a fixed number (called the beam width) of the best candidates based on a heuristic function.

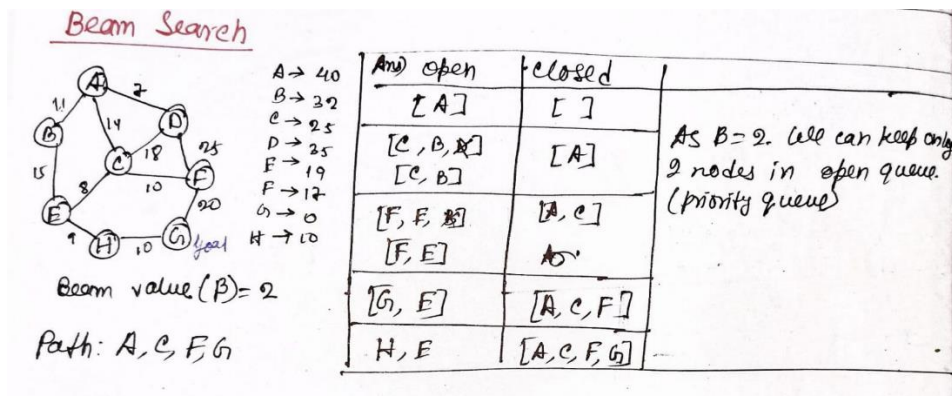
- **Completeness:** Beam search is **not complete**, as it can discard paths that may lead to the goal due to the beam width restriction.
- **Optimality:** Beam search is **not optimal** because it may discard better paths early in the search due to the beam width constraint.

Advantages:

- **Memory Efficient:** The beam width limits the number of nodes in memory, making it more memory-efficient compared to BFS.
- **Speed:** By discarding fewer promising paths early, it often runs faster than exhaustive search algorithms.

Disadvantages:

- **Risk of Missing Solutions:** Beam search might discard promising paths and fail to find the solution if the beam width is too narrow.
- **Non-Optimality:** The algorithm does not guarantee the optimal solution due to the discarding of potential paths.



3.3 A* Search

A* is an informed search algorithm that combines both the path cost $g(n)$ and the heuristic estimate $h(n)$ to select the node with the lowest total cost, using the evaluation function $f(n)=g(n)+h(n)$.

- $g(n)$: Cost of the path from the start node to the current node.
- $h(n)$: Estimated cost from the current node to the goal (heuristic value).

Properties:

- **Complete:** Yes, if the heuristic is admissible (i.e., it never overestimates the true cost).
- **Optimal:** Yes, if the heuristic is admissible and consistent (monotonic).
- **Time/Space Complexity:** $O(b^d)$, but can be reduced with a good heuristic. Memory usage is often the limiting factor because it stores all generated nodes.

Advantages:

- Guaranteed to find the optimal solution when using an admissible heuristic.
- Combines the benefits of path cost and heuristic information.

Disadvantages:

- Can be memory-intensive, especially in large search spaces.
- Its performance depends heavily on the quality of the heuristic function.

A* Search

①

$S(7)$
 $A(6)$
 $B(2)$
 $C(1)$
 $D(0)$

$S \rightarrow A = 1 + 6 = 7$
 $S \rightarrow B = 4 + 2 = 6$ } $S \rightarrow B$ chosen
 $S \rightarrow B \rightarrow C = (4 + 2) + 1 = 7$
 $S \rightarrow B \rightarrow C \rightarrow D = (4 + 2 + 3) + 0 = 9$
 $S \rightarrow A \rightarrow B = (1 + 6) + 2 = 9$ } for $S \rightarrow A$ we see
 $S \rightarrow A \rightarrow C = (1 + 6) + 1 = 8$
 $S \rightarrow A \rightarrow D = (1 + 12) + 0 = 13$

$S \rightarrow A \rightarrow B \rightarrow C = (1 + 6 + 2) + 1 = 10$
 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = (1 + 6 + 2 + 3) + 0 = 12$ } for $S \rightarrow A \rightarrow B$
 $S \rightarrow A \rightarrow C \rightarrow D = (1 + 6 + 1) + 0 = 8$ } for $S \rightarrow A \rightarrow C$

Among the following 4 solutions we choose the least one.
 So $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$. Cost = 8

②

$S(5)$
 $A(3)$
 $B(4)$
 $C(2)$
 $D(6)$
 $E(0)$
 $G(0)$

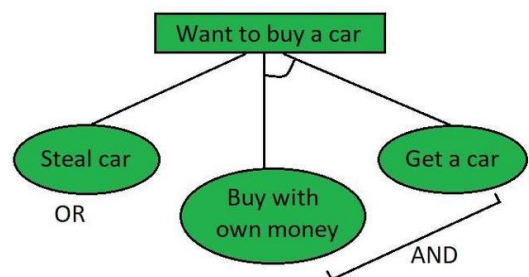
$S \rightarrow A = 1 + 3 = 4$
 $S \rightarrow B = 2 + 4 = 6$
 $S \rightarrow A \rightarrow B = 3 + 4 = 7$
 $S \rightarrow A \rightarrow C = 2 + 2 = 4$
 $S \rightarrow A \rightarrow C \rightarrow D = 5 + 6 = 11$
 $S \rightarrow A \rightarrow C \rightarrow E = 6 + 0 = 6$

$S \rightarrow A \rightarrow C \rightarrow D \rightarrow E = 7 + 0 = 7$
 $S \rightarrow A \rightarrow B \rightarrow D = 8 + 6 = 14$
 $S \rightarrow A \rightarrow B \rightarrow D \rightarrow E = 10 + 0 = 10$

Ans: $S \rightarrow A \rightarrow C \rightarrow E$
 Cost = 6

3.4 AO* Search (And-Or Graph Search)

AO* (And-Or) Search is a **graph-based search algorithm** used for solving problems with **AND/OR graphs**, where some parts of the problem require satisfying multiple conditions (AND) and others may only need one of several possibilities (OR). AO* is designed to handle problems where subproblems may have dependencies, and the solution may involve a combination of different paths.



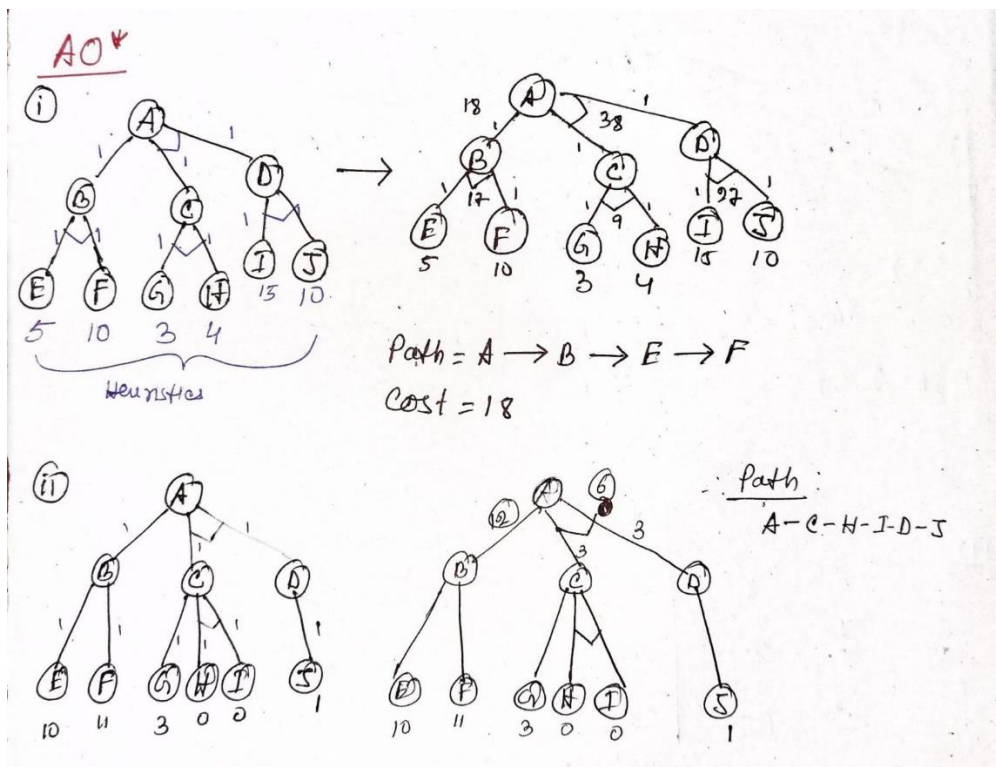
- **Completeness:** AO* is **complete** and can find a solution if one exists.
- **Optimality:** It is **optimal** when using admissible heuristics, as it will find the least-cost solution in an AND/OR graph.

Advantages:

- **Handles AND/OR Dependencies:** It can efficiently handle problems with dependencies between subproblems (AND nodes).
- **Optimal:** With an admissible heuristic, AO* is guaranteed to find the best solution.

Disadvantages:

- **Complex to Implement:** AND/OR graphs are more complex than traditional OR-only graphs, and implementing AO* requires careful handling of the dependencies.
- **Memory-Intensive:** Keeping track of multiple paths and their dependencies can be memory-intensive.



3.5 Memory-Bounded Heuristic Search

Memory-bounded heuristic search algorithms are designed to reduce the memory requirements of algorithms like A*. They provide solutions in memory-constrained environments.

Types of Memory-Bounded Heuristic Searches:

1. **Iterative Deepening A* (IDA*):** A memory-efficient variant of A* that uses depth-first search combined with an iterative deepening approach.
 - **How it works:** Performs a series of depth-first searches, each with a limit on the total cost $f(n)$. The cost limit is increased iteratively.
 - **Memory Usage:** Significantly lower than A* since it does not store all nodes.
 - **Time Complexity:** Still exponential, but more memory-efficient.
2. **Simplified Memory-Bounded A* (SMA*):** A memory-bounded version of A* that discards nodes when memory limits are reached, keeping only the most promising nodes.
 - **How it works:** When the memory limit is exceeded, SMA* prunes the least promising paths to free up space for better candidates.
 - **Memory Usage:** Bounded by a preset limit, making it more efficient than A*.

Advantages:

- **Memory Efficiency:** These algorithms reduce the memory usage compared to traditional A*, making them more scalable for large problems.
- **Optimality:** SMA* retains optimality under memory constraints (if the heuristic is admissible).

Disadvantages:

- **Increased Time Complexity:** May require more iterations or time to find a solution due to the trade-off with memory.
- **Loss of Information:** Discarded nodes may lead to re-exploring the same paths later.

3.5.1 Hill-Climbing Search

Hill-Climbing is a local search algorithm that continuously moves toward the direction of increasing value (i.e., a higher score or fitness), aiming to reach a peak (maximum value). It is a greedy approach with no backtracking.

Properties:

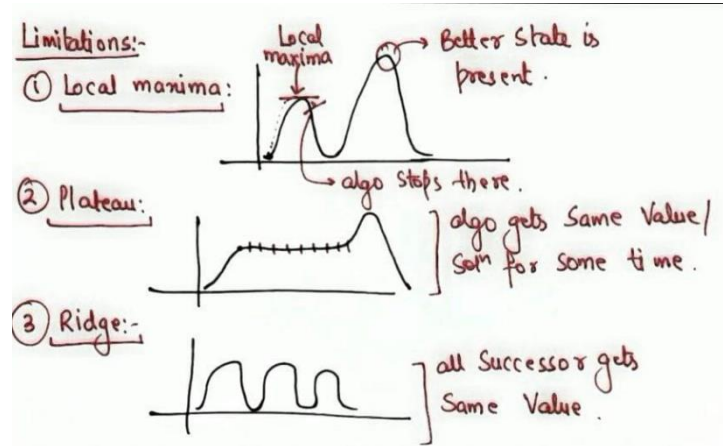
- **Complete:** No, it can get stuck in local optima.
- **Optimal:** No, unless the problem landscape is unimodal (only one peak).
- **Time Complexity:** $O(n)$, where n is the number of neighbours evaluated.

Advantages:

- Simple and easy to implement.
- Efficient for problems where a good initial solution is close to the optimal solution.

Disadvantages:

- **Local Maxima:** A local maximum is a point in the search space where all neighbouring states have a lower value (or higher cost), but it is not the highest point in the entire search space (the global maximum). The search algorithm stops at a local maximum because it appears to be the best solution in the immediate vicinity.



- **Plateau:** A plateau is a flat area of the search space where neighbouring states have the same value. On a plateau, the search may stall, as Hill Climbing relies on detecting an improvement in value to move forward. Without such improvements, the algorithm may stop making progress.

- **Ridges:** are sequence of local maxima that is very difficult for greedy algorithms to navigate. Hill Climbing may "slide off" the ridge and miss the best path. The narrow and difficult-to-navigate nature of ridges makes it challenging for the algorithm to find the right route to the global maximum.

3.5.2 Simulated Annealing Search

Simulated Annealing (SA) is a probabilistic technique for approximating the global optimum of a given function. It mimics the process of annealing in metallurgy, where materials are heated and then cooled to make them reach a state of low energy where they are very strong.



Properties:

- **Complete:** Yes, if given enough time and appropriate cooling.
- **Optimal:** It can approximate the global optimum, but not guaranteed.
- **Time Complexity:** Typically, $O(n)$, but varies with the cooling schedule.

Advantages:

- Can escape local optima by allowing worse solutions at higher temperatures.
- Effective for large search spaces with complex landscapes.

Disadvantages:

- Performance heavily depends on the cooling schedule and parameters.
- Slower convergence compared to other local search algorithms.

3.5.3 Local Beam Search Genetic Algorithm

Local Beam Search is a variant of local search that keeps multiple candidate solutions (beams) rather than one. It can also refer to Genetic Algorithms, which are inspired by natural selection and evolution to find optimal solutions.

Properties:

- **Complete:** Yes, given enough time and generations.
- **Optimal:** Generally, yes, especially with a good selection and mutation strategy, but not guaranteed.
- **Time Complexity:** Typically, $O(ng)$, where n is the number of solutions and g is the number of generations.
- **Space Complexity:** Depends on the number of solutions kept in the population.

Advantages:

- Explores multiple areas of the solution space simultaneously.
- Combines exploration and exploitation through crossover and mutation.

Disadvantages:

- Requires careful tuning of parameters (population size, mutation rates).
- Can converge prematurely on suboptimal solutions without diversity.

3.6 Constraint Satisfaction Problems (CSPs)

A CSP is a problem where the goal is to find a solution that satisfies a set of constraints. **Examples of CSPs:** Sudoku, crossword puzzles, map colouring, and scheduling problems. Each problem is defined by:

- **Variables:** A set of variables $X = \{x_1, x_2, \dots, x_n\}$.
- **Domains:** A set of possible values for each variable $D = \{D_1, D_2, \dots, D_n\}$.
- **Constraints:** A set of constraints that specify which combinations of values are valid for the variables.

Solution Methods:

1. **Backtracking Search:** A depth-first search algorithm for CSPs, which assigns values to variables one at a time and backtracks when a constraint is violated. **Properties:**
 - **Complete:** Yes, but inefficient without improvements.
 - **Optimal:** No, unless all solutions have the same cost.

2. **Constraint Propagation:** Uses techniques like arc-consistency to reduce the domain of variables by enforcing constraints early on, thereby pruning the search space.

Advantages:

- CSPs model many real-world problems naturally.
- Constraint propagation reduces the need for exhaustive search.

Disadvantages: Can be computationally expensive if the problem has many constraints or variables.

3.7 Local Search for CSPs

Local search methods attempt to solve CSPs by starting with an initial assignment of values to variables and iteratively improving the solution by making local changes.

Types of Local Search:

1. **Min-Conflicts Heuristic:** A heuristic that tries to minimize the number of constraint violations (conflicts) by iteratively changing the value of a variable to reduce the number of conflicts.
 - **How it works:** Randomly selects a conflicted variable and assigns it a value that results in the fewest conflicts.
 - **Example:** Solving the n-queens problem, where queens are placed on a chessboard in a way that no two queens threaten each other.
2. **Simulated Annealing for CSPs:** Allows worsening moves with some probability to escape local minima, gradually reducing the probability of these moves.
 - **How it works:** Like simulated annealing for optimization problems, it can escape local optima by making occasional random (worsening) moves, with a probability that decreases over time.

Advantages:

- **Memory Efficiency:** Local search methods often use less memory than backtracking or other complete search methods.
- **Effective for Large Problems:** Particularly useful for CSPs where complete algorithms would be too slow or memory-intensive.

Disadvantages:

- **No Guarantee of Optimality:** Local search may get stuck in local optima.
- **Incomplete:** Local search does not guarantee finding a solution, even if one exists.

3.8 Comparative study

Criterion	Greedy First Search	Best-A* Search	Memory-Bounded Heuristic Search	CSPs	Local Search for CSPs
Search Strategy	Expands node closest to the goal	Combines path cost and heuristic	Memory-efficient variants of A*	Find assignments for variables that satisfy constraints	Starts with an initial solution and improves iteratively

Heuristic Used	$h(n)$ (cost to goal)	$f(n)=g(n)+h(n)$	Bounded versions of $f(n)=g(n)+h(n)$	Heuristics to propagate constraints, reduce search space	Min-conflicts heuristic, simulated annealing
Completeness	No, can get stuck in loops	Yes, with admissible heuristic	Yes (SMA*)	Yes, if using backtracking	No, depends on the problem
Optimality	No, doesn't consider path cost	Yes, with admissible heuristic	Yes (SMA* is optimal under memory limits)	Not guaranteed without additional strategies	No, can get stuck in local optima
Time Complexity	Depends on heuristic quality, generally $O(b^m)$	$O(b^m)$	More efficient than A* in terms of memory	Depends on the constraint propagation and backtracking	Depends on iterations and conflicts
Memory Usage	Moderate to high	High, since all nodes are stored	Bounded by memory constraints	Moderate to high, depending on propagation	Low, only current assignments are stored
Best For	Fast solutions in well-formed heuristic problems	Optimal solution finding with admissible heuristic	Memory-constrained problems	Scheduling, planning, puzzles	Large CSPs where exhaustive search is impractical

4 Adversarial Search

Adversarial search is used in multi-agent environments, typically in competitive settings such as games. Here, agents must consider the actions of other agents, who may have conflicting goals.

4.1 Games

Games in the context of AI refer to a structured problem in which two or more players take turns making moves to maximize their own advantage while minimizing the opponent's advantage. These games can be:

- **Deterministic:** The outcome is completely determined by the players' actions (e.g., chess, checkers).
- **Zero-Sum:** A gain by one player results in an equal loss by the other (e.g., tic-tac-toe).

Examples:

- **Perfect Information Games:** Both players know the full game state (e.g., chess, Go).
- **Imperfect Information Games:** Some information is hidden from the players (e.g., poker).

4.2 Optimal Decisions & Strategies in Games

In adversarial games, an optimal strategy is one that guarantees the best possible outcome for a player, regardless of the opponent's actions. Each player aims to maximize their own payoff while minimizing the opponent's payoff.

Key Concept - Utility Function: Represents the outcome of the game, with values typically ranging from a win (+1), loss (-1), or draw (0).

Strategies:

- **Maximizing Player (MAX):** Seeks to maximize the outcome or utility.
- **Minimizing Player (MIN):** Seeks to minimize the opponent's utility, i.e., maximize their own gain by minimizing the other's.

GAMES & PROBLEMS

Contents

1. 8 Puzzle Problem.....	1
2. Vacuum Cleaner problem.....	2
3. Water Jug Problem	3
4. Missionaries and Cannibals Problem.....	4
5. N-Queens Problem	4
6. Tower of Hanoi.....	5
7. Tic-Tac-Toe	6
8. Cryptarithmic Problem.....	6
9. Minimax Algorithm	8
10. Alpha-Beta Pruning.....	9

8 Puzzle Problem

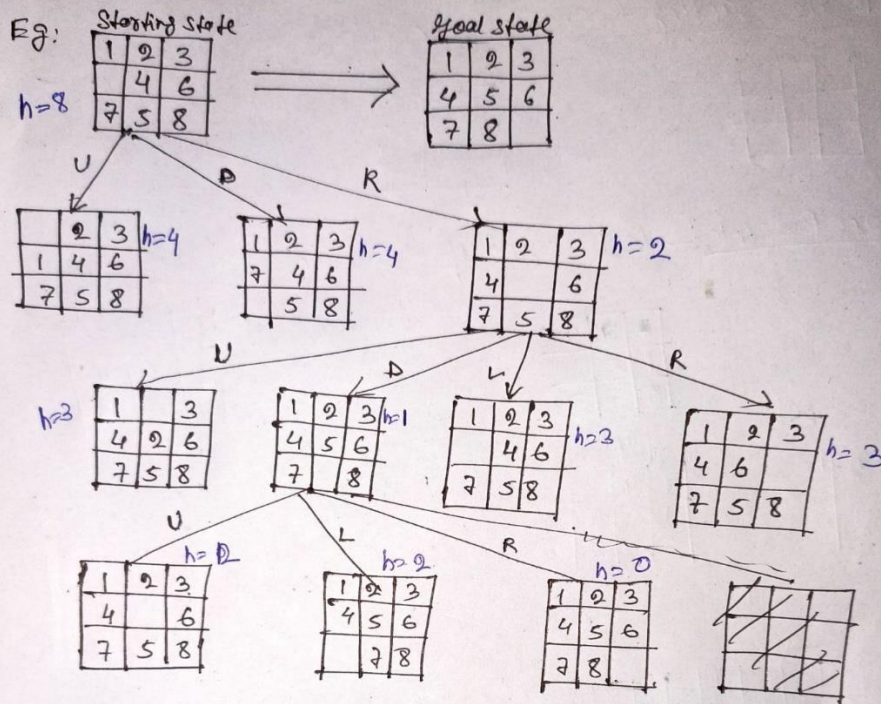
The 8-puzzle consists of a 3x3 grid with 8 numbered tiles and one empty space. The goal is to move the tiles around by sliding them into the empty space to achieve a specified goal state (usually the tiles ordered from 1 to 8).

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions:** The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.
- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

- **Constraints:**
 - Not all moves are possible in every state, as some tiles are on the boundary of the grid.
 - The number of inversions determines if the puzzle is solvable (even numbers of inversions imply solvability).
- **Search Techniques:** Often solved with A* search, **Breadth-First Search (BFS)**, or **Depth-First Search (DFS)**.

8-puzzle problem with heuristic

Here we assume the heuristic is "no of miss. placed tiles from goal state."



Vacuum Cleaner problem

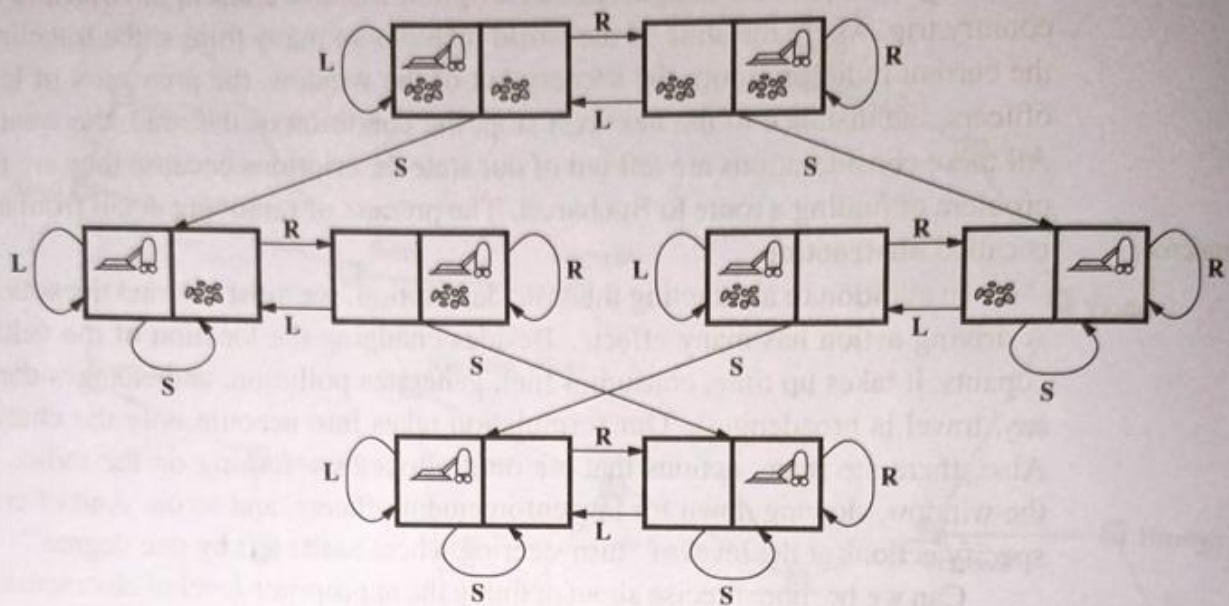


Figure 3.3 The state space for the vacuum world. Links denote actions: L = Left, R = Right, S = Suck.

3.2.1 Toy problems

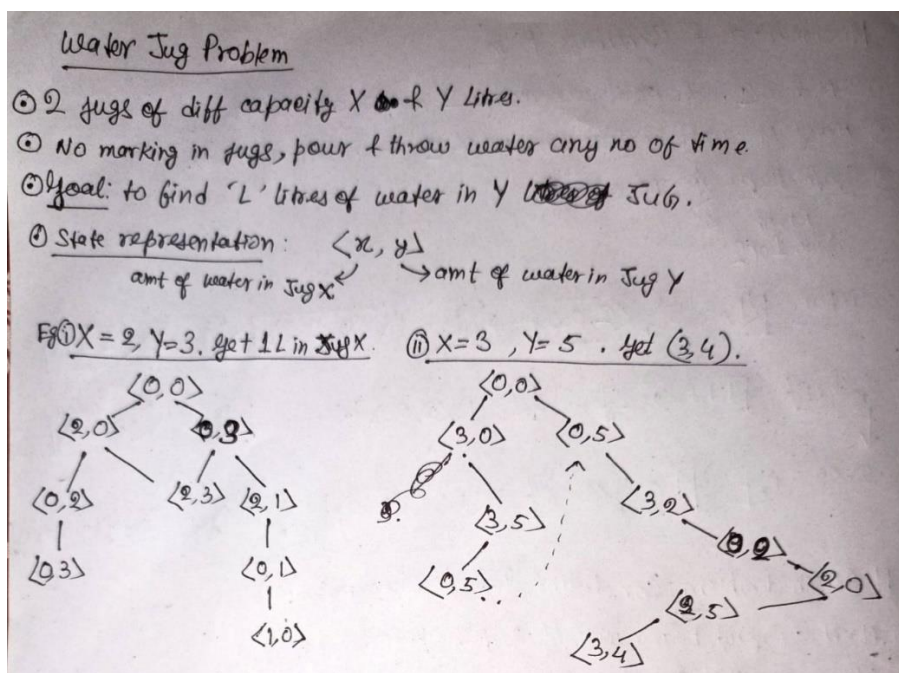
The first example we examine is the **vacuum world** first introduced in Chapter 2. (See Figure 2.2.) This can be formulated as a problem as follows:

- **States:** The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** In this simple environment, each state has just three actions: *Left*, *Right*, and *Suck*. Larger environments might also include *Up* and *Down*.
- **Transition model:** The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Sucking* in a clean square have no effect. The complete state space is shown in Figure 3.3.
- **Goal test:** This checks whether all the squares are clean.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

Water Jug Problem

The problem involves two jugs with different capacities and an infinite water supply. The goal is to measure out an exact quantity of water using the two jugs.:

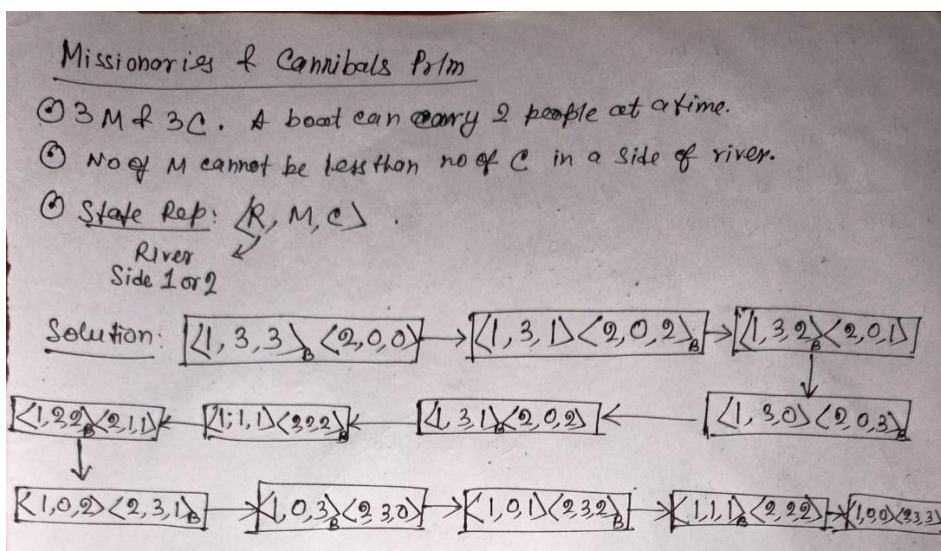
- **State space:** A state is represented as a pair of numbers (x, y) , where x is the amount of water in jug 1, and y is the amount in jug 2.
- **Initial state:** Both jugs are empty $(0, 0)$.
- **Goal state:** The target amount of water in either one of the jugs (e.g., 4 liters in one of the jugs).
- **Actions:** Fill a jug completely, empty a jug., Pour water from one jug to another until one jug is full, or the other is empty.
- **Constraints:**
 - The capacity of each jug is fixed.
 - You cannot measure intermediate amounts without performing valid actions.
- **Search Techniques:** BFS, DFS, or A* are common approaches for solving the problem



Missionaries and Cannibals Problem

The problem involves 3 missionaries and 3 cannibals who need to cross a river using a boat. The challenge is to ensure that at no point do the cannibals outnumber the missionaries on either side of the river (as this would lead to the missionaries being eaten).

- **State space:** A state can be represented as a tuple (M_left, C_left, boat), where M_left and C_left are the number of missionaries and cannibals on the left side of the river, and the boat indicates its position (left or right).
- **Initial state:** All 3 missionaries and 3 cannibals are on the left bank, with the boat on the left.
- **Goal state:** All missionaries and cannibals are on the right bank.
- **Actions:** Move 1 or 2 people across the river in the boat (combinations of missionaries and cannibals).
- **Constraints:**
 - The boat can only hold up to 2 people.
 - You cannot have more cannibals than missionaries on either side of the river (except when there are no missionaries).
- **Search Techniques:** The problem can be solved using **BFS**, **DFS**, or **Iterative Deepening Search (IDS)**.

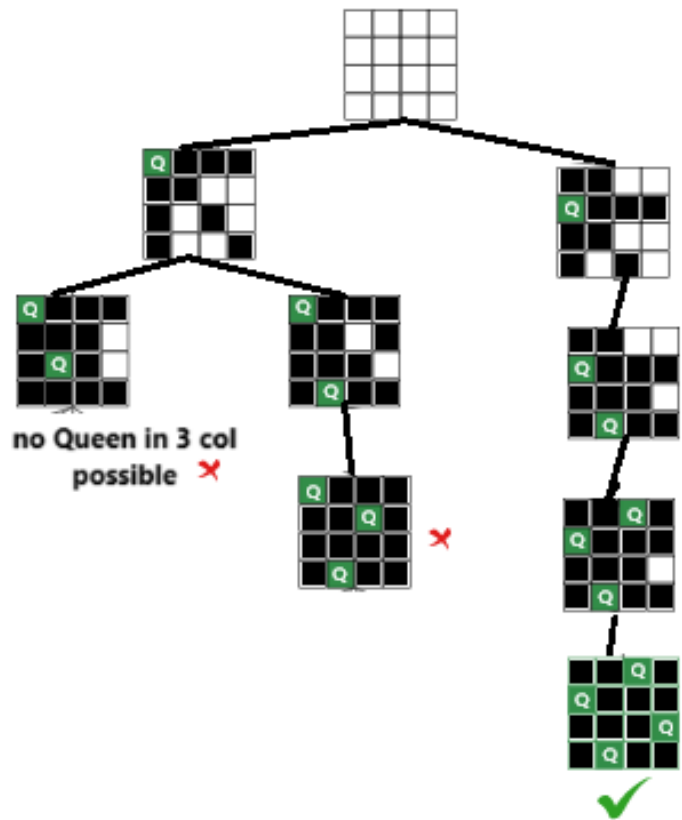
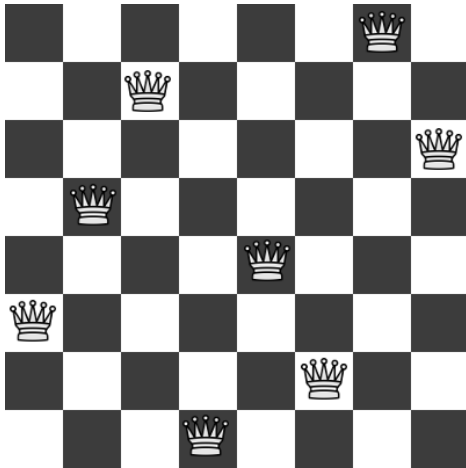


N-Queens Problem

The objective is to place N queens on an N×N chessboard such that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal).

- **State space:** Every arrangement of queens on the board represents a state.
- **Initial state:** An empty board.
- **Goal state:** A valid arrangement of N queens where no two queens are attacking each other.
- **Actions:** Place a queen in an available, non-attacking position on the board.
- **Constraints:** No two queens can be in the same row, column, or diagonal.
- **Number of Solutions:**
 - 4 Queens: There are 2 solutions.
 - 8 Queens: There are 92 solutions.

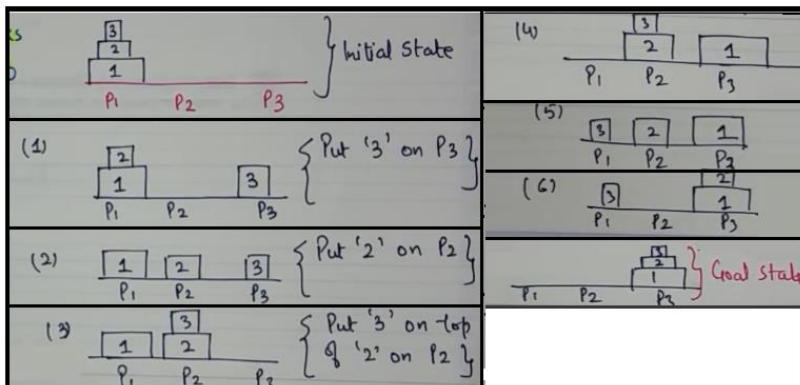
- **Search Techniques:** Solved using Backtracking, Constraint Satisfaction, or Local Search (Hill Climbing, Simulated Annealing).



Tower of Hanoi

The problem involves 3 pegs and N disks of different sizes. The objective is to move all disks from the first peg to the third peg, following these rules:

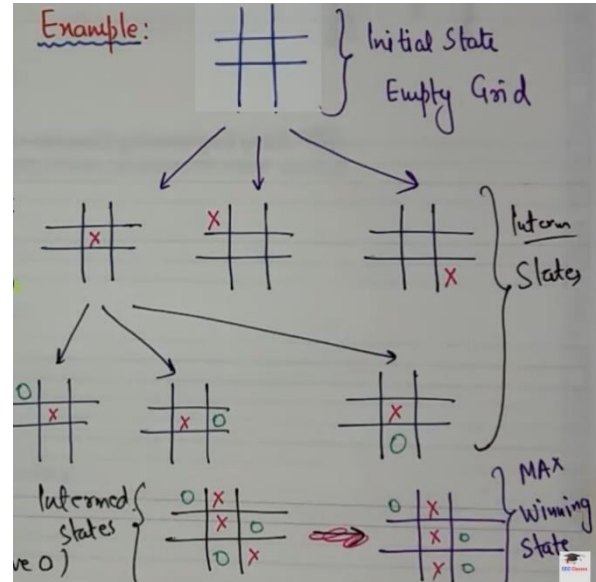
- Only one disk can be moved at a time.
- A disk can only be placed on top of a larger disk.
- You can use the second peg as an auxiliary.
- **State space:** Every arrangement of the disks on the three pegs represents a state.
- **Initial state:** All disks are on the first peg.
- **Goal state:** All disks are on the third peg, in the same order.
- **Actions:** Move a disk from one peg to another, subject to the rules.
- **Constraints:**
 - Only one disk can be moved at a time.
 - A larger disk cannot be placed on a smaller disk.
- **Number of Moves:**
 - For N disks, the minimum number of moves required is $2^N - 1$.
 - Example for 3 disks: **7 moves**.
- **Search Techniques:** Solved using **recursion** or **dynamic programming**.



Tic-Tac-Toe

A two-player game played on a 3x3 grid. The players take turns marking a space on the grid with their symbol (either X or O). The goal is to get three of their symbols in a row (either horizontally, vertically, or diagonally).

- **State space:** Every arrangement of Xs and Os on the board represents a state.
- **Initial state:** An empty 3x3 board.
- **Goal state:** A configuration where either player has aligned 3 of their symbols in a row, or a draw if no moves are left.
- **Actions:** Place X or O in an empty square.
- **Constraints:**
 - Players alternate turns.
 - A player can only place their symbol in an empty square.
 - The game ends when a player wins or there are no more moves (draw).
- **Search Techniques:** Solved using **Minimax Algorithm** (with or without alpha-beta pruning), which explores all possible game states to find an optimal move for the current player.



Cryptarithmic Problem

A **cryptarithmic** problem is a mathematical puzzle where the digits in a given arithmetic equation are replaced by letters (or symbols). The objective is to assign digits to the letters such that the arithmetic equation holds true. Each letter represents a unique digit.

- **Goal:** Here, each letter represents a unique digit, and the goal is to find the digit assignments such that the equation is valid.
- **State space:** Each possible assignment of digits to letters is a state.
- **Initial state:** No letters are assigned digits (all letters are variables).
- **Goal state:** A valid digit assignment where the arithmetic equation is satisfied.
- **Actions:** Assign a digit (0-9) to a letter, ensuring no two letters are assigned the same digit. Once all letters are assigned, verify whether the arithmetic equation holds true.
- **Constraints:**
 - Each letter must represent a unique digit.
 - No two letters can have the same digit.
 - The arithmetic equation must be correct after the assignment of digits.
 - Often, the leftmost letter of a number cannot be zero (e.g., in "SEND", 'S' cannot be 0).
- **Search Techniques:**
 - **Backtracking:** A common approach to solving cryptarithmic problems, where we try assigning digits to letters one by one, backtracking if we find a conflict or if the arithmetic equation is not satisfied.
 - **Constraint Satisfaction Problem (CSP):** The problem can be formulated as a CSP where the variables are the letters, the domain is the set of digits, and the constraints are based on the uniqueness of the digits and the correctness of the arithmetic operation.

Cryptarithmic Problem

Q.1) $\begin{array}{r} T O \\ G O \\ \hline O U T \end{array}$ | Ans) Left most value of Answer that is O is always 1 (due to carry). $\boxed{O=1}$

$\begin{array}{r} T 1 \\ G 1 \\ \hline 1 U T \end{array} \rightarrow \begin{array}{r} 2 1 \\ G 1 \\ \hline 1 U 2 \end{array}$ we see. $2+G=U$ $\therefore \boxed{T=2}$

Now $T+G$ must be >9 so that we get carry for $O=1$.

Let $\boxed{G=8}$ $\begin{array}{r} 2 1 \\ 8 1 \\ \hline 1 0 2 \end{array}$ ✓ $\boxed{U=0}$

$\boxed{\begin{array}{l} T=2 \\ O=1 \\ U=0 \\ G=8 \end{array}}$

Q.2) $\begin{array}{r} S E N D \\ M O R E \\ \hline M O N E Y \end{array} \rightarrow \begin{array}{r} S E N D \\ M O R E \\ \hline M O N E Y \end{array}$, Now $S+1 > 10$. Assume $\boxed{S=9}$, then $\boxed{O=0}$ $\boxed{M=1}$

$\begin{array}{r} 9 E N D \\ 1 0 R E \\ \hline 1 1 0 N E Y \end{array}$, As $E \neq N$, So there should be a carry from $(N+R)$. Let $\boxed{E=4}$, $N \Rightarrow E+O+1=6 \Rightarrow \boxed{N=6}$

$\begin{array}{r} 9 5 6 D \\ 1 0 R 5 \\ \hline 1 0 6 5 Y \end{array}$, $6+R=5$ with a carry 1 $\therefore R=9$. X This is wrong as $S=9$ already. So there must be a carry from $(D+5)$

$\therefore 6+1+R=15$, Now. $D+5=Y$ with a carry. $\boxed{R=8}$. Assume $\boxed{D=7}$ ($N=6$ & $E=5$ so we can't take) $\boxed{Y=2}$

Q.3) $\begin{array}{r} E A T \\ T H A T \\ \hline A P P L E \end{array}$ $\begin{array}{r} 8 1 9 \\ 9 2 1 9 \\ \hline 1 0 0 3 8 \end{array}$ $\boxed{A=1} \rightarrow \boxed{1=2} (A+A)$ $\boxed{T=9} \rightarrow \boxed{P=0} (T+T=10)$ $E=T+T=18 \Rightarrow \boxed{E=8}$ $H+E=P$ $\Rightarrow H+8=0 \Rightarrow \boxed{H=2}$

Q.4) SOME TIME SPENT

1	9	3	4	
8	5	3	4	
1	0	4	6	8

$S = 1$
 $T = 2E, \therefore T = \text{even}$
 $N = 2M, \therefore N = \text{even}$

Assume $T = 8 \rightarrow E = 4$ | $P = S + T = 1 + 8 = 9 \Rightarrow P = 9$ (1 for carry)

Assume $N = 6 \rightarrow M = 3$ | $O + I = 4$

Let $D = 9, I = 5$

Q.5) BASE BALL GAMES

7	4	7	2	
7	4	5	5	
1	4	9	2	7

$G = 1$

We see $E + L = 8, S + L = E$

Assume $E + L = 8$ with carry $\Rightarrow E + L = S + 10$

$\Rightarrow E = S - L + 10$ | $\therefore S - E = 5$

$\Rightarrow S + L = S - L + 10$ | $(S, E) \in \{(5, 0), (6, 1), (7, 2), (8, 3), (9, 4)\}$

$\Rightarrow L = 5$ | $\times$ $\times$

Now, $B + B + \text{carry} = A + \text{carry} \rightarrow B$ can be 5, 6, 7, 8, 9 (as $L = 5$)

Case 1: Let $(S, E) = (7, 2), S = 7, E = 2, B = 6$

$\therefore A = B + B = 6 + 6 = 12 \Rightarrow A = 2$ X NOT possible

Case 2: Let $S = 8, E = 3, B = 7$

$A = 7 + 7 = 14 \Rightarrow A = 4$ | $M = 4 + 4 = 8 \Rightarrow M = 8 + 1 \Rightarrow M = 9$

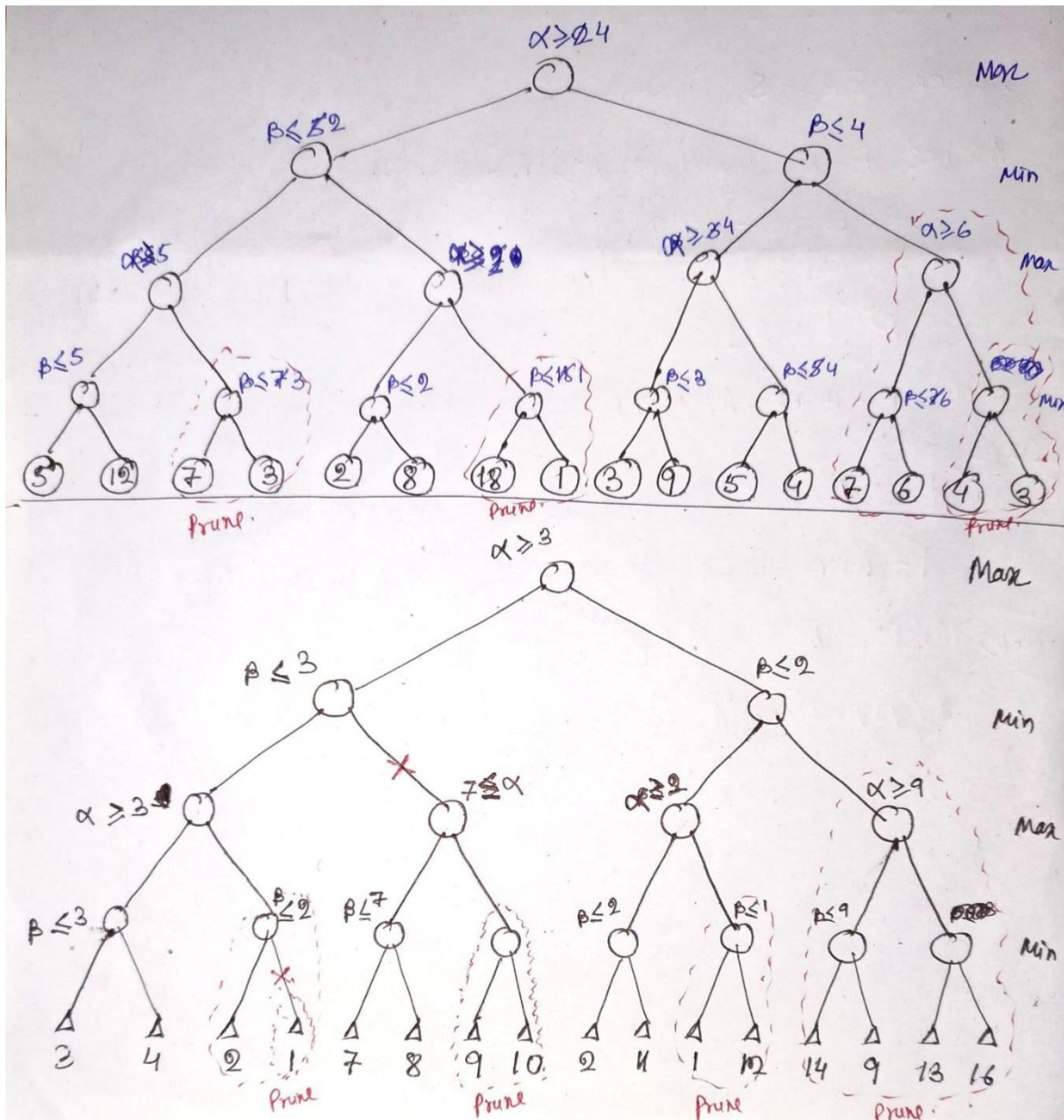
Minimax Algorithm

The **Minimax Algorithm** is a decision-making algorithm used in **two-player zero-sum games**, where one player's gain is the other player's loss. It's commonly used in games like **Tic-Tac-Toe**, **Chess**, and **Checkers**. The algorithm assumes that both players play optimally. **Key Concepts:**

- **Maximiser:** One player tries to **maximize** their score.
- **Minimizer:** The opponent tries to **minimize** the maximiser's score.
- **Game Tree:** Represents all possible moves for both players. Each node in the tree is a state of the game.
- Solution is found if it exists.

Example of Alpha-Beta Pruning:

1. **Max** is trying to find the best move.
2. **Min** is evaluating a branch, but finds that its value is less than **alpha**, meaning the Max player will never allow it to happen.
3. The algorithm stops evaluating the remaining branches (prunes them) because they cannot lead to a better outcome for Min.



MODULE 3: KNOWLEDGE & REASONING

Contents

1. Key Points.....	1
2. Steps of the Knowledge Engineering Process.....	2
3. Inference Engine	2
4. Forward Chaining (Data-Driven Reasoning).....	3
5. Backward Chaining (Goal-Driven Reasoning).....	4
6. What is Knowledge Representation?	7
7. Types of Knowledge.....	7
8. Knowledge Representation Challenges	7
9. Knowledge Representation Methods.....	7
10. Mapping Knowledge to Representation	8
11. Issues in Knowledge Representation.....	8
12. Inference Mechanisms.....	9

Key Points

1. **Logical Agents:** Agents that make decisions based on logical reasoning and knowledge representation. Use formal logic (like propositional or first-order logic) to derive conclusions from known facts and rules.
2. **Knowledge-Based Agents:** Agents that maintain a knowledge base of facts about the world and use it to make decisions. They apply logical inference rules to the knowledge base to decide actions.
3. **Semantics:** In logic, semantics refers to the meaning of symbols, sentences, or expressions in a formal system. It connects logical expressions to real-world situations, ensuring that the agent's inferences are meaningful.
4. **Forward and Backward Chaining:**
 - a. **Forward Chaining:** A method of reasoning that starts from known facts and applies inference rules to derive new facts (data-driven approach).
 - b. **Backward Chaining:** A goal-driven reasoning method that starts with a goal and works backward to find facts that support the goal.
5. **Agents Based on Propositional Logic:** Agents that use propositional logic to represent their knowledge base. Propositional logic deals with statements that are either true or false. **Example:** An agent uses a set of true/false statements about the world to make decisions.
6. **Hybrid Agent:** An agent that combines different approaches, such as logical reasoning and machine learning, to enhance decision-making. These agents use multiple techniques (logic-based reasoning, probability, learning, etc.) for more versatile and flexible behaviour.

7. Atomic and Complex Sentences:

- a. **Atomic Sentences:** Simple, indivisible statements in logic (e.g., "It is raining").
- b. **Complex Sentences:** Combinations of atomic sentences using logical connectives (e.g., "It is raining AND it is cold").

8. Term, Ground Term:

- a. **Term:** Represents objects in the domain of discourse in logic. Terms can be variables, constants, or functions.
- b. **Ground Term:** A term that contains no variables, only constants (e.g., "John" or "5").

9. First-Order Predicate Logic (FOPL): A more expressive form of logic than propositional logic. It includes quantifiers (e.g., $\forall$ for all, $\exists$ for some) and predicates that can take objects as arguments. **Key Feature:** Allows for expressing relationships between objects (e.g., "All humans are mortal").

Steps of the Knowledge Engineering Process

1. **Identify the Problem:** Define the specific problem the knowledge-based system will solve. Understand the domain and the scope of the problem.
2. **Knowledge Acquisition:** Collect and gather knowledge from experts, books, or data sources. Identify the key facts, rules, and procedures needed for decision-making.
3. **Knowledge Representation:** Choose a method for representing the acquired knowledge, such as:
 - Logic (propositional or first-order predicate logic).
 - Semantic networks.
 - Frames or rules.
4. **Design the Knowledge Base:** Organize the facts, rules, and knowledge into a structured format (e.g., knowledge base). Ensure it can be updated or modified as needed.
5. **Define Inference Mechanism:** Select an inference engine or reasoning technique (e.g., forward or backward chaining). Determine how the system will apply knowledge to solve problems.
6. **Develop the Knowledge-Based System:** Build the system, integrating the knowledge base and inference mechanism. Implement the system using AI programming languages (like Prolog or Python).
7. **Testing and Validation:** Test the system on sample problems or scenarios to ensure it behaves correctly. Validate its performance by comparing the system's output with expert solutions.
8. **Refinement:** Continuously refine the knowledge base and rules based on feedback and testing results. Address gaps or errors in the system's knowledge or reasoning.
9. **Deployment:** Deploy the system in the intended environment where users can interact with it.
10. **Maintenance and Updates:** Regularly update the knowledge base with new information. Adapt the system as the problem domain or requirements evolve.

Inference Engine

An **Inference Engine** is a component of a **knowledge-based system**, such as expert systems or artificial intelligence systems, that applies logical rules to a knowledge base (containing facts and rules) to deduce new information or solve problems. The inference engine processes the knowledge base and works through reasoning to arrive at conclusions. **Functionality:**

- It applies **rules of inference** to derive new facts from existing facts.

- The process of inferring new knowledge involves **applying rules** in a systematic way to the available data.
- It can handle both **forward chaining** and **backward chaining** as reasoning strategies.
- **Components:**
 - **Knowledge Base:** Contains facts and rules (in the form of IF-THEN statements).
 - **Inference Mechanism:** Applies logical reasoning techniques to draw conclusions from the knowledge base.
 - **Working Memory:** Stores intermediate facts during the reasoning process.

Forward Chaining (Data-Driven Reasoning)

Forward chaining starts with known facts and applies inference rules to extract new facts or conclusions. This process is repeated until the desired goal is reached or no more rules can be applied.

How it Works:

- Starts from the **initial facts** in the knowledge base.
- Uses **if-then rules** to infer new facts (if the **if** part of the rule is true, then the **then** part can be added to the knowledge base).
- Continues adding inferred facts to the knowledge base, progressing **forward**, until the goal or conclusion is reached.

Example (Rule-based system):

- **Rules:**
 - Rule 1: If it is raining, then the grass is wet.
 - Rule 2: If the grass is wet, then it is slippery outside.
- **Facts:** It is raining.
- **Process:**
 - From the fact "It is raining," Rule 1 is triggered, and the system concludes that "the grass is wet."
 - With the fact "the grass is wet," Rule 2 is triggered, leading to the conclusion that "it is slippery outside."

Applications:

- **Expert Systems:** Systems that diagnose diseases or recommend actions based on symptoms (e.g., medical diagnosis).
- **Production Systems:** Systems that make decisions in response to sensor inputs, like industrial robots or automated reasoning systems.

Advantages:

- **Data-Driven:** Starts from available data, making it suitable for situations where the system has access to many known facts.
- **Easy to Implement:** Straightforward logic flow, ideal for situations with many rules that generate many possible conclusions.

Disadvantages:

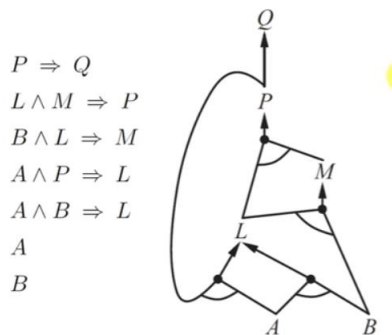
- **Inefficiency:** It may generate unnecessary conclusions that do not lead to the goal, especially if there are many irrelevant rules.

- **No Goal Focus:** It does not start with a goal, so it may explore paths that are not relevant to the problem.

Forward Chaining is useful when:

- You have many known facts but don't have a specific goal.
- You want to derive all possible conclusions from the facts.
- You are dealing with sensor-driven systems or production systems (e.g., automation, real-time systems).

Example



Backward Chaining (Goal-Driven Reasoning)

Backward chaining works by starting from the desired goal or conclusion and working **backward** to determine which facts must be true to support the goal. The process continues until the system reaches known facts or finds that the goal is unachievable.

How it Works:

- Starts from a **goal** (a fact or condition you want to prove or achieve).
- Searches for rules where the **then** part (conclusion) matches the goal.
- Checks whether the **if** part of that rule is true (i.e., tries to prove the conditions).
- If the conditions are not known, backward chaining attempts to prove them as sub-goals, working backward through the chain of logic.

Example (Rule-based system):

- **Goal:** Prove that "it is slippery outside."
- **Rules:**
 - Rule 1: If it is raining, then the grass is wet.
 - Rule 2: If the grass is wet, then it is slippery outside.
- **Process:**
 - The goal is "it is slippery outside," which matches Rule 2's conclusion.
 - To prove this, the system needs to check if the grass is wet (from the **if** part of Rule 2).
 - It now works backward to see if Rule 1's conditions hold (i.e., if "it is raining"), which is a known fact.
 - Once it verifies "it is raining," the system confirms the goal "it is slippery outside."

Applications:

- **Expert Systems:** Used in systems where a specific conclusion needs to be verified, such as legal reasoning or troubleshooting guides.

- **Prolog Programming:** Prolog, a logic programming language, uses backward chaining as its primary inference mechanism.

Advantages:

- **Goal-Driven:** More efficient when you have a specific goal, as it focuses on proving or disproving that goal.
- **Focused Search:** Avoids unnecessary exploration of irrelevant rules or facts, unlike forward chaining.

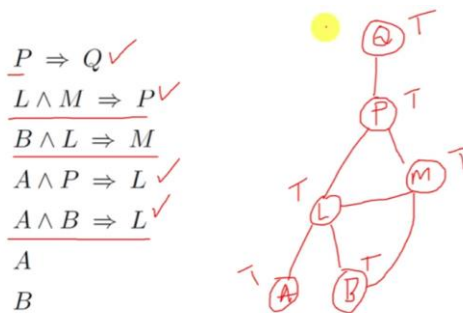
Disadvantages:

- **Limited Exploration:** Can miss relevant conclusions if the goal isn't properly defined.
- **Data Availability:** Needs to have rules that support working backward from the goal, and might struggle in domains with insufficient rules or facts.

Backward Chaining is useful when:

- You have a specific goal in mind and want to find out whether it can be achieved.
- You need to prove or disprove a hypothesis or diagnosis.
- You want to focus on a specific conclusion rather than explore all possible outcomes (e.g., Prolog systems, medical diagnosis).

Example



Feature	Forward Chaining	Backward Chaining
Type	Data-Driven (Starts from facts)	Goal-Driven (Starts from a goal)
Starting Point	Known facts in the knowledge base	A specific goal to be achieved
Direction	Works forward from facts to conclusions	Works backward from the goal to find supporting facts
Efficiency	May explore unnecessary rules or facts	More efficient when a specific goal is known
Use Cases	When many facts are available, and the goal isn't specified	When the goal is known and facts are limited
System Type	Used in expert systems and production systems	Used in diagnostic systems and legal reasoning
Computational Complexity	May be computationally expensive due to the wide exploration of facts	More efficient in smaller problem spaces with focused goals

Example

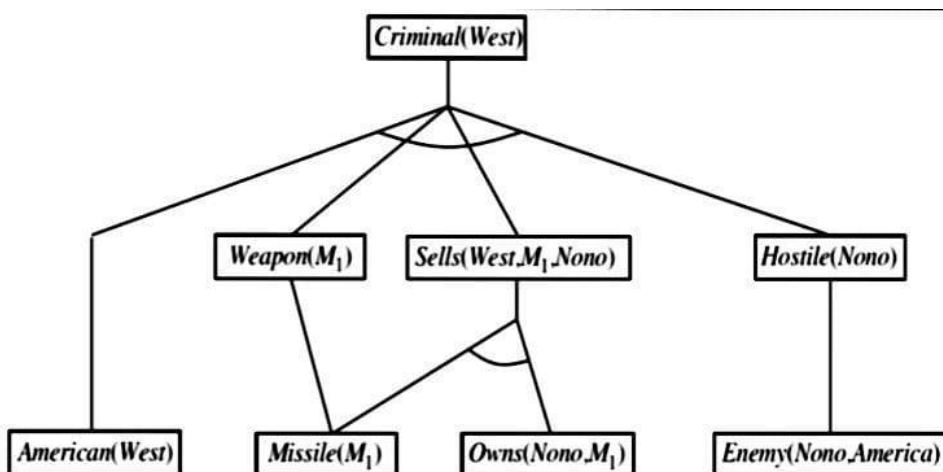
The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American. An enemy of America counts as "hostile".

Prove that: West is a Criminal using i) Forward chaining ii) Backward chaining

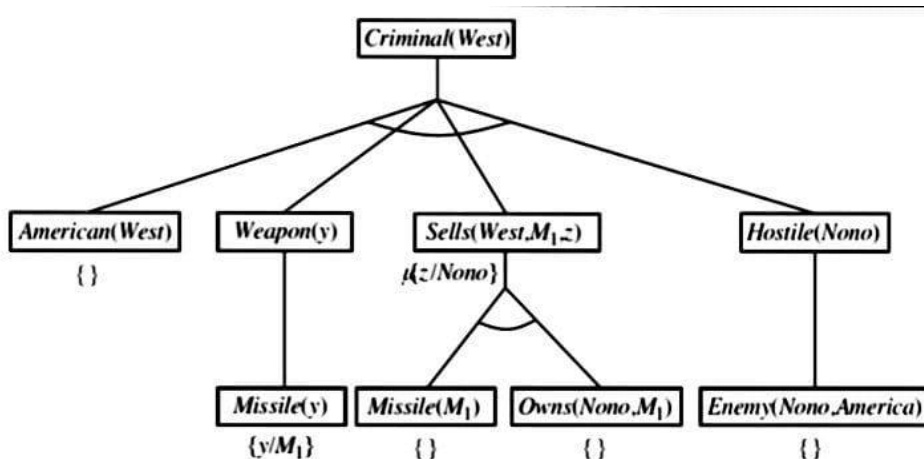
Solution:

1. $\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
2. $\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
3. $\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
4. $\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$
5. $\text{American}(\text{West})$
6. $\text{Enemy}(\text{Nono}, \text{America})$
7. $\text{Owns}(\text{Nono}, M_1)$ and
8. $\text{Missile}(M_1)$

Forward Chaining:



Backword Chaining:



What is Knowledge Representation?

Knowledge Representation (KR) is the field of AI concerned with how knowledge about the world can be formally represented and structured in a way that a computer can process, reason, and make decisions.

KR allows machines to store, retrieve, and manipulate data to simulate human-like understanding and reasoning.

- **Role in AI:** KR plays a central role in AI because it forms the foundation for tasks like problem-solving, learning, decision-making, and natural language processing. The way knowledge is represented impacts the performance and efficiency of AI systems.
- **Importance in Reasoning and Decision-Making:**
 - It allows AI systems to **reason** about the world by drawing inferences from the represented knowledge.
 - It is crucial for decision-making processes, enabling the system to select optimal actions based on the available facts, constraints, and goals.

Types of Knowledge

1. **Declarative Knowledge:** Represents **facts** about the world (e.g., "Paris is the capital of France"). **Static knowledge:** Can be directly stated or queried without needing action to be performed (e.g., facts about objects or entities). Example: "John is a student."
2. **Procedural Knowledge:** Represents the knowledge of **how to do things** or the steps required to perform a task (e.g., how to cook a dish or solve a problem). **Dynamic knowledge:** Often involves algorithms, procedures, or workflows. Example: The steps to bake a cake.
3. **Factual vs. Inferential Knowledge:** **Factual Knowledge:** Represents **what** is true, raw information (e.g., "The sky is blue"). Example: "It is raining."
4. **Inferential Knowledge:** Knowledge derived from facts or rules, **how** things are related (e.g., "If it rains, the ground will be wet"). Example: "The ground is wet."

Knowledge Representation Challenges

1. **Expressiveness:** The system's ability to represent diverse real-world knowledge, including relationships, hierarchies, and constraints.
2. **Efficiency:** The system's capability to quickly access and manipulate knowledge for fast inference and decision-making.
3. **Scalability:** The system's ability to handle large knowledge bases without losing performance or accuracy as new facts or rules are added.
4. **Interpretability:** The ease with which humans can understand and interpret the system's knowledge and outputs.

Knowledge Representation Methods

1. **Semantic Networks:**
 - a. A graphical representation where **nodes represent concepts** or entities, and **edges represent relationships** between them.
 - b. Useful for representing **hierarchies** (e.g., "A dog is an animal").
 - c. Example: **Node:** "Dog", **Node:** "Animal", **Edge:** "is a"
2. **Frames:**

- a. Data structures used to represent **stereotyped situations**, often containing slots (attributes) and fillers (values).
- b. Frames are like objects in object-oriented programming, where specific data is stored in predefined attributes.
- c. Example: A frame for a "Person" could have attributes like "Name," "Age," and "Gender."
- 3. **Logic (Propositional, Predicate Logic):**
 - a. **Propositional Logic:** Deals with simple statements that can be true or false (e.g., "The sky is blue").
 - b. **Predicate Logic:** Allows representation of more complex statements involving **quantifiers** (like "For all" or "There exists") and predicates (e.g., "Person(John)").
 - c. Predicate logic allows expressing facts like "John is a student" and applying rules to infer new facts.
- 4. **Rules (IF-THEN):**
 - a. **Rule-based representation** where facts or conditions (IF) lead to actions or conclusions (THEN).
 - b. Example: "IF it is raining THEN the ground is wet."
 - c. Commonly used in **expert systems**.

Mapping Knowledge to Representation

- 1. **Formalization:**
 - a. The process of converting human-readable knowledge into a **formal system** that a computer can process.
 - b. Formalizing knowledge ensures that it can be reasoned about by machines in a consistent and logical way.
- 2. **Ontologies:**
 - a. A formal specification of a **conceptualization** (i.e., a shared understanding of a domain).
 - b. Ontologies represent a set of **concepts** and the **relationships** between them, allowing for structured and explicit knowledge representation.
 - c. Example: An ontology for an e-commerce system might include concepts like "Product", "Customer", and "Order".
- 3. **Taxonomies:**
 - a. Hierarchical classification of concepts or objects in a domain.
 - b. Used to categorize entities into **parent-child** relationships (e.g., "Mammals" is a parent to "Dogs" and "Cats").
- 4. **Knowledge Graphs:**
 - a. A graph-based structure that represents **entities** as nodes and **relationships** between them as edges.
 - b. Knowledge graphs integrate various types of knowledge and allow for **reasoning** based on connected nodes (e.g., Google Knowledge Graph).

Issues in Knowledge Representation

- 1. **Ambiguity and Vagueness**
 - a. **Handling Imprecise Information (Fuzzy Logic):**
 - i. Fuzzy logic is a system that allows for **gradual truth values** rather than the binary true/false used in classical logic.
 - ii. It handles uncertainty and vagueness, for example, "tall" can be represented as a range instead of a precise value.
 - b. **Example of Ambiguous Terms:**

- i. Words like “**tall**”, “**soon**”, or “**hot**” have fuzzy boundaries and can vary depending on context. Fuzzy logic provides a way to represent and reason with such imprecise terms.

2. Incompleteness

a. **Dealing with Incomplete Knowledge (Assumption-based Reasoning):**

- i. In many situations, knowledge is incomplete, and assumptions are made to proceed with reasoning.
- ii. **Default reasoning:** Using defaults when information is missing (e.g., "If the person's age is not specified, assume the person is an adult").

3. Consistency

- a. **Maintaining Consistency in the Knowledge Base:** Knowledge systems must ensure that the facts stored do not contradict each other. Inconsistent knowledge leads to incorrect reasoning or conclusions.
- b. **Conflict Resolution:** Mechanisms to resolve conflicts in knowledge bases, such as prioritizing rules or using conflict resolution strategies (e.g., **prefer the most recent fact**).

4. Modularity

- a. **Importance of Modularity:** Knowledge systems must support **modularity** so that new knowledge can be added, updated, or removed without affecting other parts of the system.
- b. **Benefits for Adding/Removing Knowledge:** Modularity allows for **easy updates** and **maintenance**, as components of the knowledge base can be modified independently.

Inference Mechanisms

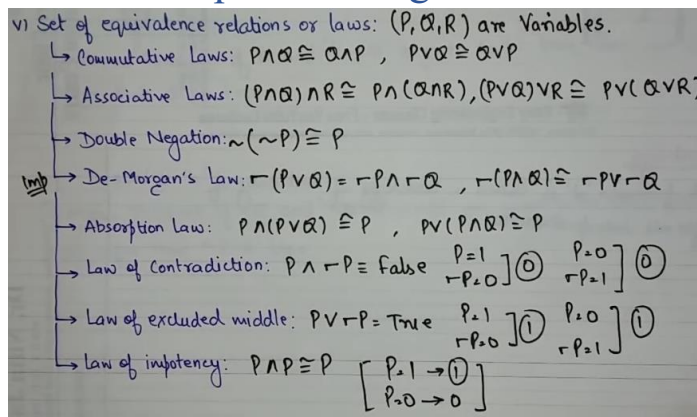
1. **Deductive Reasoning:** The process of drawing specific conclusions from general premises or facts. In deductive reasoning, if the premises are true, the conclusion must also be true. **Used in:** Logical proofs, formal systems, and rule-based systems. **Example:**
 - a. Premise 1: All humans are mortal.
 - b. Premise 2: Socrates is a human.
 - c. Conclusion: Socrates is mortal.
2. **Inductive Reasoning:** The process of generalizing from specific observations or examples to form a general rule or conclusion. The conclusions in inductive reasoning are probable, not certain. **Used in:** Machine learning, pattern recognition, and statistical reasoning. **Example:**
 - a. Observation: The sun has risen every day in the past.
 - b. Conclusion: The sun will rise tomorrow.
3. **Non-monotonic Reasoning:** In non-monotonic reasoning, conclusions drawn from a set of facts can be **revised** if new information is added, unlike in deductive reasoning where conclusions remain true regardless of additional information. **Used in:** Reasoning with incomplete, evolving, or contradictory knowledge (e.g., legal or medical reasoning). **Example:**
 - a. Fact 1: All birds can fly.
 - b. Fact 2: A penguin is a bird.
 - c. Conclusion: A penguin can fly.
 - d. **New Fact:** Penguins cannot fly.
 - e. Revised Conclusion: Penguins cannot fly.

MODULE 4: PROPOSITION LOGIC

Contents

1. Laws in Proposition Logic.....	1
2. Types of Inference rules	2
3. Inference Rules	5
4. Well-formed formula to Predicate Logic.....	7
5. Steps to Convert Logical Statement to CNF.....	9
6. Resolution.....	11
7. Bayesian Networks	15
8. Dempster-Shafer Theory (DST).....	18
9. Fuzzy Sets.....	21
10. Fuzzy Logic	21
a. Fuzzy logic Operations.....	22
b. Algebraic sum & product, bounded sum & difference.....	24
c. Cartesian Product	25
d. Max Min composition & Max product composition.....	25
e. Membership value for triangles using inference approach.....	26
11. Defuzzification.....	27

Laws in Proposition Logic



Example: Using First-Order Logic for Reasoning

To illustrate the use of FOL in reasoning, consider the following knowledge base:

1. $\forall x (Cat(x) \rightarrow Mammal(x))$ (All cats are mammals)
2. $\forall x (Mammal(x) \rightarrow Animal(x))$ (All mammals are animals)
3. $Cat(Tom)$ (Tom is a cat)

From these statements, we can infer:

1. $Mammal(Tom)$ (Since Tom is a cat, and all cats are mammals)
2. $Animal(Tom)$ (Since Tom is a mammal, and all mammals are animals)

These inferences demonstrate how FOL can be used to derive new knowledge from existing facts. The ability to reason logically about relationships and properties is a key strength of FOL in AI.

Q The CORRECT formula for the sentence, "not all rainy days are cold" is

a) $\forall_d (\text{Rainy}(d) \wedge \sim \text{Cold}(d))$ b) $\forall_d (\sim \text{Rainy}(d) \rightarrow \text{Cold}(d))$
c) $\exists_d (\sim \text{Rainy}(d) \rightarrow \text{Cold}(d))$ d) $\exists_d (\text{Rainy}(d) \wedge \sim \text{Cold}(d))$

Q Consider the statement "Not all that glitters is gold"
Predicate glitters(x) is true if x glitters and predicate gold(x) is true if x is gold. Which one of the following logical formulae represents the above statement?

a) $\forall_x: \text{glitters}(x) \Rightarrow \sim \text{gold}(x)$ c) $\exists_x: \text{gold}(x) \wedge \sim \text{glitters}(x)$
b) $\forall_x: \text{gold}(x) \Rightarrow \text{glitters}(x)$ d) $\exists_x: \text{glitters}(x) \wedge \sim \text{gold}(x)$

Q What is the logical translation of the following statement?
"None of my friends are perfect."

A) $\exists_x (F(x) \wedge \sim P(x))$ B) $\exists_x (\sim F(x) \wedge P(x))$
C) $\exists_x (\sim F(x) \wedge \sim P(x))$ D) $\sim \exists_x (F(x) \wedge P(x))$

Q Which one of the following is the most appropriate logical formula to represent the statement? "Gold and silver ornaments are precious". The following notations are used:
G(x): x is a gold ornament
S(x): x is a silver ornament
P(x): x is precious

(A) $\forall_x (P(x) \rightarrow (G(x) \wedge S(x)))$
(B) $\forall_x ((G(x) \wedge S(x)) \rightarrow P(x))$
(C) $\exists_x ((G(x) \wedge S(x)) \rightarrow P(x))$
D) $\forall_x ((G(x) \vee S(x)) \rightarrow P(x))$

Q Which one of the first order predicate calculus statements given below correctly express the following English statement? "Tigers and lions attack if they are hungry or threatened"

a) $\forall_x ((\text{tiger}(x) \wedge \text{lion}(x)) \rightarrow (\text{hungry}(x) \vee \text{threatened}(x)) \rightarrow \text{attacks}(x))$
b) $\forall_x ((\text{tiger}(x) \vee \text{lion}(x)) \rightarrow (\text{hungry}(x) \vee \text{threatened}(x)) \wedge \text{attacks}(x))$
c) $\forall_x ((\text{tiger}(x) \vee \text{lion}(x)) \rightarrow \text{attacks}(x) \rightarrow (\text{hungry}(x) \vee \text{threatened}(x)))$
d) $\forall_x ((\text{tiger}(x) \vee \text{lion}(x)) \rightarrow (\text{hungry}(x) \vee \text{threatened}(x)) \rightarrow \text{attacks}(x))$

Types of Inference rules

1. Modus Ponens: The Modus Ponens rule is one of the most important rules of inference, and it states that if P and $P \rightarrow Q$ is true, then we can infer that Q will be true. It can be represented as:

Notation for Modus ponens:
$$\frac{P \rightarrow Q, P}{\therefore Q}$$

Example:

Statement-1: "If I am sleepy then I go to bed" $\Rightarrow P \rightarrow Q$

Statement-2: "I am sleepy" $\Rightarrow P$

Conclusion: "I go to bed." $\Rightarrow Q$.

Hence, we can say that, if $P \rightarrow Q$ is true and P is true then Q will be true.

P	Q	$P \rightarrow Q$
0	0	0
0	1	1
1	0	0
1	1	1

2. Modus Tollens: The Modus Tollens rule state that if $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ will also true. It can be represented as:

Notation for Modus Tollens:
$$\frac{P \rightarrow Q, \sim Q}{\sim P}$$

Statement-1: "If I am sleepy then I go to bed" $\Rightarrow P \rightarrow Q$

Statement-2: "I do not go to the bed." $\Rightarrow \sim Q$

Statement-3: Which infers that "I am not sleepy" $\Rightarrow \sim P$

P	Q	$\sim P$	$\sim Q$	$P \rightarrow Q$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	0
1	1	0	0	1

3. Hypothetical Syllogism: The Hypothetical Syllogism rule state that if $P \rightarrow R$ is true whenever $P \rightarrow Q$ is true, and $Q \rightarrow R$ is true. It can be represented as the following notation:

Example:

Statement-1: If you have my home key then you can unlock my home. $P \rightarrow Q$

Statement-2: If you can unlock my home then you can take my money. $Q \rightarrow R$

Conclusion: If you have my home key then you can take my money. $P \rightarrow R$

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	0	0
1	1	1	1	1	1

4. Disjunctive Syllogism: The Disjunctive syllogism rule state that if $P \vee Q$ is true, and $\neg P$ is true, then Q will be true. It can be represented as:

Notation of Disjunctive syllogism:
$$\frac{P \vee Q, \neg P}{Q}$$

Example:

Statement-1: Today is Sunday or Monday. $\Rightarrow P \vee Q$

Statement-2: Today is not Sunday. $\Rightarrow \neg P$

Conclusion: Today is Monday. $\Rightarrow Q$

P	Q	$\neg P$	$P \vee Q$
0	0	1	0
0	1	1	1
1	0	0	1
1	1	0	1

5. Addition: The Addition rule is one the common inference rule, and it states that If P is true, then $P \vee Q$ will be true.

Notation of Addition:
$$\frac{P}{P \vee Q}$$

Example:

Statement: I have a vanilla ice-cream. $\implies P$

Statement-2: I have Chocolate ice-cream.

Conclusion: I have vanilla or chocolate ice-cream. $\implies (P \vee Q)$

P	Q	$P \vee Q$
0	0	0
1	0	1
0	1	1
1	1	1

6. Simplification: The simplification rule state that if $P \wedge Q$ is true, then Q or P will also be true. It can be represented as:

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

Proof by Truth-Table:

P	Q	$P \wedge Q$
0	0	0
1	0	0
0	1	0
1	1	1

7. Resolution: The Resolution rule state that if $P \vee Q$ and $\neg P \vee R$ is true, then $Q \vee R$ will also be true. It can be represented as

Notation of Resolution $\frac{P \vee Q, \neg P \vee R}{Q \vee R}$

Proof by Truth-Table:

P	$\neg P$	Q	R	$P \vee Q$	$\neg P \vee R$	$Q \vee R$
0	1	0	0	0	1	0
0	1	0	1	0	1	1
0	1	1	0	1	1	1
0	1	1	1	1	1	1
1	0	0	0	1	0	0
1	0	0	1	1	1	1
1	0	1	0	1	0	1
1	0	1	1	1	1	1

Inference Rules

Inference Rules

① Modus Ponens

$$\frac{P \rightarrow Q \quad P}{Q}$$

② Modus Tollens

$$\frac{P \rightarrow Q \quad \sim Q}{\sim P}$$

③ Hypothetical Syllogism: $P \rightarrow Q$
 $Q \rightarrow R$
 $\hline P \rightarrow R$

④ Disjunctive Syllogism: $P \vee Q$
 $\sim P$
 $\hline Q$

⑤ OR Introduction

$$\frac{P}{P \vee Q} \quad \text{or} \quad \frac{Q}{P \vee Q}$$

⑥ Simplification Rule

$$\frac{P \wedge Q}{P} \quad \text{or} \quad \frac{P \wedge Q}{Q}$$

⑦ Resolution

$$\frac{P \vee Q \quad \sim P \vee R}{Q \vee R}$$

① $(A \vee B) \wedge C \rightarrow E$
 $D \wedge F \rightarrow A$
 $\sim E \vee D$
 $B \wedge C$
 F
 $\hline A$

② $\frac{B \wedge C}{B} + \frac{B \wedge C}{C}$ {Simplification}

$\therefore B \wedge C$ are true.

③ $\frac{B}{(A \vee B)}$ {OR introduction}

④ we see $(A \vee B)$ true & C is true.

$\therefore (A \vee B) \wedge C$ is true.

⑤ $(A \vee B) \wedge C \rightarrow E$
 $(A \vee B) \wedge C$
 $\hline E$

⑥ $\sim E \vee D \equiv E \rightarrow D$ {Modus Ponens}
 E
 $\hline D$

⑦ we have F true & D true $\therefore (D \wedge F)$ is true.
 $D \wedge F \rightarrow A$
 $D \wedge F$
 $\hline A$ {Modus Ponens}

$$\begin{array}{l|l}
 \textcircled{2} (A \vee \neg C) \rightarrow B & \textcircled{1} \neg C \vee D \equiv C \rightarrow D \quad \{ \text{modus ponens} \} \\
 \neg C \vee D & \frac{C}{D} \\
 D \rightarrow A & \\
 \frac{C}{A \wedge B} & \textcircled{3} \frac{D \rightarrow A}{D} \quad \{ \text{MP} \} \\
 & A
 \end{array}$$

$$\textcircled{2} \frac{A}{A \vee \neg C} \quad \{ \text{OR Introduction} \}$$

$$\begin{array}{l}
 \textcircled{1} (A \vee \neg C) \rightarrow B \\
 \frac{(A \vee \neg C)}{B}
 \end{array}$$

$$\begin{array}{l}
 \textcircled{1} A \\
 \frac{B}{A \wedge B}
 \end{array}$$

① All Birds can fly: $\forall x \text{ Bird}(x) \rightarrow \text{fly}(x)$

② Every man respects his parent: $\forall x \text{ Man}(x) \rightarrow \text{respects}(x, \text{parent})$

③ Some boys play cricket: $\exists x \text{ Boys}(x) \rightarrow \text{play}(x, \text{cricket})$

④ Not all students like both Maths & Science.

$\neg \forall x [\text{Student}(x) \rightarrow (\text{like}(x, \text{Maths}) \wedge \text{like}(x, \text{Science}))]$

⑤ Only one student failed in Mathematics.

$\exists (x) [\text{Student}(x) \rightarrow \text{failed}(x, \text{Maths}) \wedge \forall (y) [\neg (x=y) \wedge \text{Student}(y) \rightarrow \neg \text{failed}(y, \text{Maths})]]$

⑥ Not all that glitters is gold: $\neg \forall x \text{ Glitters}(x) \rightarrow \text{gold}(x)$

⑦ Some real nos are rational: $\exists x \text{ Real}(x) \rightarrow \text{Rational}(x)$

Well-formed formula to Predicate Logic

Q.1) Sentences to Well formed Formulas in Predicate Logic

① Marcus was a man	$\text{man}(\text{Marcus})$
② Marcus was a Pompeian.	$\text{Pompeian}(\text{Marcus})$
③ All Pompeians were Romans.	$\forall x \text{ Pompeian}(x) \rightarrow \text{Roman}(x)$
④ Caesar was a Ruler	$\text{ruler}(\text{Caesar})$
⑤ All Romans were either loyal to Caesar or hated him.	$\forall x \text{ Roman}(x) \rightarrow \text{loyal to}(x, \text{Caesar}) \vee \text{hated}(x, \text{Caesar})$
⑥ Everyone is loyal to someone.	$\forall x \exists y \text{ loyal to}(x, y)$
⑦ People only try to assassinate rulers they are not loyal to.	$\forall x \forall y \text{ People}(x) \wedge \text{Ruler}(y) \wedge \text{TA}(x, y) \rightarrow \sim \text{Loyal}(x, y)$
⑧ Marcus tried to assassinate Caesar	$\text{TA}(\text{Marcus}, \text{Caesar})$
⑨ All men are people	$\forall x \text{ Man}(x) \rightarrow \text{People}(x)$

Q.1 Was Marcus loyal to Caesar?

Q.2 Was Marcus hate Caesar?

Ans 1 $\text{man}(\text{Marcus}) \xrightarrow{(9)} \text{People}(\text{Marcus}) \xrightarrow{(8)} \text{People}(\text{Marcus}) \wedge \text{TA}(\text{Marcus}, \text{Caesar}) \xrightarrow{(7)} \text{People}(\text{Marcus}) \wedge \text{ruler}(\text{Caesar}) \wedge \text{TA}(\text{Marcus}, \text{Caesar}) \xrightarrow{(3)} \sim \text{loyal}(\text{Marcus}, \text{Caesar}) \xrightarrow{(10)} \text{False}$

Ans 2 $\text{Pompeian}(\text{Marcus}) \xrightarrow{(2)} \text{Roman}(\text{Marcus}) \xrightarrow{(10)} \text{Roman}(\text{Marcus}) \xrightarrow{(5)} \sim \text{loyal}(\text{Marcus}, \text{Caesar}) \rightarrow \text{hated}(\text{Marcus}, \text{Caesar})$

Q. 2) ① Marcus was a man	$\text{man}(\text{Marcus})$
② Marcus was a Pompeian	$\text{Pompeian}(\text{Marcus})$
③ Marcus was born in 40 A.D	$\text{born}(\text{Marcus}, 40 \text{ A.D.})$
④ All men are mortal	$\forall x \text{ Man}(x) \rightarrow \text{Mortal}(x)$
⑤ All Pompeians died when the volcano erupted in 79 A.D	$\forall x \text{ Pompeian}(x) \rightarrow \text{died}(x, 79) \quad (S.i)$ $\text{erupt}(\text{volcano}, 79) \quad (S.ii)$
⑥ No mortal lives longer than 150 yrs.	$\forall x \forall t_1 \forall t_2 \text{ mortal}(x) \wedge \text{born}(x, t_1) \wedge \text{gt}(t_2 - t_1, 150) \rightarrow \text{dead}(x, t_2)$
⑦ It is now 1991	$\text{now} = 1991$
⑧ A live means not dead.	$\forall x \forall t (\text{alive}(x, t) \rightarrow \neg \text{dead}(x, t) \wedge (\text{dead}(x, t) \rightarrow \neg \text{alive}(x, t)))$
⑨ If someone dies, then he is dead at all later times.	$\forall x \forall t_1 \forall t_2 \text{ died}(x, t_1) \wedge \text{gt}(t_2, t_1) \rightarrow \text{dead}(x, t_2)$

Q. Was Marcus dead?

Ans) $\text{gt}(1991, 79) \xrightarrow{(7)} \text{gt}(\text{now}, 79) \xrightarrow{(2)} \text{Pompeian}(\text{Marcus}) \wedge \text{gt}(\text{now}, 79)$
 $\xrightarrow{(S.i)} \text{died}(\text{Marcus}, 79) \wedge \text{gt}(\text{now}, 79) \xrightarrow{(9)} \text{dead}(\text{Marcus}, \text{now})$

CNF form

- ① $\forall (x) \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
 ② $\text{food}(\text{apples}) \wedge \text{food}(\text{vegetables})$
 ③ $\forall x \forall y \neg \text{eat}(x, y) \vee \text{killed}(y) \vee \text{food}(y)$
 ④ $\text{eats}(\text{Anil}, \text{peanut}) \wedge \text{alive}(\text{Anil})$
 ⑤ $\forall (x) \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
 ⑥ $\text{like}(\text{John}, \text{peanuts})$

- ⑦ $\forall x \neg \text{killed}(x) \vee \text{alive}(x)$
 ⑧ $\forall x \text{alive}(x) \rightarrow \neg \text{killed}(x)$

Removing $\forall x$

- ① $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
 ② $\text{food}(\text{apples}) \wedge \text{food}(\text{vegetables})$
 ③ $\neg \text{eat}(x, y)$

Steps to Convert Logical Statement to CNF

Steps to Convert Logical Statement into CNF

Conjunctive normal form states that a formula is in CNF, if it is a conjunctive of one or more ^{Has one} clauses, where each clause is a disjunction of literals.

Eg: $(\neg A \vee \neg B \vee C) \wedge (\neg A \vee B \vee \neg C)$

Steps

→ Replace $a \rightarrow b$ with $\neg a \vee b$ & $(a \leftrightarrow b) \equiv (a \rightarrow b) \wedge (b \rightarrow a)$

→ Push negation inside. Eg.

① $\neg(\forall x p) \equiv \exists x \neg p$

② $\neg(a \vee b) = \neg a \wedge \neg b$

③ $\neg(\exists x p) \equiv \forall x \neg p$

④ $\neg(a \wedge b) = \neg a \vee \neg b$

→ Each quantifier should use diff var.

Eg: $\exists x p(x) \wedge \forall x q(x) \times$ ~~⑤~~ $\exists x p(x) \wedge \forall y q(y) \checkmark$

→ Each existential quant is ~~also~~ transformed into Skolem constant or func.

Eg ① $\exists x \text{Rich}(x)$: becomes $\text{Rich}(c_1)$, c_1 = Skolem constant.

② Everyone has a heart: $\forall x \text{Person}(x) \rightarrow \exists y (\text{Heart}(y) \wedge \text{Has}(x, y))$
becomes: $\forall x \text{Person}(x) \rightarrow \text{Heart}(H(x)) \wedge \text{Has}(x, H(x))$
 H = Skolem func/const.

→ Each universal quant is ~~replaced~~ removed.

Eg: $\forall x \text{Person}(x)$ becomes $\text{Person}(x)$

→ Distribute $\wedge$ over $\vee$

Eg: $(a \wedge b) \vee c = (a \vee c) \wedge (b \vee c)$

Eg: $A \rightarrow (B \leftrightarrow C)$

$\equiv \neg A \vee (B \leftrightarrow C) \equiv \neg A \vee ((B \rightarrow C) \wedge (C \rightarrow B))$

$\equiv (\neg A \vee (\neg B \vee C)) \wedge (\neg A \vee (\neg C \vee B)) \equiv (\neg A \vee \neg B \vee C) \wedge (\neg A \vee \neg C \vee B)$

Example: FOL to CNF

$$\forall x [\forall y \text{Animal}(y) \rightarrow \text{Loves}(x, y)] \rightarrow [\exists y \text{Loves}(y, x)]$$

Answers

Step 1: Eliminate bidirectional & implications

$$= \forall x \neg [\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

Step 2: Move negation inwards.

$$= \forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

Step 3: Standardize variables

$$= \forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{Loves}(z, x)]$$

Step 4: Skolemize

$$= \forall x [\text{Animal}(F(y)) \wedge \neg \text{Loves}(x, F(y))] \vee [\text{Loves}(G(z), x)]$$

Step 5: Remove universal quantifiers

$$= [\text{Animal}(F(y)) \wedge \neg \text{Loves}(x, F(y))] \vee [\text{Loves}(G(z), x)]$$

Step 6: Distribute

$$= [\text{Animal}(F(y)) \vee \text{Loves}(G(z), x)] \wedge [\neg \text{Loves}(x, F(y)) \vee \text{Loves}(G(z), x)]$$

Resolution

Resolution $\rightarrow$

- $\rightarrow$ Is a theorem proving technique that proceeds by building proofs by contradiction
- $\rightarrow$ Used when various statements are given & we need to prove a conclusion from them
- $\rightarrow$ Is a single inference rule which can efficiently operate on the CNF

Steps in Resolution

- $\rightarrow$ Conversion of facts into FOPL
- $\rightarrow$ " " FOPL to CNF (also remove $\forall$ & $\exists$)
- $\rightarrow$ Negate the statements needed to prove (proof by contradiction)
- $\rightarrow$ Draw Resolution graph (unification)

Q.1) ① John likes all kind of food

② Apple & vegetable are food

③ Anything Anyone eats & not killed is food

④ Anil eats peanuts & still alive.

⑤ Harry eats everything that Anil eats.

⑥ John likes peanuts.

Step 1: Facts $\rightarrow$ FOPL

$\forall x \text{ food}(x) \rightarrow \text{likes}(\text{John}, x)$

$\text{food}(\text{apple}) \wedge \text{food}(\text{vegetable})$

$\forall x \forall y \text{ eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$

$\text{eats}(\text{Anil}, \text{peanut}) \wedge \text{alive}(\text{Anil})$

$\forall x \text{ eat}(\text{Anil}, x) \rightarrow \text{eat}(\text{Harry}, x)$

$\text{likes}(\text{John}, \text{peanut})$

We can also say that: $\forall x \neg \text{killed}(x) \rightarrow \text{alive}(x)$

$\forall x \text{ alive}(x) \rightarrow \neg \text{killed}(x)$

Assume: $\text{likes}(\text{John}, \text{peanut})$ needs to be proved.

Step 2: FOPL to CNF

① $\neg \forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$

② $\text{food}(\text{apples}) \wedge \text{food}(\text{vegetables})$

③ $\forall x \forall y \text{ eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$

④ $\text{eats}(\text{Anil}, \text{peanut}) \wedge \text{alive}(\text{Anil})$

⑤ $\forall x \neg \text{eat}(\text{Anil}, x) \vee \text{eat}(\text{Harry}, x)$

⑥ $\text{likes}(\text{John}, \text{peanut})$

⑦ $\forall x \text{ killed}(x) \vee \text{alive}(x)$

⑧ $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$

Step 3

Renaming variables

$\forall y \forall z \text{ eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$

$\forall u \neg \text{eat}(\text{Anil}, u) \vee \text{eat}(\text{Harry}, u)$

$\forall g \text{ killed}(g) \vee \text{alive}(g)$

$\forall k \neg \text{alive}(k) \vee \neg \text{killed}(k)$

Step 5: Eliminate Quantifiers

① $\neg \text{food}(x) \vee \text{likes}(\text{John}, x)$

② $\text{food}(\text{apples})$

③ $\text{food}(\text{vegetables})$

④ $\neg \text{eat}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$

⑤ $\text{eats}(\text{Anil}, \text{peanut})$

⑥ $\text{alive}(\text{Anil})$

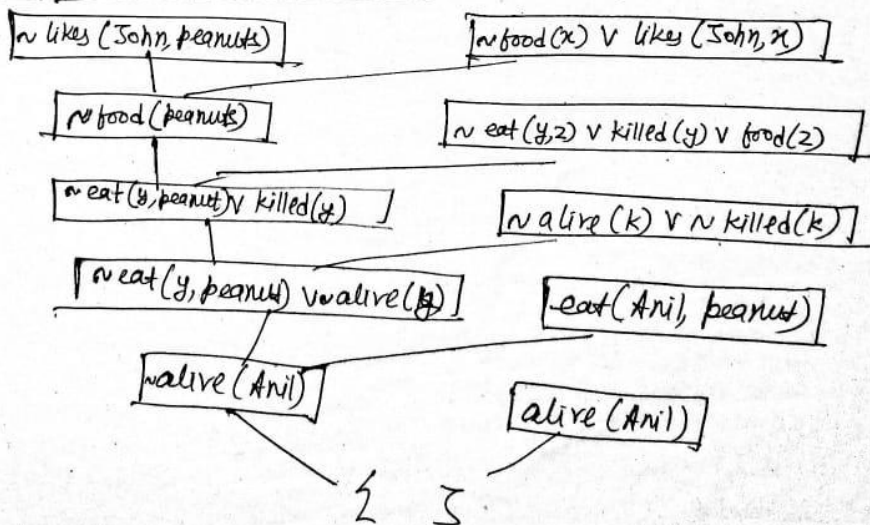
⑦ $\neg \text{eat}(\text{Anil}, u) \vee \text{eat}(\text{Harry}, u)$

⑧ $\text{likes}(\text{John}, \text{peanut})$

⑨ $\text{killed}(g) \vee \text{alive}(g)$

⑩ $\neg \text{alive}(k) \vee \neg \text{killed}(k)$

Step 6: Draw Resolution Tree



Empty set means. $\neg \text{likes}(\text{John}, \text{peanuts})$ is false.
 $\therefore \text{likes}(\text{John}, \text{peanuts})$ is True.

Example

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles. All of its missiles were sold to it by Colonel West, who is American. An enemy of American counts as hostile.

Prove By Resolution: West is criminal.

Answer

Step 1: Facts to FOL

Rule 1: It is crime for an American to sell weapons to hostile nations

$$\forall x, y, z [American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z)] \Rightarrow Criminal(x)$$

Rule 2: Nono has some missiles.

$$\exists x [Owns(Nono, x) \wedge Missiles(x)]$$

Rule 3: Nono, ~~owns~~ has missiles sold to West.

$$\forall x Missiles(x) \wedge Owns(Nono, x) \rightarrow Sells(West, x, Nono)$$

Rule 4: $\forall x Enemy(x, America) \rightarrow Hostile(x)$

Rule 5: American(West)

Rule 6: Enemy(Nono, America)

Rule 7: $\forall x Missile(x) \rightarrow Weapon(x)$

Step 1: $\forall x, y, z [American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z)] \rightarrow Criminal(x)$

2) Owns(Nono, M1)

3) Missile(M1)

4) $\forall x [Missile(x) \wedge Owns(Nono, x) \rightarrow Sells(West, x, Nono)]$

5) $\forall x Enemy(x, America) \rightarrow Hostile(x)$

6) American(West)

7) Enemy(Nono, America)

8) $\forall x Missile(x) \rightarrow Weapon(x)$

Step 2: FOL to CNF

i) Eliminate $\rightarrow$ & $\leftrightarrow$ & move negation inside

1) $\forall x, y, z [American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)]$

2) Owns(Nono, M1)

3) Missile(M1)

4) $\forall x [\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)]$

5) $\forall x \neg Enemy(x, America) \vee Hostile(x)$

6) American(West)

7) Enemy(Nono, America)

8) $\forall x \neg Missile(x) \vee Weapon(x)$

9) $\neg American(West)$

ii) No standardize var & Skolemize model
Remove universal quant

1) $\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$

2) Owns(Nono, M1)

3) Missile(M1)

4) $\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$

5) $\neg Enemy(x, America) \vee Hostile(x)$

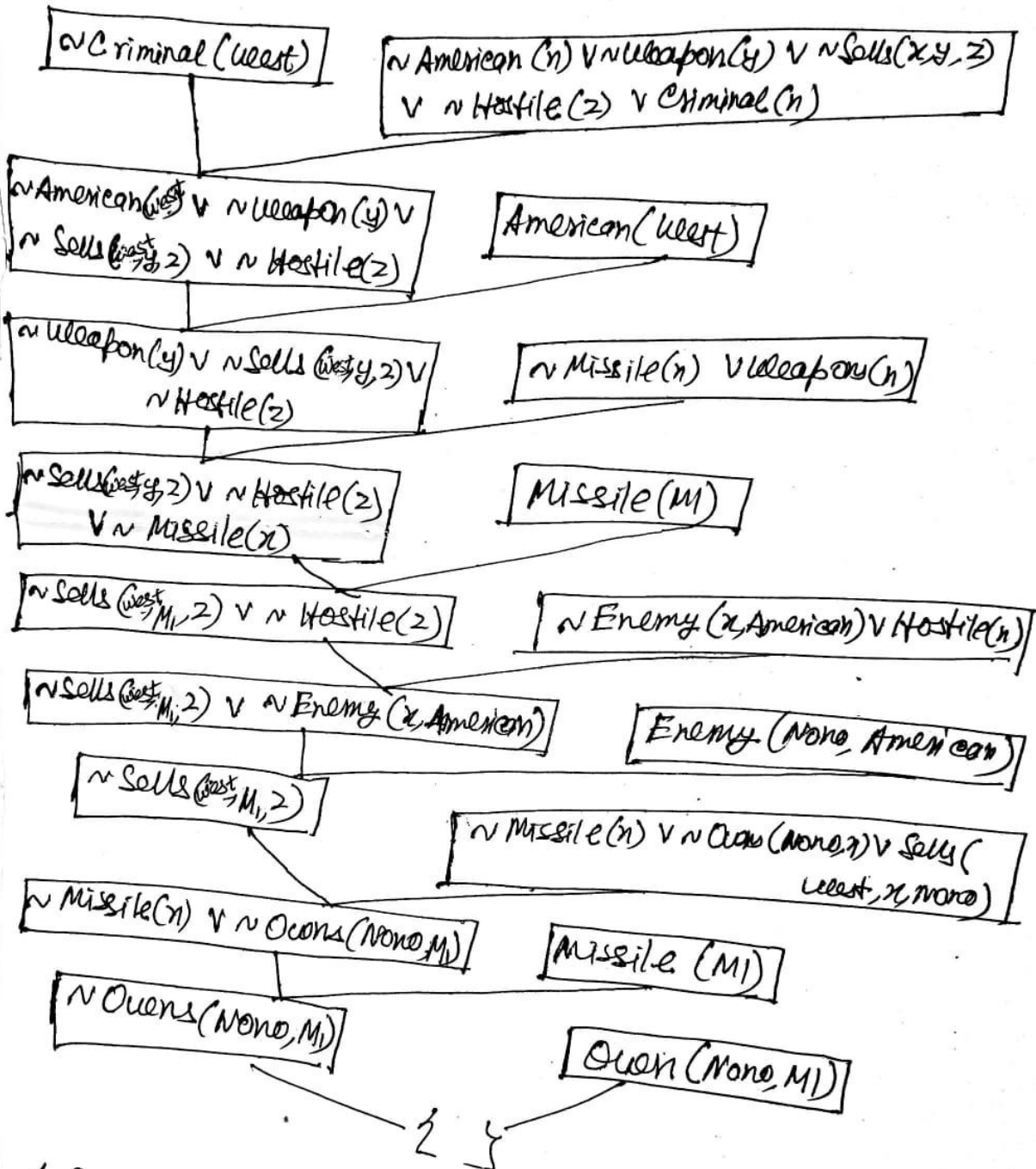
6) American(West)

7) Enemy(Nono, America)

8) $\neg Missile(x) \vee Weapon(x)$

Step 3: Resolution Tree.

To prove: $\text{West is Criminal} \equiv \text{Criminal}(\text{West})$



$\bot$ means, $\neg \text{Criminal}(\text{West})$ is false by contradiction.
 $\therefore \text{Criminal}(\text{West})$ is true.

Bayesian Networks

A **Bayesian Network** (also known as a **Bayes Network** or **Belief Network**) is a **probabilistic graphical model** that represents a set of variables and their conditional dependencies through a directed acyclic graph (DAG). Due to its feature of joint probability, the probability in Bayesian Belief Network is derived, based on a condition — $P(\text{attribute/parent})$ i.e probability of an attribute, true over parent attribute.

It is used for reasoning under uncertainty, where the relationships between variables are expressed in terms of probability theory.

Components of a Bayesian Network:

1. **Nodes (Variables):** Each node in the network represents a random variable, which can be discrete or continuous. These variables can represent various phenomena or events (e.g., weather, diseases, sensor readings).
2. **Edges (Dependencies):** The directed edges between nodes represent **conditional dependencies**. If there is a directed edge from node A to node B, it implies that **B is conditionally dependent** on A.
3. **Conditional Probability Distribution (CPD):** Each node has a **conditional probability distribution** that defines the probability of the node given its parent nodes. For example, the probability of an event occurring given certain other events.
4. **Joint Probability Distribution:** A Bayesian network encodes the joint probability distribution of the set of variables by factoring it into a product of conditional probabilities.

How Bayesian Networks Work:

1. **Inference:** Given some observed data (evidence), Bayesian networks can be used to **infer** the probabilities of unknown variables. This allows for reasoning about uncertain events based on known information.
2. **Learning:** Bayesian networks can also be **learned** from data, either **parameter learning** (estimating the CPD values) or **structure learning** (determining the network's structure).

Applications of Bayesian Networks:

1. **Medical Diagnosis:** Bayesian networks are used to model the relationships between symptoms, diseases, and medical tests. They allow medical experts to reason about the likelihood of a disease given the presence of symptoms and test results.
2. **Decision Support Systems:** In uncertain environments, Bayesian networks help in making decisions based on incomplete information by calculating the probabilities of various outcomes.
3. **Risk Analysis:** Bayesian networks can model risks and uncertainties in systems such as engineering, finance, and project management.
4. **Natural Language Processing (NLP):** Used in machine translation, speech recognition, and text prediction by modelling the probabilistic dependencies between words or phrases.

Advantages of Bayesian Networks:

1. **Handling Uncertainty:** Bayesian networks provide a formalism for reasoning with uncertain information.
2. **Modular:** They are flexible and can be easily extended with new variables or dependencies.
3. **Interpretability:** The graphical representation makes it easy to interpret the relationships between variables.

Limitations of Bayesian Networks:

1. **Computational Complexity:** Exact inference in large networks can be computationally expensive due to the large number of possible combinations of variables.
2. **Data Availability:** Learning the structure or parameters of a Bayesian network requires significant data, especially when dealing with complex networks.

Example of a Simple Bayesian Network: Imagine a simple network with the following variables:

- **Rain:** Whether it's raining (True/False).
- **Traffic:** Whether there is traffic (True/False).
- **Accident:** Whether there is an accident (True/False).

The network might look like this: ***Rain* → *Traffic* → *Accident***

Where:

- The probability of traffic is dependent on whether it's raining.
- The probability of an accident is dependent on both traffic and rain.

The CPDs might look like:

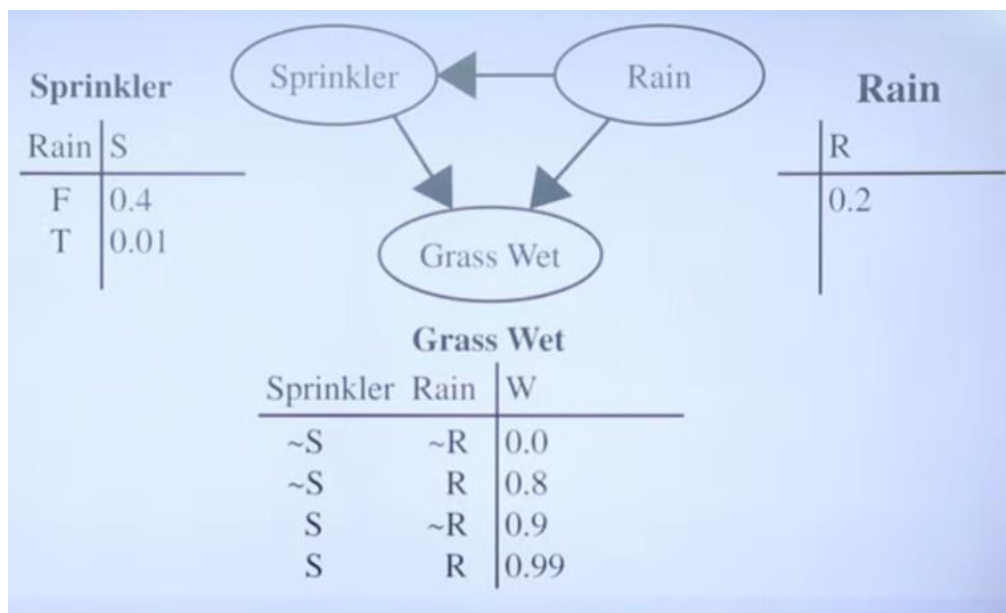
- **P(Traffic | Rain):** The probability of traffic given that it's raining.
- **P(Accident | Traffic, Rain):** The probability of an accident given the traffic and rain.

Bayesian Network Representation:

- **Rain** and **Traffic** have independent probabilities.
- **Traffic** depends on **Rain**, and **Accident** depends on both **Traffic** and **Rain**.

Question 1: Find the probability

- i) **when the grass is wet and there is sprinkler and rain.**
- ii) **When the grass is wet but no sprinkler and no rain**



Answer

R	$\sim R$
0.2	0.8

R	S	$\sim S$
F	0.4	0.6
T	0.01	0.99

Conditional Probability

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

S	R	W	$\sim W$
$\sim S$	$\sim R$	0	1
$\sim S$	R	0.8	0.2
S	$\sim R$	0.9	0.1
S	R	0.99	0.01

i) $P(W \cap S \cap R) = ?$

$$= P(W|S, R) \times P(S|R) \times P(R)$$

$$= 0.99 \times 0.01 \times 0.2$$

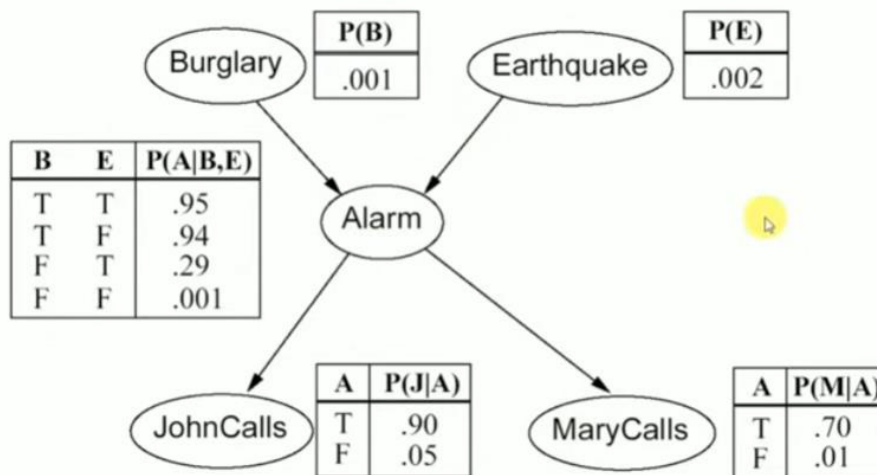
$$= 0.00198$$

ii) $P(W \cap \sim S \cap \sim R) = P(W|\sim S, \sim R) \times P(\sim S|\sim R) \times P(\sim R)$

$$= 0 \times 0.6 \times 0.8 = 0$$

Question 2: Find the probability that:

- The alarm has sounded but neither a burglary nor an earthquake has occurred, and both John and Merry called.
- John called.
- There is a burglary given that John and Merry calls.



- You have a new burglar alarm installed at home.
- It is fairly reliable at detecting burglary, but also sometimes responds to minor earthquakes.
- You have two neighbors, John and Merry, who promised to call you at work when they hear the alarm.
- John always calls when he hears the alarm, but sometimes confuses telephone ringing with the alarm and calls too.
- Merry likes loud music and sometimes misses the alarm.
- Given the evidence of who has or has not called, we would like to estimate the probability of a burglary.

Answer

$$\begin{aligned} \text{i) } P(A \cap \neg B \cap \neg E \cap J \cap M) &= P(A | \neg B, \neg E) \times P(\neg B) \times P(\neg E) \times P(J | A) \times P(M) \\ &= 0.001 \times 0.999 \times 0.992 \times 0.9 \times 0.7 \\ &= 0.00062 \end{aligned}$$

ii) John can call in 2 cases — he heard alarm
— he not heard alarm but thought telephone as alarm

$$= P(J | A) P(A) + P(J | \neg A) P(\neg A)$$

$$\begin{array}{l|l} \text{Now, } P(A) = 0.95 \times 0.001 \times 0.002 & P(\neg A) = 0.05 \times 0.001 \times 0.002 \\ + 0.94 \times 0.001 \times 0.998 & + 0.06 \times 0.001 \times 0.998 \\ + 0.29 \times 0.999 \times 0.002 & + 0.71 \times 0.999 \times 0.002 \\ + 0.001 \times 0.999 \times 0.998 & + 0.999 \times 0.999 \times 0.998 \\ \hline = 0.00252 & = 0.99748 \end{array}$$

$$\begin{aligned} &= 0.90 \times 0.00252 + 0.05 \times 0.99748 \\ &= 0.052143 \end{aligned}$$

Dempster-Shafer Theory (DST)

The **Dempster-Shafer Theory (DST)**, also known as the **Theory of Evidence**, is a mathematical framework for modelling and reasoning under uncertainty. It provides a method for combining all evidence from different sources and arriving at a degree of belief (also called "belief function") about an event or hypothesis.

Unlike traditional probability theory, DST allows for **partial belief** and **ignorance** when the information available is uncertain or incomplete.

This theory was released because of the following reason:

- Bayesian theory is only concerned about single evidence.
- Bayesian probability cannot describe ignorance.

Key Concepts in Dempster-Shafer Theory:

1. **Frame of Discernment (Θ):** The set of all possible outcomes (or hypotheses) in each situation.
Example: For a weather prediction system, the frame of discernment might be **{Rain, No Rain}**.
2. **Basic Probability Assignment (BPA):** A function that assigns probabilities to subsets of the frame of discernment. This function reflects the degree of belief in each subset based on available evidence.
 - a. Denoted as **$m(A)$** , where **A** is a subset of the frame of discernment.
 - b. Conditions:
 - i. $m(\emptyset) = 0$, meaning the empty set has zero probability.
 - ii. The sum of all BPAs must be: $\sum_{A \subseteq \Theta} m(A) = 1$.

3. **Belief Function (Bel):** The belief function gives the total amount of belief that supports a hypothesis based on the available evidence.

$$\text{Bel}(A) = \sum_{B \subseteq A} m(B), \text{ where } B \text{ is a subset of } A.$$

4. **Plausibility Function (Pl):** –The plausibility function represents how likely a hypothesis is, considering all evidence and excluding the possibility of disbelief.

$$\text{Pl}(A) = 1 - \text{Bel}(\neg A), \text{ where } \text{Bel}(\neg A) \text{ is the belief in the complement of } A.$$

5. **Disbelief Function (Disbelief or Dis):** The disbelief function represents the degree to which a hypothesis is not supported by the evidence. It is complementary to belief.

$$\text{Dis}(A) = 1 - \text{Pl}(A)$$

6. **Combination Rule (Dempster's Rule of Combination):** Dempster's Rule is a method for combining multiple pieces of evidence to derive a more complete belief.

- a. The combination rule combines two belief functions, **m1** and **m2**, into a new belief function **m**:

- $$m(A) = \sum_{B \cap C = A} \frac{m_1(B) \cdot m_2(C)}{1 - K}$$
 - **K** is a normalization factor to handle conflicts:

$$K = \sum_{B \cap C = \emptyset} m_1(B) \cdot m_2(C)$$
 - This rule is used when there is overlap between sources of evidence and enables the merging of independent sources of information.

Example: Scenario: Diagnosing a Disease (Flu vs. COVID-19)

You are trying to determine whether a patient has **Flu** or **COVID-19**, based on two sources of evidence:

1. **Symptom Evidence (Source 1):** Fever, cough, and fatigue.
2. **Test Results Evidence (Source 2):** Results from a rapid test.

1. Frame of Discernment (Θ)

The possible outcomes or hypotheses: $\Theta = \{\text{Flu}, \text{COVID-19}, \text{Neither}\}$

2. Basic Probability Assignments (BPA)

Each source provides the following belief values:

Source 1: Symptom Evidence

- $m_1(\text{Flu}) = 0.5$ (50% belief symptoms are due to Flu).
- $m_1(\text{COVID-19}) = 0.3$ (30% belief symptoms are due to COVID-19).
- $m_1(\text{Neither}) = 0.2$ (20% belief symptoms are unrelated to either).

Source 2: Test Results Evidence

- $m_2(\text{Flu}) = 0.4$ (40% belief test indicates Flu).
- $m_2(\text{COVID-19}) = 0.5$ (50% belief test indicates COVID-19).
- $m_2(\text{Neither}) = 0.1$ (10% belief test is inconclusive).

3. Combining Evidence (Dempster's Rule of Combination)

We combine m_1 and m_2 using Dempster's rule:

Step 1: Calculate Combined Beliefs

- $m(\text{Flu}) = \frac{\sum \text{belief combinations supporting Flu}}{1-K}$
 $= \frac{[m_1(\text{Flu}) \times m_2(\text{Flu}) + m_1(\text{Flu}) \times m_2(\text{Neither}) + m_1(\text{Neither}) \times m_2(\text{Flu})]}{1-K}$
 $= \frac{[0.5 \times 0.4 + 0.5 \times 0.1 + 0.2 \times 0.4]}{1-K}$
 $= \frac{0.2 + 0.05 + 0.08}{1-K} = \frac{0.33}{1-K}$
- Similarly, calculate $m(\text{COVID-19})$ and $m(\text{Neither})$.

Step 2: Resolve Conflicts (Normalization)

- Calculate conflict factor $K =$
 $\sum \text{belief combinations with no overlap:}$
 $K = m_1(\text{Flu}) \times m_2(\text{COVID-19}) + m_1(\text{COVID-19}) \times m_2(\text{Flu}) = 0.5 \times 0.5 + 0.3 \times 0.4 = 0.25 + 0.12 = 0.37$
- Normalize results by dividing $1 - K$.

4. Final Belief Distribution

After combining and normalizing:

- $m(\text{Flu}) = 0.40$ (40% belief patient has Flu).
- $m(\text{COVID-19}) = 0.50$ (50% belief patient has COVID-19).
- $m(\text{Neither}) = 0.10$ (10% belief symptoms are unrelated).

Interpretation: The combined evidence suggests a **higher belief (50%) that the patient has COVID-19**, though there is still a 40% belief that it could be Flu. Further tests or evidence may help reduce uncertainty.

Applications of Dempster-Shafer Theory:

1. **Decision Support Systems:** DST is widely used in fields where information is uncertain or incomplete, such as medical diagnosis, expert systems, and risk analysis.
2. **Sensor Fusion:** DST is effective in combining data from different sensors with varying levels of reliability, where conflicts in the data can arise.
3. **Artificial Intelligence:** DST helps AI systems reason with uncertain knowledge, such as autonomous vehicles using DST to combine sensor data for decision-making.
4. **Robotics:** In robotics, DST helps robots to process and combine sensory inputs to make decisions under uncertainty.

Advantages of Dempster-Shafer Theory:

1. **Handling Uncertainty:** DST can deal with both uncertainty and **ignorance** (lack of information), which is a major advantage over traditional probability theory.
2. **Flexibility:** DST allows for more **flexible reasoning** by representing partial beliefs and conflicts.
3. **Combining Evidence:** DST provides a formal way to combine conflicting evidence from different sources, making it useful in multi-source decision-making.

Limitations of Dempster-Shafer Theory:

1. **Computational Complexity:** Combining large amounts of evidence can lead to computational challenges.

2. **Conflict Handling:** The normalization factor KKK can be very small when there is significant conflict between sources, leading to loss of information.
3. **No clear rule for handling "negative" evidence or strongly conflicting data.**

Fuzzy Sets

A **fuzzy set** is a class of objects with **unprecise boundaries**. Unlike classical sets (where an element either belongs to the set or does not), fuzzy sets allow for **partial membership**. Each element in a fuzzy set has a degree of membership, which is a value between 0 and 1. **Key Concepts:**

- **Membership Function:** A function that assigns a degree of membership (between 0 and 1) to each element of the universe of discourse.
- **Universal Set:** The set of all possible elements in the domain being considered.
- **Degree of Membership (μ_A):** A value between 0 and 1 indicating how strongly an element belongs to the fuzzy set. For example, in a fuzzy set "tall people," a person who is 6 feet tall may have a degree of membership of 0.8.

Example:

Consider the fuzzy set "Tall People" in the domain of human height.

- **Universe of Discourse:** The set of all possible human heights: $\{4.5, 5.0, 5.5, 6.0, 6.5, 7.0\}$ feet.
- **Membership Function:**
 - $\mu_{\text{Tall}}(5.5) = 0.5$
 - $\mu_{\text{Tall}}(6.0) = 0.8$
 - $\mu_{\text{Tall}}(7.0) = 1.0$

In this case, 6 feet tall individuals are considered fairly tall with a membership value of 0.8, whereas 5.5 feet tall individuals only partially belong to the set with a membership value of 0.5.

Fuzzy Logic

Fuzzy Logic is a mathematical system that allows reasoning about **imprecise or vague concepts** by using fuzzy sets. It extends classical (binary) logic by allowing truth values to range between **completely true (1) and completely false (0)**. Instead of using only 0 or 1, fuzzy logic uses a range of values between 0 and 1 to represent the **degree of truth**. **Basic Concepts in Fuzzy Logic:**

- **Fuzzy Variables:** Variables that can take on a range of values, not just discrete ones. For instance, "temperature" can range from "cold" to "hot," with various degrees of "warm."
- **Fuzzy Rules:** The basis of fuzzy logic is a set of IF-THEN rules that describe how to map fuzzy inputs to fuzzy outputs. **Example:** IF the temperature is "high," THEN the fan speed should be "fast."
- **Fuzzy Inference System (FIS):** The framework that maps inputs to outputs based on fuzzy rules. It processes fuzzy input values and applies fuzzy logic operations to produce a fuzzy output.

Key Operations in Fuzzy Logic:

- **Fuzzy AND (Intersection):** The minimum of two membership values.
 - $\mu_{A \text{ AND } B}(x) = \min(\mu_A(x), \mu_B(x))$
- **Fuzzy OR (Union):** The maximum of two membership values.
 - $\mu_{A \text{ OR } B}(x) = \max(\mu_A(x), \mu_B(x))$
- **Fuzzy NOT (Negation):** The complement of a membership value.
 - $\mu_{\text{NOT } A}(x) = 1 - \mu_A(x)$

Example:

Let's say we want to control the speed of a fan based on the temperature. We define fuzzy sets for "Temperature" and "Fan Speed":

- **Temperature:** Cold, Warm, Hot.
- **Fan Speed:** Low, Medium, High.

The fuzzy rules could look like:

- IF Temperature is "Hot" THEN Fan Speed is "High".
- IF Temperature is "Warm" THEN Fan Speed is "Medium".
- IF Temperature is "Cold" THEN Fan Speed is "Low".

If the temperature is 75°F, it could have a membership value of 0.5 in "Warm" and 0.5 in "Hot." The fuzzy inference system combines these degrees of membership and outputs a fuzzy fan speed value, which is then defuzzified (explained below) to provide a crisp output.

Fuzzy logic Operations

Example 1: 2 fuzzy sets A & B. In A degree of membership for element 2,4,6,8 are 1,0.3,0.5,0.2. In B degree of membership for element 2,4,6,8 are 0.5,0.4,0.1,1. Find union, intersection, complement and difference between these.

$$\begin{aligned} \underline{A} &= \left\{ \frac{1}{2} + \frac{0.3}{4} + \frac{0.5}{6} + \frac{0.2}{8} \right\} \\ \underline{B} &= \left\{ \frac{0.5}{2} + \frac{0.4}{4} + \frac{0.1}{6} + \frac{1}{8} \right\} \end{aligned}$$

Solution:

UNION $\underline{A} \cup \underline{B} = \max\{\mu_{\underline{A}}(x), \mu_{\underline{B}}(x)\} = \left\{ \frac{1}{2} + \frac{0.4}{4} + \frac{0.5}{6} + \frac{1}{8} \right\}$	COMPLEMENT $\underline{A} = 1 - \mu_{\underline{A}}(x) = \left\{ \frac{0}{2} + \frac{0.7}{4} + \frac{0.5}{6} + \frac{0.8}{8} \right\}$
INTERSECTION $\underline{A} \cap \underline{B} = \min\{\mu_{\underline{A}}(x), \mu_{\underline{B}}(x)\} = \left\{ \frac{0.5}{2} + \frac{0.3}{4} + \frac{0.1}{6} + \frac{0.2}{8} \right\}$	$\underline{B} = 1 - \mu_{\underline{B}}(x) = \left\{ \frac{0.5}{2} + \frac{0.6}{4} + \frac{0.9}{6} + \frac{0}{8} \right\}$
DIFFERENCE $\underline{A} \setminus \underline{B} = \underline{A} \cap \overline{\underline{B}} = \left\{ \frac{0.5}{2} + \frac{0.3}{4} + \frac{0.5}{6} + \frac{0}{8} \right\}$	
$\underline{B} \setminus \underline{A} = \underline{B} \cap \overline{\underline{A}} = \left\{ \frac{0}{2} + \frac{0.4}{4} + \frac{0.1}{6} + \frac{0.8}{8} \right\}$	

Example 2:

$$\tilde{B}_1 = \left\{ \frac{1}{1.0} + \frac{0.75}{1.5} + \frac{0.3}{2.0} + \frac{0.15}{2.5} + \frac{0}{3.0} \right\}$$

$$\tilde{B}_2 = \left\{ \frac{1}{1.0} + \frac{0.6}{1.5} + \frac{0.2}{2.0} + \frac{0.1}{2.5} + \frac{0}{3.0} \right\}$$

Solution:

$$\bullet \tilde{B}_1 \cup \tilde{B}_2 = \left\{ \frac{1}{1.0} + \frac{0.75}{1.5} + \frac{0.3}{2.0} + \frac{0.15}{2.5} + \frac{0}{3.0} \right\}$$

$$\bullet \overline{\tilde{B}_1} = \left\{ \frac{0}{1.0} + \frac{0.25}{1.5} + \frac{0.7}{2.0} + \frac{0.85}{2.5} + \frac{1}{3.0} \right\}$$

$$\bullet \tilde{B}_1 \cap \tilde{B}_2 = \left\{ \frac{1}{1.0} + \frac{0.6}{1.5} + \frac{0.2}{2.0} + \frac{0.1}{2.5} + \frac{0}{3.0} \right\}$$

$$\bullet \overline{\tilde{B}_2} = \left\{ \frac{0}{1.0} + \frac{0.4}{1.5} + \frac{0.8}{2.0} + \frac{0.9}{2.5} + \frac{1}{3.0} \right\}$$

$$\bullet \tilde{B}_1 | \tilde{B}_2 = \tilde{B}_1 \cap \overline{\tilde{B}_2} = \left\{ \frac{0}{1.0} + \frac{0.4}{1.5} + \frac{0.3}{2.0} + \frac{0.15}{2.5} + \frac{0}{3.0} \right\}$$

$$\bullet \overline{\tilde{B}_1 \cup \tilde{B}_2} = \left\{ \frac{0}{1.0} + \frac{0.25}{1.5} + \frac{0.7}{2.0} + \frac{0.85}{2.5} + \frac{1}{3.0} \right\}$$

Example 3: It is necessary to compare two sensors based upon their detection levels and gain settings. The table of gain settings and sensor detection levels with a standard item being monitored providing typical membership values to represent the detection levels for each sensor is given in the table.

Now given the universe of discourse $X = \{0, 10, 20, 30, 40, 50\}$

and the membership functions for the two sensors in discrete form as

$$\tilde{D}_1 = \left\{ \frac{0}{0} + \frac{0.2}{10} + \frac{0.35}{20} + \frac{0.65}{30} + \frac{0.85}{40} + \frac{1}{50} \right\}$$

$$\tilde{D}_2 = \left\{ \frac{0}{0} + \frac{0.35}{10} + \frac{0.25}{20} + \frac{0.8}{30} + \frac{0.95}{40} + \frac{1}{50} \right\}$$

Gain	Detection Level Sensor 1	Detection Level Sensor 2
0	0	0
10	0.2	0.35
20	0.35	0.25
30	0.65	0.8
40	0.85	0.95
50	1	1

Solution:

$$(a) \mu_{\tilde{D}_1 \cup \tilde{D}_2}(x) = \left\{ \frac{0}{0} + \frac{0.35}{10} + \frac{0.35}{20} + \frac{0.8}{30} + \frac{0.95}{40} + \frac{1}{50} \right\}$$

$$(d) \mu_{\tilde{D}_1 \cup \overline{\tilde{D}_1}}(x) = \max\{\mu_{\tilde{D}_1}(x), \mu_{\overline{\tilde{D}_1}}(x)\}$$

$$= \left\{ \frac{1}{0} + \frac{0.8}{10} + \frac{0.65}{20} + \frac{0.65}{30} + \frac{0.85}{40} + \frac{1}{50} \right\}$$

$$(b) \mu_{\tilde{D}_1 \cap \tilde{D}_2}(x) = \left\{ \frac{0}{0} + \frac{0.2}{10} + \frac{0.25}{20} + \frac{0.65}{30} + \frac{0.85}{40} + \frac{1}{50} \right\}$$

$$(c) \mu_{\overline{\tilde{D}_1}}(x) = \left\{ \frac{1}{0} + \frac{0.8}{10} + \frac{0.65}{20} + \frac{0.35}{30} + \frac{0.15}{40} + \frac{0}{50} \right\}$$

$$(e) \mu_{\tilde{D}_1 \cap \overline{\tilde{D}_1}}(x)$$

$$= \left\{ \frac{0}{0} + \frac{0.2}{10} + \frac{0.35}{20} + \frac{0.35}{30} + \frac{0.15}{40} + \frac{0}{50} \right\}$$

$$\mu_{\overline{\tilde{D}_2}}(x) = \left\{ \frac{1}{0} + \frac{0.65}{10} + \frac{0.75}{20} + \frac{0.2}{30} + \frac{0.05}{40} + \frac{0}{50} \right\}$$

Algebraic sum & product, bounded sum & difference

Example 1: Find the algebraic sum, algebraic product, bounded sum and bounded difference of the given fuzzy sets

$$\tilde{A} = \left\{ \frac{0.2}{1} + \frac{0.3}{2} + \frac{0.4}{3} + \frac{0.5}{4} \right\}$$

$$\tilde{B} = \left\{ \frac{0.1}{1} + \frac{0.2}{2} + \frac{0.2}{3} + \frac{1}{4} \right\}$$

Solution

$$(a) \text{ Algebraic sum} = \mu_{\tilde{A} + \tilde{B}}(x) = [\mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)] - [\mu_{\tilde{A}}(x) \cdot \mu_{\tilde{B}}(x)]$$

$$= \left\{ \frac{0.3}{1} + \frac{0.5}{2} + \frac{0.6}{3} + \frac{0.5}{4} \right\} - \left\{ \frac{0.02}{1} + \frac{0.06}{2} + \frac{0.08}{3} + \frac{0.5}{4} \right\} = \left\{ \frac{0.28}{1} + \frac{0.44}{2} + \frac{0.52}{3} + \frac{0}{4} \right\}$$

$$(b) \text{ Algebraic product} = \mu_{\tilde{A} \cdot \tilde{B}}(x) = \mu_{\tilde{A}}(x) \cdot \mu_{\tilde{B}}(x) = \left\{ \frac{0.02}{1} + \frac{0.06}{2} + \frac{0.08}{3} + \frac{0.5}{4} \right\}$$

$$(c) \text{ Bounded sum} = \mu_{\tilde{A} \oplus \tilde{B}}(x) = \min[1, \mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)]$$

$$= \min \left\{ 1, \left\{ \frac{0.3}{1} + \frac{0.5}{2} + \frac{0.6}{3} + \frac{0.5}{4} \right\} \right\} = \left\{ \frac{0.3}{1} + \frac{0.5}{2} + \frac{0.6}{3} + \frac{0.5}{4} \right\}$$

$$(d) \text{ Bounded difference} = \mu_{\tilde{A} \ominus \tilde{B}}(x) = \max[0, \mu_{\tilde{A}}(x) - \mu_{\tilde{B}}(x)]$$

$$= \max \left\{ 0, \left\{ \frac{0.1}{1} + \frac{0.1}{2} + \frac{0.2}{3} + \frac{0.5}{4} \right\} \right\} = \left\{ \frac{0.1}{1} + \frac{0.1}{2} + \frac{0.2}{3} + \frac{0.5}{4} \right\}$$

Example 2: Find the algebraic sum, algebraic product, bounded sum, and bounded difference of the given fuzzy sets

$$\mu_{\tilde{T}} = \left\{ \frac{0}{0} + \frac{0.2}{1} + \frac{0.7}{2} + \frac{0.8}{3} + \frac{0.9}{4} + \frac{1}{5} \right\}$$

$$\mu_{\tilde{R}} = \left\{ \frac{0}{0} + \frac{0.1}{1} + \frac{0.3}{2} + \frac{0.2}{3} + \frac{0.4}{4} + \frac{0.5}{5} \right\}$$

Solution:

$$(a) \text{ Algebraic sum} = \left\{ \frac{0}{0} + \frac{0.3}{1} + \frac{1.0}{2} + \frac{1.0}{3} + \frac{1.3}{4} + \frac{1.5}{5} \right\} - \left\{ \frac{0}{0} + \frac{0.02}{1} + \frac{0.21}{2} + \frac{0.16}{3} + \frac{0.36}{4} + \frac{0.5}{5} \right\}$$

$$= \left\{ \frac{0}{0} + \frac{0.28}{1} + \frac{0.79}{2} + \frac{0.84}{3} + \frac{0.94}{4} + \frac{1}{5} \right\}$$

$$(b) \text{ Algebraic product} = \left\{ \frac{0}{0} + \frac{0.02}{1} + \frac{0.21}{2} + \frac{0.16}{3} + \frac{0.36}{4} + \frac{0.5}{5} \right\}$$

$$(c) \text{ Bounded sum} = \min \left\{ 1, \left\{ \frac{0}{0} + \frac{0.3}{1} + \frac{1.0}{2} + \frac{1.0}{3} + \frac{1.3}{4} + \frac{1.5}{5} \right\} \right\} = \left\{ \frac{0}{0} + \frac{0.3}{1} + \frac{1.0}{2} + \frac{1.0}{3} + \frac{1.0}{4} + \frac{1.0}{5} \right\}$$

$$(d) \text{ Bounded difference} = \max \left\{ 0, \left\{ \frac{0}{0} + \frac{0.1}{1} + \frac{0.4}{2} + \frac{0.6}{3} + \frac{0.5}{4} + \frac{0.5}{5} \right\} \right\}$$

Cartesian Product

Example: Find the cartesian product of A and B

Solution:

$$\underline{A} = \left\{ \frac{0.3}{x_1} + \frac{0.7}{x_2} + \frac{1}{x_3} \right\}$$

$$\underline{B} = \left\{ \frac{0.4}{y_1} + \frac{0.9}{y_2} \right\}$$

$$\begin{aligned} \mu_{\underline{R}}(x_1, y_1) &= \min[\mu_{\underline{A}}(x_1), \mu_{\underline{B}}(y_1)] \\ &= \min(0.3, 0.4) = 0.3 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{R}}(x_2, y_2) &= \min[\mu_{\underline{A}}(x_2), \mu_{\underline{B}}(y_2)] \\ &= \min(0.7, 0.9) = 0.7 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{R}}(x_1, y_2) &= \min[\mu_{\underline{A}}(x_1), \mu_{\underline{B}}(y_2)] \\ &= \min(0.3, 0.9) = 0.3 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{R}}(x_3, y_1) &= \min[\mu_{\underline{A}}(x_3), \mu_{\underline{B}}(y_1)] \\ &= \min(1, 0.4) = 0.4 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{R}}(x_2, y_1) &= \min[\mu_{\underline{A}}(x_2), \mu_{\underline{B}}(y_1)] \\ &= \min(0.7, 0.4) = 0.4 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{R}}(x_3, y_2) &= \min[\mu_{\underline{A}}(x_3), \mu_{\underline{B}}(y_2)] \\ &= \min(1, 0.9) = 0.9 \end{aligned}$$

$$\underline{R} = \begin{bmatrix} 0.3 & 0.3 \\ 0.4 & 0.7 \\ 0.4 & 0.9 \end{bmatrix}$$

Max Min composition & Max product composition

Example: Find the Max-Min composition and Max- Product composition

$$\underline{R} = \begin{matrix} & y_1 & y_2 \\ \begin{matrix} x_1 \\ x_2 \end{matrix} & \begin{bmatrix} 0.6 & 0.3 \\ 0.2 & 0.9 \end{bmatrix} \end{matrix} \quad \underline{S} = \begin{matrix} & z_1 & z_2 & z_3 \\ \begin{matrix} y_1 \\ y_2 \end{matrix} & \begin{bmatrix} 1 & 0.5 & 0.3 \\ 0.8 & 0.4 & 0.7 \end{bmatrix} \end{matrix}$$

Solution:

Max Min Composition:

$$\begin{aligned} \mu_{\underline{T}}(x_1, z_1) &= \max\{\min[\mu_{\underline{R}}(x_1, y_1), \mu_{\underline{S}}(y_1, z_1)], \\ &\quad \min[\mu_{\underline{R}}(x_1, y_2), \mu_{\underline{S}}(y_2, z_1)]\} \\ &= \max[\min(0.6, 1), \min(0.3, 0.8)] \\ &= \max(0.6, 0.3) = 0.6 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{T}}(x_1, z_2) &= \max[\min(0.6, 0.5), \min(0.3, 0.4)] \\ &= \max(0.5, 0.3) = 0.5 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{T}}(x_1, z_3) &= \max[\min(0.6, 0.3), \min(0.3, 0.7)] \\ &= \max(0.3, 0.3) = 0.3 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{T}}(x_2, z_1) &= \max[\min(0.2, 1), \min(0.9, 0.8)] \\ &= \max(0.2, 0.8) = 0.8 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{T}}(x_2, z_2) &= \max[\min(0.2, 0.5), \min(0.9, 0.4)] \\ &= \max(0.2, 0.4) = 0.4 \end{aligned}$$

$$\begin{aligned} \mu_{\underline{T}}(x_2, z_3) &= \max[\min(0.2, 0.3), \min(0.9, 0.7)] \\ &= \max(0.2, 0.7) = 0.7 \end{aligned}$$

$$\underline{T} = \underline{R} \circ \underline{S} = \begin{matrix} & z_1 & z_2 & z_3 \\ \begin{matrix} x_1 \\ x_2 \end{matrix} & \begin{bmatrix} 0.6 & 0.5 & 0.3 \\ 0.8 & 0.4 & 0.7 \end{bmatrix} \end{matrix}$$

Max Product Composition:

$$\begin{aligned}\mu_T(x_1, z_1) &= \max\{[\mu_R(x_1, y_1) \cdot \mu_S(y_1, z_1)], \\ &\quad [\mu_R(x_1, y_2) \cdot \mu_S(y_2, z_1)]\} \\ &= \max(0.6, 0.24) = 0.6\end{aligned}$$

$$\begin{aligned}\mu_T(x_1, z_2) &= \max[(0.6 \times 0.5), (0.3 \times 0.4)] \\ &= \max(0.3, 0.12) = 0.3\end{aligned}$$

$$\begin{aligned}\mu_T(x_1, z_3) &= \max[(0.6 \times 0.3), (0.3 \times 0.7)] \\ &= \max(0.18, 0.21) = 0.21\end{aligned}$$

$$\begin{aligned}\mu_T(x_2, z_1) &= \max[(0.2 \times 1), (0.9 \times 0.8)] \\ &= \max(0.2, 0.72) = 0.72\end{aligned}$$

$$\begin{aligned}\mu_T(x_2, z_2) &= \max[(0.2 \times 0.5), (0.9 \times 0.4)] \\ &= \max(0.1, 0.36) = 0.36\end{aligned}$$

$$\begin{aligned}\mu_T(x_2, z_3) &= \max[(0.2 \times 0.3), (0.9 \times 0.7)] \\ &= \max(0.06, 0.63) = 0.63\end{aligned}$$

$$\tilde{T} = \begin{matrix} & \begin{matrix} z_1 & z_2 & z_3 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \end{matrix} & \begin{bmatrix} 0.6 & 0.3 & 0.21 \\ 0.72 & 0.36 & 0.63 \end{bmatrix} \end{matrix}$$

Membership value for triangles using inference approach

Example: Using the inference approach, find the membership values for the triangular shapes I, R, E, IR, T for a triangle with angles 45, 55 and 80 degrees.

Solution:

Let the universe of discourse be

$$U = \{(X \ Y \ Z) : X = 80^\circ \geq Y = 55^\circ \geq Z = 45^\circ \text{ and } X + Y + Z = 80^\circ + 55^\circ + 45^\circ = 180^\circ\}$$

Membership value of isosceles triangle, I :

$$\begin{aligned}\mu_I &= 1 - \frac{1}{60^\circ} \min(X - Y, Y - Z) \\ &= 1 - \frac{1}{60^\circ} \min(80^\circ - 55^\circ, 55^\circ - 45^\circ) \\ &= 1 - \frac{1}{60^\circ} \min(25^\circ, 10^\circ) \\ &= 1 - \frac{1}{60^\circ} \times 10^\circ \\ &= 1 - 0.1667 = 0.833\end{aligned}$$

Membership value of Right-Angle triangle, R :

$$\begin{aligned}\mu_R &= 1 - \frac{1}{90^\circ} |X - 90^\circ| \\ &= 1 - \frac{1}{90^\circ} |80^\circ - 90^\circ| \\ &= 1 - \frac{1}{90^\circ} \times 10^\circ = 0.889\end{aligned}$$

Membership value of Equilateral triangle, E :

$$\begin{aligned}\mu_E &= 1 - \frac{1}{180^\circ} (X - Z) \\ &= 1 - \frac{1}{180^\circ} (80^\circ - 45^\circ) \\ &= 1 - \frac{1}{180^\circ} \times 35^\circ = 0.8056\end{aligned}$$

Membership value of Isosceles and Right-Angle Triangle, IR :

$$\begin{aligned}\mu_{IR} &= \min[\mu_I, \mu_R] \\ &= \min[0.833, 0.889] \\ &= 0.833\end{aligned}$$

Membership value of Other Triangles, T :

$$\begin{aligned}\mu_T &= \min[1 - \mu_I, 1 - \mu_E, 1 - \mu_R] \\ &= \min[0.167, 0.1944, 0.111] \\ &= 0.111\end{aligned}$$

$\mu_I = 0.833$
 $\mu_E = 0.8056$
 $\mu_R = 0.889$

Defuzzification

Defuzzification is the process of converting fuzzy output values (which range from 0 to 1) into a single crisp value that can be used in practical applications, such as controlling a device. **Common Defuzzification Methods:**

1. **Centroid Method:** Calculates the centre of gravity of the area under the membership function curve.
2. **Maximum Membership:** Chooses the output value with the highest degree of membership.
3. **Mean of Maxima:** Averages the outputs that have the maximum membership values.

Applications of Fuzzy Sets and Fuzzy Logic

1. **Control Systems:** Used in devices like air conditioners, washing machines, and microwave ovens to manage varying inputs (e.g., temperature, pressure) and adjust output in a human-like manner.
2. **Decision-Making Systems:** Employed in expert systems, decision support systems, and artificial intelligence applications where decisions are based on uncertain or vague information (e.g., medical diagnosis).
3. **Image Processing:** Helps in tasks such as image segmentation, noise reduction, and edge detection, where pixel values may not be well-defined or precise.
4. **Consumer Electronics:** Used in robotics, camera autofocus, and household appliances to interpret vague inputs and provide smoother, more adaptable responses.
5. **Data Classification:** Applied in data mining and machine learning to classify data into categories when exact boundaries are not clear, enhancing performance in systems that need to handle ambiguity.

Advantages of Fuzzy Sets and Fuzzy Logic

1. **Handles Uncertainty and Vagueness:** Fuzzy logic is ideal for dealing with imprecise or incomplete data, representing vague concepts (e.g., "high temperature", "tall person") with degrees of membership.
2. **Human-Like Reasoning:** Mimics human reasoning by dealing with approximate rather than exact reasoning, making systems more intuitive and adaptive.
3. **Flexibility:** Easily integrates into existing systems to improve performance without needing exact mathematical precision, making it highly adaptable for real-world applications.
4. **Simple to Implement:** Fuzzy logic systems are easier to design and implement compared to traditional algorithms for handling uncertainty, especially when working with rule-based systems.
5. **Improved Performance:** Can provide more accurate and efficient decision-making in systems where traditional binary logic would fail, especially in control and automation tasks.

Disadvantages of Fuzzy Sets and Fuzzy Logic

1. **Complexity in Design:** While fuzzy logic is easy to implement in certain applications, designing the rules and membership functions can be complex and time-consuming, especially in large systems.
2. **Lack of Precision:** The results in fuzzy logic are not always precise, as it works with approximate values, which can sometimes lead to less accurate outcomes in some scenarios.
3. **Computational Overhead:** The calculation of fuzzy membership functions and rule evaluation can introduce computational overhead, especially when dealing with large data sets.

4. **Lack of Standardization:** There is no universal framework or standard for defining fuzzy sets and membership functions, leading to inconsistencies across different applications.
5. **Difficulty in Modelling Complex Systems:** For highly intricate or dynamic systems, the design of fuzzy rules may become cumbersome, and it may be challenging to achieve optimal performance.

MODULE 5: NATURAL LANGUAGE PROCESSING

Contents

1. Learning.....	1
2. Supervised Learning.....	2
3. Classification.....	3
4. Regression.....	4
5. Unsupervised Learning.....	5
6. Clustering in Unsupervised Learning	6
7. Reinforcement Learning	6
8. Natural Language Processing.....	7
9. Artificial Neural Networks (ANNs)	9
10. Syntactic Processing.....	10
11. Semantic Analysis	11
12. Discourse and Pragmatic Processing.....	11
13. Representing and Using Domain Knowledge	12
14. Expert System Shells.....	12
15. Knowledge Acquisition	12

Learning

- 1. Inductive Learning:** Inductive learning involves generalizing from specific examples to broader concepts. It focuses on identifying patterns in a dataset to create a model that can predict outcomes for unseen instances. **Applications:** Classifying emails as spam or not spam, predicting house prices based on historical data.
 - Utilizes training data to infer general rules or relationships.
 - Examples include supervised learning algorithms like decision trees, support vector machines, and neural networks.
- 2. Learning Decision Trees:** Decision tree learning is a method used for classification and regression tasks. It represents decisions and their possible consequences as a tree structure, where each node represents a feature or attribute, branches represent decision rules, and leaves represent outcomes. **Applications:** Customer segmentation, medical diagnosis, credit scoring.
 - Constructs a tree based on the feature that provides the highest information gain or minimizes impurity (e.g., Gini impurity or entropy).
 - Easy to interpret and visualize.
- 3. Explanation-Based Learning (EBL)** Explanation-based learning is a form of learning that uses background knowledge and explanations to generalize from specific instances. It focuses on understanding why a particular example is correct and using that understanding to improve generalization. **Applications:** Natural language understanding, learning concepts in educational software.

- The learner generates explanations for observed behaviour and extracts general rules from these explanations.
 - Often involves deductive reasoning and requires a rich knowledge base.
4. **Learning Using Relevance Information:** This approach involves using additional information to improve learning efficiency and accuracy by focusing on relevant features or instances. It emphasizes filtering out irrelevant data that may hinder the learning process. **Applications:** Document retrieval systems, personalized recommendations, and medical diagnosis where only relevant patient history is considered
 - Can use techniques like feature selection or instance selection to identify and retain only the most relevant data points.
 - Helps in reducing noise and dimensionality in the dataset.
 5. **Neural Net Learning:** Neural network learning is inspired by the structure and functioning of the human brain. It involves the use of interconnected nodes (neurons) organized in layers to learn complex patterns and relationships in data. **Applications:** Image recognition, natural language processing, speech recognition, and game playing (e.g., AlphaGo).
 - Capable of handling large datasets and learning non-linear relationships.
 - Involves techniques like backpropagation for training the network by adjusting weights based on errors.
 6. **Genetic Learning:** Genetic learning is inspired by the principles of natural selection and genetics. It uses evolutionary algorithms to optimize solutions by simulating the process of evolution through selection, crossover, and mutation. **Applications:** Feature selection, optimization problems in engineering, game strategies, and evolving neural network architectures.
 - Population-based approach, where multiple potential solutions (individuals) evolve over generations.
 - Effective in solving complex optimization problems where traditional methods may struggle.

Supervised Learning

Supervised learning is a type of machine learning where an algorithm is trained on labelled data, meaning the input data is paired with the correct output. The goal is for the model to learn from this data and make accurate predictions on new, unseen data. **Key Concepts:**

- **Labelled Data:** The training data consists of input-output pairs (e.g., images labelled as "cat" or "dog").
- **Training:** The model learns by mapping inputs to the correct outputs through iterative learning.
- **Loss Function:** Measures how far the model's predictions are from the true outputs. The model minimizes this function during training.
- **Prediction:** After training, the model can predict outputs for new inputs.

Steps in Supervised Learning

1. **Data Collection:** Gather a labelled dataset with input-output pairs (e.g., images with labels, sales data with outcomes).
2. **Data Preprocessing:** Clean and prepare the data by handling missing values, normalizing features, and splitting into training and test sets.
3. **Feature Selection/Extraction:** Identify the most relevant features or create new features that will help the model make accurate predictions.

4. **Model Selection:** Choose an appropriate supervised learning algorithm (e.g., linear regression, decision tree, SVM, etc.) based on the problem type (classification or regression).
5. **Model Training:** Feed the training data into the chosen algorithm, allowing it to learn patterns from the data by adjusting its parameters.
6. **Model Evaluation:** Test the model using a separate test set to evaluate its performance using metrics such as accuracy, precision, recall, or mean squared error (MSE).
7. **Hyperparameter Tuning:** Optimize the model's hyperparameters (e.g., learning rate, number of layers) to improve its performance.
8. **Prediction:** Use the trained model to make predictions on new, unseen data.
9. **Model Deployment:** Implement the model into a system where it can be used to make real-time predictions or decisions.
10. **Model Monitoring and Maintenance:** Continuously monitor the model's performance over time and retrain it if necessary to maintain accuracy.

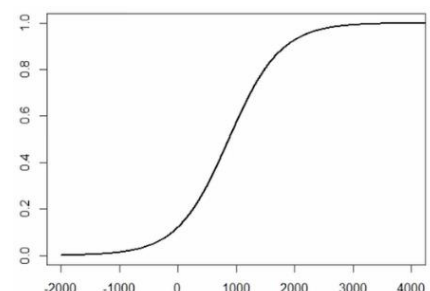
Challenges in Supervised Learning:

1. **Data Labelling:** Requires a large amount of labelled data, which can be expensive and time-consuming to obtain.
2. **Overfitting:** The model may perform well on training data but poorly on new, unseen data if it memorizes rather than generalizes.
3. **Imbalanced Data:** When one class is overrepresented in the data, the model may become biased towards that class.
4. **Scalability:** Training models on very large datasets can be computationally expensive and time-consuming.
5. **Noise in Data:** Errors or inconsistencies in the labelled data can lead to incorrect learning and poor model performance.
6. **Limited Generalization:** A model trained on one dataset may not perform well on a different dataset if the data distributions are different.
7. **Feature Engineering:** Identifying the right features can be difficult and may require domain expertise.

Classification

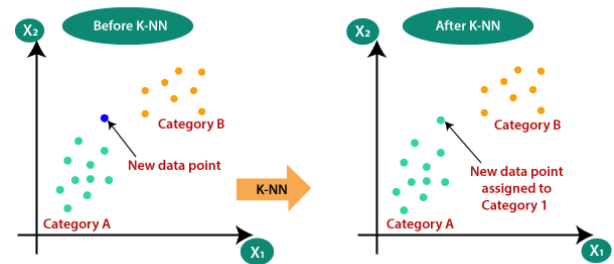
Classification is a type of supervised learning where the goal is to predict a categorical label or class for a given input based on the patterns learned from labelled data. The input is assigned to one of several predefined categories.

1. **Logistic Regression:** Statistical models that use logistic functions to model binary dependent variables. Outputs probabilities and can be used for binary classification tasks. Use information: Email search spam.

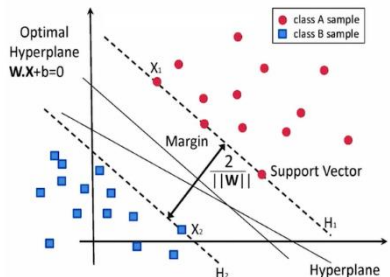


2. **Decision Trees:** Decision tree model. Each node represents a feature (or attribute), each branch represents a rule, and each leaf represents a result. Reference: Customer segmentation.

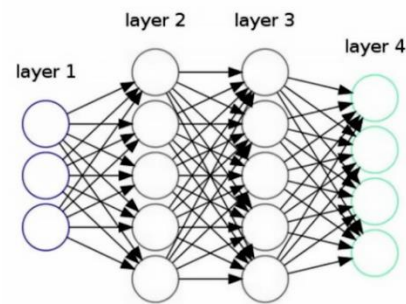
3. **k-Nearest Neighbours (k-NN):** Nonparametric classification technique that assigns labels to points based on their k-nearest training examples within a particular space. Makes use of a distance metric, usually the Euclidean distance, to determine neighbours. Using data: Handwritten digit recognition.



4. **Support Vector Machines (SVM):** The educational supervision model allows it to have the best visibility in separating different classes. Good performance at high altitudes. Data usage: Image classification.



5. **Random Forest:** An ensemble method that combines multiple decision trees to improve accuracy and reduce overfitting. It aggregates the predictions of several trees to make the final decision.
6. **Neural Networks:** Especially with deep learning, neural networks are used for complex classification tasks like image recognition, where input-output relationships are highly non-linear.

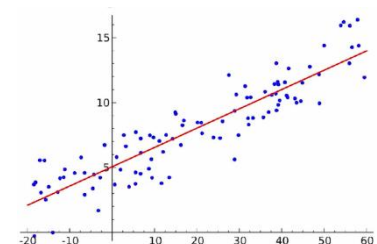


Regression

Regression is a type of supervised learning task that aims to predict a continuous numerical value based on input features. Unlike classification, where the output is discrete, regression deals with estimating relationships among variables and forecasting continuous outcomes.

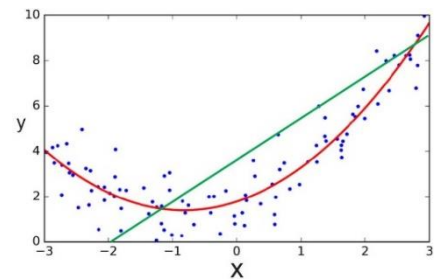
- **Dependent Variable:** The output variable that you are trying to predict (e.g., house prices).
- **Independent Variables:** The input features used to make predictions (e.g., size of the house, location, number of bedrooms).
- **Loss Function:** Measures the difference between the predicted values and the actual values (commonly Mean Squared Error).
- **Training:** The process of adjusting the model parameters to minimize the loss function using historical data.

1. **Linear Regression:** Determine the relationship of dependent with one or more independent variables by fitting a linear equation to observed data. Use data: Predict home values based on various characteristics of home values.



2. **Multiple Linear Regression:** An extension of linear regression that uses multiple independent variables to predict a dependent variable. **Use Cases:** Predicting a person's weight based on height, age, and diet.

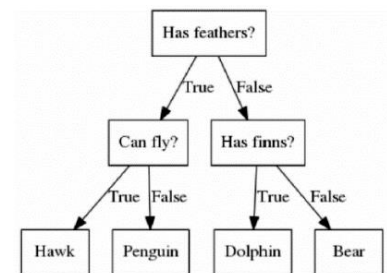
3. **Polynomial Regression:** By specifying polynomial terms of input features, it can achieve linear regression. It allows modelling non-linear relationships between these input features. **Possible use:** model growth rates in biology or finance.



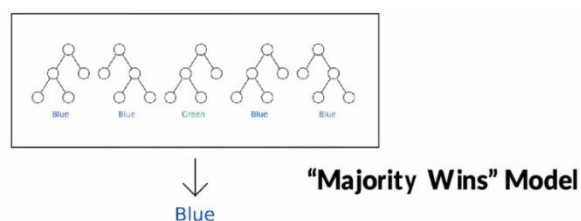
4. **Ridge Regression (L2 Regularization):** A linear regression model that adds a regularization term (penalty) to the cost function to reduce overfitting by minimizing the impact of less significant features

5. **Lasso Regression:** Like Ridge regression, different choices can be made by using the L1 constant and reducing some coefficients to zero. **Use data:** Select the most useful features in the data with multiple estimates.

6. **5. Decision Trees Regression:** The algorithm splits the data into subsets depending on feature value and computes the mean value of the target variable for each subset to construct a model tree. **Usage data:** Based on usage patterns and environment to predict device lifetime.



7. **Random Forest Regression:** It is an ensemble approach that involves large numbers of decision trees to multiply the effectiveness of any single prediction. Each tree is trained on a random subset of the data, and the final prediction is the mean prediction of the individual trees.



Unsupervised Learning

Unsupervised learning is a type of machine learning where the model is trained on data without labelled outcomes. The goal is to discover hidden patterns or intrinsic structures within the input data. Unlike supervised learning, there is no predefined output, and the model learns from the raw data itself. **Key Concepts:**

- **No Labels:** Data used in unsupervised learning does not have associated target outputs or labels.
- **Data Exploration:** The focus is on exploring the data and finding relationships, clusters, or patterns.
- **Dimensionality Reduction:** Techniques used to reduce the number of features while retaining essential information.

Applications of Unsupervised Learning

1. **Market Segmentation:** Grouping customers based on purchasing behaviour to tailor marketing strategies.
2. **Anomaly Detection:** Identifying fraudulent transactions or network intrusions by detecting outliers in data.
3. **Recommendation Systems:** Suggesting products or content to users based on similar preferences found in data.
4. **Customer Profiling:** Understanding customer behaviour patterns to improve user experience.
5. **Image Compression:** Reducing the size of images while retaining important features using dimensionality reduction techniques.
6. **Document Clustering:** Grouping similar documents together for information retrieval or topic modelling.

Challenges in Unsupervised Learning

1. **Evaluation:** Lack of ground truth makes it difficult to evaluate the quality of results.
2. **Interpretability:** Understanding and interpreting the patterns or clusters found can be complex.
3. **Scalability:** Some algorithms may struggle to scale effectively with large datasets.
4. **Choosing the Right Algorithm:** Selecting the appropriate technique for a specific task can be challenging without labelled data.
5. **Data Quality:** The effectiveness of unsupervised learning heavily depends on the quality and preprocessing of the input data.

Clustering in Unsupervised Learning

Clustering is a type of unsupervised learning that involves grouping a set of objects (data points) into clusters based on their similarities. The objective is to ensure that objects within the same cluster are more like each other than to those in other clusters. Clustering helps in identifying natural groupings in data, making it easier to analyse and interpret. **Key Concepts:**

- **Distance Metrics:** Measures used to quantify the similarity or dissimilarity between data points. Common metrics include Euclidean distance, Manhattan distance, and cosine similarity.
 - **Centroids:** The central point of a cluster, often used in algorithms like K-Means to represent the cluster.
 - **Inertia:** A measure of how tightly the clusters are packed together, often used to assess the quality of the clustering.
1. **K-Means Clustering:** Partitions the data into K clusters by assigning each data point to the nearest centroid and updating the centroids based on the mean of the points in each cluster. **Use Cases:** Customer segmentation, image compression. **Limitations:** Requires the number of clusters (K) to be specified in advance and is sensitive to outliers.
 2. **Hierarchical Clustering:** Builds a hierarchy of clusters using either a bottom-up approach (agglomerative) or a top-down approach (divisive). The result is often represented in a dendrogram. **Use Cases:** Taxonomy classification, social network analysis. **Limitations:** Computationally expensive for large datasets.

Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent takes actions in this environment to maximize a cumulative

reward over time. Unlike supervised learning, where the model is trained on labeled data, in reinforcement learning, the agent learns from the consequences of its actions.

- **Agent:** The learner or decision-maker that interacts with the environment.
- **Environment:** The setting in which the agent operates. The agent observes the state of the environment and takes actions.
- **State:** A representation of the current situation of the agent within the environment.
- **Action:** The choices available to the agent to interact with the environment.
- **Reward:** A feedback signal received by the agent after taking an action in a particular state, indicating the success of that action.
- **Policy:** A strategy that the agent employs to determine which actions to take in different states. It can be deterministic or stochastic.
- **Value Function:** A function that estimates the expected cumulative reward that can be obtained from a given state or state-action pair.

Applications of Reinforcement Learning:

1. **Game Playing:** RL has been successfully applied in games like chess, Go, and video games (e.g., AlphaGo and OpenAI's Dota 2 agent).
2. **Robotics:** Training robots to perform tasks such as walking, grasping objects, or navigating environments.
3. **Autonomous Vehicles:** Making real-time decisions based on sensor inputs to navigate and respond to dynamic environments.
4. **Healthcare:** Personalizing treatment plans for patients based on their responses to different interventions.
5. **Recommendation Systems:** Adapting recommendations based on user interactions and feedback.

Challenges in Reinforcement Learning:

1. **Exploration vs. Exploitation:** Balancing the need to explore new actions (to discover their rewards) with the need to exploit known actions that yield high rewards.
2. **Sample Efficiency:** Many RL algorithms require a large number of interactions with the environment to learn effectively, which can be impractical in real-world applications.
3. **Sparse Rewards:** In some environments, the agent may receive very few rewards, making it difficult to learn effective policies.
4. **Generalization:** Ensuring that learned policies can generalize well to unseen states or environments.
5. **Stability and Convergence:** Ensuring that the learning process is stable and converges to optimal policies can be challenging, especially in complex environments.

Natural Language Processing

Natural Language Processing (NLP) is a subfield of Artificial Intelligence (AI) focused on enabling machines to understand, interpret, and generate human language in a way that is both meaningful and useful. NLP combines linguistics, computer science, and AI to bridge the gap between human communication and machine understanding. **Key Components of NLP:**

1. **Tokenization:** Breaking down a sentence into words or smaller parts called tokens.
2. **Lemmatization and Stemming:** Reducing words to their root form or base meaning.
 - a. **Stemming:** Trimming words to their base form (e.g., "running" to "run").
 - b. **Lemmatization:** Reducing words to their dictionary form (e.g., "better" to "good").
3. **Stop Words Removal:** Removing common words like 'is', 'the', 'and' which add little meaning.

4. **Named Entity Recognition (NER):** Identifying proper nouns like names of people, places, organizations, etc.
5. **Part of Speech (POS) Tagging:** Identifying the grammatical role of each word (e.g., noun, verb, adjective).
6. **Sentiment Analysis:** Determining the emotional tone behind a piece of text (positive, negative, or neutral).
7. **Text Classification:** Assigning predefined categories to text (e.g., spam detection in emails).
8. **Word Embeddings:** Representing words in a numerical format, often used for understanding semantic similarities between words (e.g., Word2Vec, GloVe).

Applications of NLP:

1. **Chatbots and Virtual Assistants:** Provide automated, real-time customer support and perform tasks based on voice commands.
2. **Machine Translation:** Automatically translate languages (e.g., Google Translate), facilitating cross-lingual communication.
3. **Sentiment Analysis:** Analyse text data to understand opinions in social media, reviews, or feedback.
4. **Text Summarization:** Generate concise summaries of long documents for faster information consumption.
5. **Speech Recognition:** Convert spoken language into text for transcription or interaction with systems.
6. **Spam Detection:** Classify and filter out unwanted or harmful emails using text analysis.
7. **Healthcare:** Assist in medical document analysis, summarization of reports, and extracting useful patient information.

Advantages of NLP:

1. **Automation of Routine Tasks:** Reduces human effort in text processing, like customer service and translation.
2. **Enhanced Customer Experience:** Provides faster, personalized support via chatbots and virtual assistants.
3. **Better Insights from Data:** Helps extract meaningful information from large volumes of unstructured text data.
4. **Improved Decision-Making:** Facilitates better business decisions through sentiment analysis and text mining.
5. **Accessibility:** Enables interaction with devices through natural language, improving accessibility for users with disabilities.
6. **Scalability:** NLP solutions can handle vast amounts of text data efficiently, making them scalable across industries.
7. **Cross-Language Communication:** Helps bridge communication gaps between speakers of different languages through real-time translation.

Challenges in NLP:

1. **Ambiguity:** Words or sentences can have multiple meanings, making it hard for machines to interpret correctly.
2. **Context Understanding:** Grasping the broader context or intent behind words is complex for NLP systems.
3. **Sarcasm and Irony:** Difficult to detect subtle expressions like sarcasm or humor, leading to incorrect interpretations.

4. **Low-Resource Languages:** Limited data and resources for many languages make it harder to build NLP models for them.
5. **Data Privacy:** Processing sensitive personal data like emails or conversations raises privacy concerns.
6. **Idioms and Slang:** Unique cultural expressions are tough for machines to understand accurately.
7. **Real-Time Processing:** Ensuring efficient, accurate language understanding in real-time applications is challenging.

Artificial Neural Networks (ANNs)

Definition: Artificial Neural Networks (ANNs) are computational models inspired by the biological neural networks found in the human brain. They consist of interconnected groups of nodes (neurons) that work together to process input data and generate output. ANNs are particularly effective for tasks involving pattern recognition, classification, regression, and data modelling.

- **Neurons:** The fundamental building blocks of an ANN, analogous to biological neurons. Each neuron receives input, processes it, and produces an output.
- **Layers:**
 - **Input Layer:** The first layer that receives the input data.
 - **Hidden Layers:** Intermediate layers where neurons process inputs. ANNs can have one or more hidden layers, enabling them to learn complex patterns.
 - **Output Layer:** The final layer that produces the output of the network.
- **Weights and Biases:** Each connection between neurons has an associated weight, which determines the importance of that connection. Biases are additional parameters added to neurons to adjust the output.
- **Activation Functions:** Functions applied to the output of neurons to introduce non-linearity into the model. Common activation functions include:
 - **Sigmoid:** Outputs values between 0 and 1.
 - **Tanh (Hyperbolic Tangent):** Outputs values between -1 and 1.
 - **ReLU (Rectified Linear Unit):** Outputs the input directly if positive; otherwise, it outputs zero.

How ANNs Work:

1. **Forward Propagation:** Data is fed into the input layer, and it propagates through the network layer by layer. Each neuron calculates its output based on the inputs, weights, biases, and activation function.
2. **Loss Function:** The difference between the predicted output and the actual output (ground truth) is quantified using a loss function. Common loss functions include Mean Squared Error for regression tasks and Cross-Entropy Loss for classification tasks.
3. **Backpropagation:** The network adjusts its weights and biases based on the error computed by the loss function. This is done using an optimization algorithm (e.g., Stochastic Gradient Descent) to minimize the loss.
4. **Training:** The process of repeatedly feeding input data through the network and adjusting weights through backpropagation continues until the model converges, meaning it achieves satisfactory performance on the training data.

Types of Artificial Neural Networks:

1. **Feedforward Neural Networks:** Information moves in one direction—from input to output—without loops. Used for simple tasks like regression and classification.
2. **Convolutional Neural Networks (CNNs):** Designed specifically for processing grid-like data, such as images. They use convolutional layers to detect patterns and features, making them ideal for image recognition tasks.
3. **Recurrent Neural Networks (RNNs):** Designed for sequential data, RNNs have connections that loop back, allowing them to maintain memory of previous inputs. Commonly used for tasks like natural language processing and time-series forecasting.
4. **Long Short-Term Memory (LSTM) Networks:** A special type of RNN that effectively handles long-range dependencies by using memory cells and gating mechanisms to control the flow of information.
5. **Generative Adversarial Networks (GANs):** Consist of two networks (generator and discriminator) that compete against each other. The generator creates fake data, while the discriminator evaluates whether the data is real or fake. Used for image generation, style transfer, and more.

Applications of Artificial Neural Networks:

1. **Image Recognition:** Identifying objects, faces, or scenes in images (e.g., facial recognition systems).
2. **Natural Language Processing:** Tasks such as sentiment analysis, translation, and chatbots.
3. **Speech Recognition:** Converting spoken language into text and vice versa.
4. **Medical Diagnosis:** Analysing medical images and predicting diseases based on patient data.
5. **Financial Predictions:** Stock market forecasting and credit scoring.
6. **Game Playing:** Developing intelligent agents that can play complex games (e.g., AlphaGo).

Advantages of ANNs:

1. **Flexibility:** Capable of learning complex relationships and patterns from data.
2. **Generalization:** Can generalize learned patterns to unseen data, making them robust in various applications.
3. **Scalability:** Can handle large datasets and improve performance with more data.

Challenges of ANNs:

1. **Data Requirements:** Require large amounts of labelled data for effective training.
2. **Computational Resources:** Can be resource-intensive, requiring significant computational power and memory.
3. **Overfitting:** Risk of learning noise and outliers in the training data, which can lead to poor performance on new data.
4. **Interpretability:** Often considered "black boxes," making it difficult to understand the reasoning behind their predictions.

Syntactic Processing

Syntactic processing involves analysing the grammatical structure of sentences to ensure they conform to the rules of a specific language grammar. It determines how words are organized to form meaningful phrases or sentences. **Parsing Techniques:**

1. **Top-Down Parsing:**

- Starts with the root of a parse tree and expands it using grammar rules until the input sentence is matched.
- **Advantage:** Focuses directly on deriving the input sentence.
- **Example:** Starting from "S" (Sentence) and expanding to "NP + VP" (Noun Phrase + Verb Phrase).

2. **Bottom-Up Parsing:**

- Begins with the input words and attempts to combine them into larger components until the root of the parse tree is reached.
- **Advantage:** Ensures input is valid before constructing higher-level grammar.
- **Example:** Combining "the cat" into a Noun Phrase (NP).

Grammar Types:

1. **Context-Free Grammar (CFG):**

- A set of production rules where the left-hand side of each rule contains a single non-terminal symbol.
- **Example:** $S \rightarrow NP VP$ (Sentence $\rightarrow$ Noun Phrase + Verb Phrase).

2. **Dependency Grammar:**

- Focuses on the relationships (dependencies) between words in a sentence.
- **Example:** In "The cat sleeps," "sleeps" is the root, and "cat" depends on "sleeps."

Semantic Analysis

Semantic analysis aims to understand the **meaning** of words, phrases, and sentences. It ensures that the derived meaning is contextually accurate. **Techniques:**

1. **Semantic Roles:** Assign roles to sentence components based on their function (e.g., agent, action, object). **Example:** In "John ate an apple," "John" is the agent, "ate" is the action, and "apple" is the object.
2. **Word Sense Disambiguation (WSD):** Determines the correct meaning of a word based on context. **Example:** For "bank," WSD decides whether it refers to a "riverbank" or a "financial institution" depending on surrounding words.

Representations:

1. **Semantic Networks:** Graph-based structures representing relationships between concepts. **Example:** "Cat $\rightarrow$ isA $\rightarrow$ Animal."
2. **Knowledge Graphs:** A more advanced representation connecting entities and their relationships. **Example:** "Barack Obama $\rightarrow$ wasPresidentOf $\rightarrow$ USA."

Discourse and Pragmatic Processing

Discourse: Focuses on understanding the relationships between sentences or phrases in a text. **Key Tasks:**

- **Coherence:** Ensuring logical flow between sentences. **Example:** Linking "John bought a car. He drove it home" (linking "He" to "John").
- **Coreference Resolution:** Identifying when two expressions refer to the same entity. **Example:** "The dog barked. It was hungry" (linking "It" to "The dog").

Pragmatic Processing: Interprets meaning based on the **context** and **user intent**, going beyond the literal meaning. **Example:** Sentence: "Can you pass the salt?" **Pragmatic meaning:** A request, not a literal question about ability. Includes understanding sarcasm, politeness, and implied meanings.

Representing and Using Domain Knowledge

1. **Knowledge Representation:** Techniques for organizing and structuring domain-specific knowledge to be used by expert systems:
 - **Rules:** Represented as IF-THEN statements for decision-making. **Example:** IF temperature > 100°F THEN alert = "High Fever."
 - **Frames:** Structured data representations using attributes and values. **Example:** Frame for "Patient" might include attributes like "Name," "Age," and "Symptoms."
 - **Semantic Networks:** Graph-based structures linking concepts through relationships. **Example:** "Flu → causes → Fever."
1. **Rule-Based Systems:**
 - Decision-making systems based on predefined rules.
 - Rules are triggered by specific conditions, leading to a conclusion or action.
 - **Example:** A medical system diagnosing a disease based on symptoms.

Expert System Shells

A software framework for building expert systems without starting from scratch. Provides a generic structure that developers customize for specific domains. **Components:**

- **Knowledge Base:** Stores facts and rules about the domain. **Example:** Disease symptoms and treatments in a medical expert system.
- **Inference Engine:** Applies logical reasoning to the knowledge base to derive conclusions. **Example:** Matches patient symptoms to rules to suggest a diagnosis.
- **User Interface:** Allows users to interact with the system, input data, and view results. **Example:** A graphical interface for entering symptoms and receiving diagnoses.

Knowledge Acquisition

1. **Techniques:** Methods for gathering domain-specific knowledge to populate the knowledge base:
 - **From Domain Experts:** Interviewing or observing experts to extract their knowledge. **Example:** Consulting doctors for medical knowledge.
 - **From Databases:** Using existing data sources to identify patterns and relationships. **Example:** Analyzing hospital records for diagnostic rules.
2. **Tools:** Automated tools for extracting and organizing knowledge:
 - **Machine Learning:** Algorithms learn patterns and rules from data. **Example:** Training a model on patient data to predict diseases.
 - **Data Mining:** Techniques to discover patterns and relationships in large datasets. **Example:** Identifying correlations between symptoms and diseases.